
Large Language Models (LLM) for Range Prediction of Electric Trucks in Pre-Development Software and Electronics

Masterthesis No. 916/25

Editor: Ilyass Afkir

| 2901372

Supervisors: Tom Lehmann M.Sc.

Dang Chuong M.Sc.

| Daimler Truck AG



TECHNISCHE
UNIVERSITÄT
DARMSTADT



Ilyass Afkir
Matriculation No.: 2901372
Study program: Maschinenbau

Masterthesis No. 916/25
Topic: Large Language Models (LLM) for Range Prediction of Electric Trucks in Pre-Development
Software and Electronics

Submitted: March 12, 2026

Technische Universität Darmstadt
Fachgebiet Fahrzeugtechnik
Prof. Dr.-Ing. Steven Peters
Otto-Berndt-Straße 2
64287 Darmstadt



Externe Masterthesis Nr. 916/25 im Studiengang Maschinenbau (PO 2021) (30 CP)

von Ilyass Afkir

Vertiefung: Allgemeiner Maschinenbau

Diese Thesis liegt im Themenbereich Aerospace Engineering und wird auf Englisch verfasst.

Voraussichtlicher Beginn: 20.06.2025
Bearbeitungsdauer: 24/Wochen Monate

Thema: Reichweitenprädiktion von elektrischen LKW mit Large Language Models in der Vorentwicklungsphase von Software und Elektronik

Topic: *Large Language Models (LLM) for Range Prediction of Electric Trucks in Pre-Development Software and Electronics*

Fachgebiet Fahrzeugtechnik



Prof. Dr.-Ing. Steven Peters

Otto-Berndt-Straße 2
64287 Darmstadt

Bearbeiter:
Tom Lehmann M. Sc.
☎ +49 6151 16 – 24208

tom.lehmann@tu-darmstadt.de
www.fahrzeugtechnik-darmstadt.de

Datum
17.06.2025

Am Fachgebiet Fahrzeugtechnik der TU Darmstadt (FZD) wird in Zusammenarbeit mit der Industrie an der Verbrauchsprädiktion und Energieeffizienz von elektrischen Fahrzeugen geforscht.

Die präzise Vorhersage der Reichweite von elektrischen Nutzfahrzeugen ist ein zentrales Element für das Energiemanagement, die Einsatzplanung und das Vertrauen in Elektromobilitätslösungen. Insbesondere bei elektrischen Lkw führen stark variierende Nutzlasten, Streckenprofile und Energiebedarfe zu hohen Anforderungen an bestehende Modelle. Aktuell werden maßgeschneiderte Machine-Learning-Modelle eingesetzt, die auf großen Mengen hochaufgelöster Fahrdaten trainiert werden. Diese Modelle zeigen bereits eine hohe Genauigkeit, sind jedoch in der Skalierung begrenzt, da sie aufwendig gepflegt und individuell angepasst werden müssen. Um den durch eine große Vielfalt an Fahrzeugen, Fahrprofilen und Anwendungsbereichen entstehenden Herausforderungen bei der Skalierbarkeit bestehender Modelle zu begegnen, sollen selbstlernende, skalierbare Systeme zur Reichweitenvorhersage entwickelt werden.

In dieser Arbeit soll untersucht werden, ob sich Large Language Models (LLMs) – angepasst an strukturierte Zeitreihendaten – zur Erweiterung bestehender Prognosepipelines eignen. Mögliche Anwendungen reichen von der automatisierten Szenarienklassifikation über die Generierung synthetischer Fahrdaten bis hin zur Klassifikation von Fahrverhalten. Ziel ist es, das Potenzial dieser neuen Modellklasse im Kontext realer Herausforderungen der Fahrzeugentwicklung auszuloten und zur Skalierbarkeit datengetriebener Vorhersagemodelle beizutragen.

Als Ergebnisse der Arbeit sind nachzuweisen:

1. Einarbeitung in den Stand der Technik zur Reichweitenprognose elektrischer Nutzfahrzeuge, insbesondere unter Berücksichtigung datenbasierter Verfahren, bestehender ML-Ansätze und Herausforderungen im OEM-Kontext (z. B. Datenqualität, Skalierbarkeit, Modellwartung).

Seite: 1/2



2. Untersuchung der Grundlagen und Einsatzmöglichkeiten von Large Language Models für strukturierte Zeitreihendaten, inkl. Erforderlicher Vorverarbeitung und potenzieller Modellarchitekturen.
3. Begründete Auswahl und Implementation geeigneter Datenrepräsentationen für die Verarbeitung von Zeitreihendaten im gegebenen Kontext.
4. Entwicklung und Umsetzung eines Versuchsaufbaus zur Anwendung von LLMs in Aufgaben wie Szenarienklassifikation, synthetische Datengenerierung oder Fahrverhaltensmuster-Erkennung.
5. Definition geeigneter Evaluationsmetriken sowie Durchführung geeigneter Experimente oder Simulationen zur Bewertung der Modellleistung.
6. Analyse, Interpretation und Bewertung der Ergebnisse, insbesondere hinsichtlich Güte, Robustheit und Skalierbarkeit.

Schwerpunkte der Bewertung:

- Methodik des Vorgehens
- Vollständigkeit
- Nachvollziehbarkeit und Belastbarkeit der Argumentation
- Qualität folgender abzuliefernder Ergebnisse:
 - Schriftliche Ausarbeitung
 - Quellcode
 - Code-Dokumentation
- Abschlusskolloquium

Die Abgabe sämtlicher Messdaten und des Quellcodes wird vorausgesetzt. Die für die Arbeit gültigen Bewertungskriterien sind der Aufgabenstellung angehängt.

Steven Peters
2025.06.17
20:18:21 +02'00'

Prof. Dr.-Ing. Steven Peters

Tom
Lehmann

Digitally signed by Tom
Lehmann
Date: 2025.06.17
11:32:11 +02'00'

Tom Lehmann M. Sc.
(Betreuer FZD)

Sworn Declaration

Erklärung zur Abschlussarbeit gemäß § 22 Abs. 7 APB TU Darmstadt

Hiermit versichere ich, Ilyass Afkir, die vorliegende Arbeit gemäß § 22 Abs. 7 APB TU Darmstadt ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt zu haben. Ich habe mit Ausnahme der zitierten Literatur und anderer in der Arbeit genannter Quellen keine fremden Hilfsmittel benutzt. Die von mir bei der Anfertigung dieser wissenschaftlichen Arbeit wörtlich oder inhaltlich benutzte Literatur und alle anderen Quellen habe ich im Text deutlich gekennzeichnet und gesondert aufgeführt. Dies gilt auch für Quellen oder Hilfsmittel aus dem Internet.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§38 Abs.2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

English Translation for information purposes only:

Thesis Statement pursuant to § 22 paragraph 7 of APB TU Darmstadt

I herewith formally declare that I, Ilyass Afkir, have written the submitted thesis independently pursuant to § 22 paragraph 7 of APB TU Darmstadt without any outside support and using only the quoted literature and other sources. I did not use any outside support except for the quoted literature and other sources mentioned in the paper. I have clearly marked and separately listed in the text the literature used literally or in terms of content and all other sources I used for the preparation of this academic work. This also applies to sources or aids from the Internet.

This thesis has not been handed in or published before in the same or similar form.

I am aware, that in case of an attempt at deception based on plagiarism (§38 Abs. 2 APB), the thesis would be graded with 5,0 and counted as one failed examination attempt. The thesis may only be repeated once.

Datum / Date:

Unterschrift / Signature:

11.03.2026

A. Uyars

Abstract

Accurate range prediction for electric trucks is crucial for energy management, operational planning, and driver confidence in commercial electromobility. As electric truck fleets grow in size and variant diversity, the scalability of existing bespoke machine learning models becomes a critical limitation, as each model must be individually maintained per vehicle variant. Motivated by the rapid progress of foundation models in natural language processing and computer vision, this thesis investigates whether Large Language Models can be adapted to multivariate time series data to provide a scalable and unified classification architecture for electric truck applications.

To this end, this thesis introduces DeepRange, a novel Large Language Model-based architecture for multivariate time series classification, evaluated on real-world electric truck scenario classification across three domains: highway type, ambient temperature, and weather condition. Two candidate architectures with distinct data representation strategies are first compared on neutral multivariate time series classification benchmarks before being applied to the electric truck domain, ensuring that model selection remains free from domain-specific assumptions. DeepRange is subsequently proposed in two backbone configurations of 1 billion and 8 billion parameters and evaluated against the established baselines InceptionTime and MiniRocket across 12 experimental settings, covering two sequence lengths of 100 seconds and 500 seconds and two data regimes, a full training set and a reduced training set of 10 percent of the full data.

The results demonstrate that DeepRange constitutes a viable and scalable alternative to bespoke per-variant machine learning models for electric truck scenario classification. Under sufficient data availability, DeepRange 8B outperforms both baselines, while DeepRange 1B provides the most favorable trade-off between classification performance and computational cost. The identified data thresholds, task-dependent performance patterns, and the architectural design of DeepRange provide a concrete foundation for integrating Large Language Model-based multivariate time series classification into electric truck range prediction pipelines.

Contents

Abstract	I
Table of Contents	II
List of Symbols and Indices	IV
List of Abbreviations	VI
List of Figures	VII
List of Tables	IX
1 Introduction.....	1
1.1 Ascertainment of Assignment	2
1.2 Methodology	3
2 Electric Truck Range Prediction.....	5
2.1 Overview of Range Determinants.....	5
2.2 Deep Learning	8
2.2.1 Feedforward Neural Networks	9
2.2.2 Convolutional Neural Networks.....	10
2.2.3 Recurrent Neural Networks.....	11
2.2.4 Large Language Models	12
2.3 Multivariate Time Series Classification.....	15
2.3.1 Models.....	16
2.3.2 Evaluation Metrics	17
3 Large Language Models for Time Series Analysis	20
3.1 Background	20
3.2 Time Series Embeddings	22
3.3 Model Architectures	22
3.3.1 Patch Embedding Layer	23
3.3.2 Patch Reprogramming Layer	23
3.3.3 Patch-based Temporal Convolution Encoder	24
4 Definition of Requirements	26
5 Methodology and Concept	30
5.1 Multi-Criteria Decision Analysis.....	30
5.1.1 Mathematical Framework	31
5.1.2 Definition of Evaluation Criteria	32
5.1.3 Selection of Candidate Architectures	33
5.1.4 Evaluation and Analysis	34
5.2 Introducing DeepRange	36
6 Implementation of Scenario Classification.....	38
6.1 Electric Truck Data	38
6.2 Data Labeling	39

6.2.1	Data Verification	39
6.2.2	Highway Labeling	40
6.2.3	Weather and Ambient Temperature Labeling	40
6.3	Data Preparation	42
7	Evaluation	43
7.1	Highway Classification	43
7.2	Ambient Temperature Classification	48
7.3	Weather classification.....	52
7.4	Analysis of robustness and scalability	56
8	Conclusion and Outlook	58
Annex		60

List of Symbols and Indices

Latin formula symbols:

Symbol	Unit	Description
Acc	—	Accuracy
c	—	Confusion matrix entry
f	Hz	Frequency
F_1	—	F_1 score
F_β	—	F_β score
FP	—	False positive
I	A	Electric current
$\mathcal{L}$	—	Loss
MCC	—	Matthews correlation coefficient
P	W	Power
Pr	—	Precision
Q	A h	Electric charge
R	—	Recall
SOC	%	State of charge
t	s	Time
TN	—	True negative
TP	—	True positive
FN	—	False negative

Greek formula symbols:

Symbol	Unit	Description
μ	—	Mean
σ	—	Standard deviation
α	—	Learning Rate
η	—	Efficiency

Indices:

Symbol	Description
bal	Balanced
mac	Macro
$mech$	Mechanical
mic	Micro
nom	Nominal
p	Patch



Symbol	Description
<i>sig</i>	Signal
<i>tot</i>	Total
<i>train</i>	Training
<i>val</i>	Validation
<i>wgt</i>	Weighted

List of Abbreviations

AI	Artificial Intelligence
CAN	Controller Area Network
CNN	Convolutional Neural Network
CRISPMQ	Cross-Industry Standard Process for Machine Learning with Quality
DNN	Deep Neural Network
ET	Electric Truck
EV	Electric Vehicle
FNN	Feedforward Neural Network
FZD	Fahrzeugtechnik Darmstadt
GPS	Global Positioning System
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
LayerNorm	Layer Normalization
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MCDA	Multiple-criteria decision analysis
ML	Machine Learning
MTSC	Multivariate Time Series Classification
NLP	Natural Language Processing
OEM	Original Equipment Manufacturer
PINN	Physics-Informed Neural Network
ReLU	Rectified Linear Unit
RevIN	Reversible Instance Normalization
RNN	Recurrent Neural Network
RoPE	Rotary Position Embedding
SoC	State of Charge
SoH	State of Health
TSC	Time Series Classification
TUDa	Technische Universität Darmstadt
UEA	University of East Anglia

List of Figures

Figure 1-1:	Methodology of the procedure.	3
Figure 2-1:	GPT-2 model architecture	13
Figure 2-2:	Overview of Masked Multi-Head Attention	14
Figure 2-3:	Inception network	16
Figure 2-4:	Inception module with a bottleneck layer	17
Figure 3-1:	Patching process of a univariate time series	21
Figure 3-2:	OneFitsAll model architecture	23
Figure 3-3:	Time-LLM model architecture.	24
Figure 3-4:	LLMFew model architecture	25
Figure 5-1:	Developed methodology for the model selection problem. ^{97b}	31
Figure 5-2:	Proposed classification head	33
Figure 5-3:	Critical difference diagram of LLM architectures on UEA benchmarks	34
Figure 5-4:	DeepRange V1 model architecture	36
Figure 5-5:	DeepRange V2 model architecture	36
Figure 5-6:	Critical difference diagram of with DeepRange	37
Figure 6-1:	Electric truck trip from Sweden to Denmark.....	38
Figure 6-2:	Highway labels for an electric truck trip	41
Figure 7-1:	Per-class recall for full-shot highway classification (100 s)	44
Figure 7-2:	Learning curves for full-shot highway classification with sequence length 100 s.....	44
Figure 7-3:	DeepRange 1B/8B validation curves for highway classification (100 s)	45
Figure 7-4:	DeepRange 1B/8B loss and learning rate for few-shot highway classification (100 s)	46
Figure 7-5:	DeepRange 1B/8B validation curves for few-shot highway classification (100 s)	46
Figure A-1:	Confusion matrices for highway classification (100 s)	68
Figure A-2:	Confusion matrices for few-shot highway classification (100 s)	71
Figure A-3:	Confusion matrices for highway classification (500 s)	73
Figure A-4:	Confusion matrices for few-shot highway classification (500 s)	75
Figure A-5:	Confusion matrices for ambient temperature classification (100 s)	77
Figure A-6:	Learning curves for ambient temperature classification (100 s)	79
Figure A-7:	Confusion matrices for few-shot ambient temperature classification (100 s)	80
Figure A-8:	Learning curves for few-shot ambient temperature classification (100 s)	82
Figure A-9:	Confusion matrices for ambient temperature classification (500 s)	83
Figure A-10:	Learning curves for ambient temperature classification (500 s)	85
Figure A-11:	Confusion matrices for few-shot ambient temperature classification (500 s)	86
Figure A-12:	Learning curves for few-shot ambient temperature classification (500 s)	88
Figure A-13:	Confusion matrices for weather classification (100 s)	89
Figure A-14:	Learning curves for weather classification (100 s)	91
Figure A-15:	Confusion matrices for few-shot weather classification (100 s)	92
Figure A-16:	Learning curves for few-shot weather classification (100 s)	94
Figure A-17:	Confusion matrices for weather classification (500 s)	95
Figure A-18:	Learning curves for weather classification (500 s)	97
Figure A-19:	Confusion matrices for few-shot weather classification (500 s)	98



Figure A-20: Learning curves for few-shot weather classification (500 s)	100
--	-----

List of Tables

Table 2-1:	Relevant activation functions.....	10
Table 2-2:	Types of time series	15
Table 4-1:	Requirements for extracting electric truck trips	27
Table 5-1:	Categories of decision problems	30
Table 5-2:	MCDA Evaluation Criteria	32
Table 5-3:	Classification performance on the UEA datasets	34
Table 5-4:	Training time and averaged model size on the UEA datasets	35
Table 5-5:	Decision matrix and final results	35
Table 5-6:	DeepRange V1/V2 performance on University of East Anglia (UEA) datasets	37
Table 5-7:	Model size and training time on the UEA datasets	37
Table 6-1:	Descriptive statistics for all electric truck trips	39
Table 6-2:	Labeling features per scenario classification task	39
Table 7-1:	Highway classification results (100 s)	43
Table 7-2:	Few-shot highway classification results (100 s)	45
Table 7-3:	Highway classification results for input sequence length 500 s.....	47
Table 7-4:	Few-shot highway classification results for input sequence length 500 s	48
Table 7-5:	Full-shot air temperature classification results for input sequence length 100 s	49
Table 7-6:	Few-shot air temperature classification results for input sequence length 100 s	50
Table 7-7:	Ambient temperature classification results for input sequence length 500 s	51
Table 7-8:	Few-shot air temperature classification results for input sequence length 500 s	52
Table 7-9:	Weather classification results (100 s)	53
Table 7-10:	Few-shot weather classification results (100 s)	54
Table 7-11:	Weather classification results (500 s)	55
Table 7-12:	Few-shot weather classification results (500 s).....	56
Table A-1:	Overview of UEA datasets	60
Table A-2:	Defined model hyperparameters for each UEA dataset	61
Table A-3:	OneFitsAll results on normalized UEA datasets	62
Table A-4:	Time-LLM results on normalized UEA datasets	63
Table A-5:	LLM-Few results on normalized UEA datasets	64
Table A-6:	Deep Range V1 results on normalized UEA datasets	65
Table A-7:	Deep Range V2 results on normalized UEA datasets	66
Table A-8:	Electric Truck (ET) data features.....	67
Table A-9:	Per-class metrics for highway classification (100 s).....	70
Table A-10:	Per-class metrics for few-shot highway classification (100 s)	72
Table A-11:	Per-class metrics for highway classification (500 s).....	74
Table A-12:	Per-class metrics for few-shot highway classification (500 s)	76
Table A-13:	Per-class metrics for ambient temperature classification (100 s)	78
Table A-14:	Per-class metrics for few-shot ambient temperature classification (100 s).....	81
Table A-15:	Per-class metrics for ambient temperature classification, (500 s).....	84
Table A-16:	Per-class metrics for few-shot ambient temperature classification, (500 s)	87
Table A-17:	Per-class metrics for weather classification (100 s)	90

Table A-18: Per-class metrics for few-shot weather classification (100 s)	93
Table A-19: Per-class metrics for weather classification (500 s)	96
Table A-20: Per-class metrics for few-shot weather classification (500 s)	99

1 Introduction

Climate change has significantly impacted the automotive industry and reshaped the political landscape. The European Union has set binding targets to achieve climate neutrality by 2050 and reduce net greenhouse gas emissions by at least 55% by 2030 compared to 1990 levels.¹ These goals align with the Paris Agreement's aim to limit global warming to well below 2 °C and pursue efforts to limit it to 1.5 °C above pre-industrial levels.² The transport sector is a major contributor to global CO₂ emissions, with road freight transport being a growing source.³ Emissions from this segment are projected to increase by 56% to 70% between 2015 and 2050.³ Achieving emission reduction targets requires decarbonizing heavy-duty road transport, where electrification of commercial vehicles plays a key role.

The department of Fahrzeugtechnik Darmstadt (FZD) at Technische Universität Darmstadt collaborates with industry and academia on research projects to improve range and energy efficiency in electric commercial vehicles. For example, the MINDED project, funded by the European Union's Horizon Europe research and innovation program, develops advanced thermal and energy management systems to increase the driving range of electric minibuses in cold climates.⁴ A further research cooperation with Daimler Truck AG, Automated Trucking Darmstadt, focuses on driver assistance and automation technologies to enhance energy efficiency and operational reliability in heavy-duty trucks.⁵

The growing electrification of commercial vehicles has intensified research efforts focused on accurately estimating remaining battery range under real-world operating conditions. In this work, commercial vehicles refer to ETs, which are used for commercial transport and are classified as N3 under the United Nations Economic Commission for Europe (UNECE) standards.⁶ This corresponds to vehicle weight classes 7 and 8 as defined by the United States Department of Energy (DOE).⁷ Examples of such ETs include the Mercedes-Benz eActros^{8,9} 300, 400, and 600.

Within this context, electric trucks present distinct challenges due to their higher energy demands and varied duty cycles. This master's thesis, conducted in collaboration with Daimler Truck AG, contributes to ongoing research by extending range prediction models for electric trucks using Large Language Model (LLM) architectures.

Accurate range prediction for electric commercial vehicles is essential for energy management, fleet operations, and building user confidence in electromobility. In particular electric trucks unlike passenger electric vehicles, face increased complexity due to variable payloads, diverse route profiles, and fluctuating energy demands. These conditions introduce significant variability that challenges the robustness and generalizability of existing prediction models. Current approaches often use Machine Learning (ML)

¹ EU–Council: European Climate Law (2021). art. 4(1); art. 2(1).

² UNFCCC: Paris Agreement (2016). art. 2(a).

³ Liimatainen, H. et al.: The potential of electric trucks (2019). p. 1.

⁴ FZD: MINDED (2025).

⁵ FZD: Automated Trucking Darmstadt (2025).

⁶ EAFO: EU classification of vehicle types (2025).

⁷ DOE: Vehicle Weight Classes and Categories (2012).

⁸ Smodej, P.; Burkhart, U.: 3000 kilometers with eActros 300 (2023).

⁹ Smodej, P. et al.: eActros 400 and 600 (2025).

models trained on high-resolution driving data. Although these methods can achieve high accuracy, they require manual calibration and maintenance, which limits their scalability. To overcome these limitations, scalable and self-learning range prediction systems are needed. Such systems must adapt to a wide range of vehicles, operating conditions, and use cases enabling a shift toward a unified, generalizable *one-model-fits-all* solution.

LLMs, while originally designed for tasks in Natural Language Processing (NLP), have gained increasing attention as general-purpose foundation models. They have been successfully extended to domains such as computer vision, audio processing, and multimodal learning.¹⁰ Recent studies have expanded the application of LLMs to time-series data, demonstrating their potential beyond traditional textual tasks in fields such as climate science, Internet of Things, healthcare, traffic, and finance.¹⁰ For example, Jin et al. propose a reprogramming framework to tailor general-purpose models for time-series forecasting, while Liu et al. employ cross-modal alignment techniques to integrate temporal and contextual information for the same task.^{11,12}

Building on previous research, this thesis examines the potential of LLMs for time-series data from electric trucks to improve range prediction. While applications such as State of Charge (SoC) estimation are well established in both the literature and industrial practice at Daimler Truck AG, the use of LLMs for driving scenario classification remains largely unexplored.¹³ Time-series classification in this context has received little attention, despite its potential to provide essential contextual information that can enhance the accuracy of range prediction models.

1.1 Ascertainment of Assignment

The main research question of this thesis is the application of LLMs to structured time-series data from electric trucks, with a focus on driving scenario classification. The objective is to assess whether LLMs architectures originally developed for NLP can be adapted to recognize and classify driving contexts such as urban streets, highways, and intersections, as well as environmental conditions including snow, rain, and other weather phenomena, using multivariate Controller Area Network (CAN) signals collected during vehicle operation.

To address this research question, the input data will be processed to meet the requirements of LLM architectures, including normalization, segmentation, and tokenization of time-series signals. Several LLMs configurations will be evaluated for their ability to capture temporal structures from vehicular data. While additional applications such as SoC estimation and forecasting, driver behavior profiling, and synthetic data generation are acknowledged, the primary focus remains on driving scenario classification. Evaluation will include comparisons with conventional ML models, robustness analysis under varying driving conditions, and testing on additional benchmark datasets beyond the truck data. The overall aim is to assess the feasibility of using LLMs for scalable and transferable context recognition that supports the range prediction pipeline.

¹⁰ Zhang, X. et al.: Large Language Models for Time Series (2024). p. 1.

¹¹ Jin, M. et al.: Reprogramming Large Language Models (2024).

¹² Liu, X. et al.: UniTime (2024).

¹³ Dang, C. et al.: Robust Electric Truck Range Estimation (2025).

1.2 Methodology

The developed methodology, shown in Fig. 1-1, outlines the approach used to accomplish the tasks of this master thesis. It consists of five main task areas including Artificial Intelligence (AI) Research, Requirements, Methodology, Implementation and Evaluation.

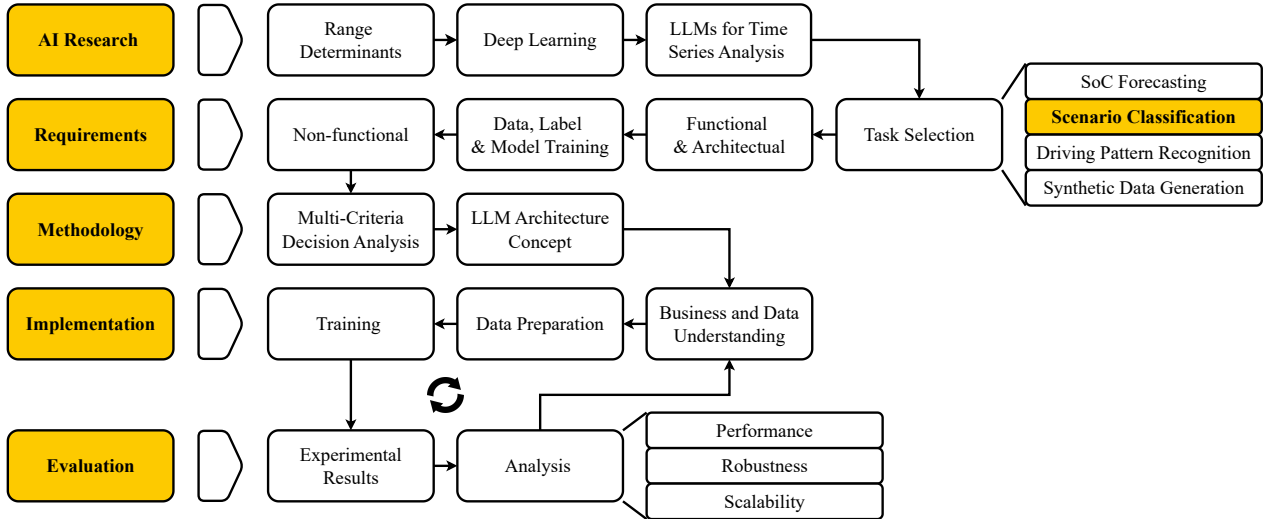


Figure 1-1: Methodology of the procedure.

The first task area, AI Research, begins with a comprehensive literature review focused specifically on the topic of range prediction for electric trucks. This includes an overview of traditional methods and a transition toward data-driven and ML-based approaches. This review is essential to understand the context in which LLMs can be applied. Following this, the foundations of LLMs are explained, as they form the conceptual basis for the entire master thesis. A solid understanding of these models is necessary to explore how they can be adapted for time-series data.

The second task area, Requirements, defines the necessary conditions for fine-tuning LLMs on time-series data in the context of electric truck range prediction. These requirements are categorized into technical, methodological, and analytical dimensions. At the technical level, sufficient computational resources must be available to support both training and inference. This includes access to GPU hardware suitable for model fine-tuning and evaluation, as well as compatible software environments covering machine learning frameworks, model libraries, runtime dependencies, and efficient handling of time-series data in suitable formats. At the methodological level, the system must support structured time-series inputs, along with appropriate preprocessing and tokenization strategies that align with transformer-based architectures. This also includes preparing the data in a form that preserves temporal order and multivariate structure, ensuring compatibility with the LLM input pipeline. The fine-tuning process must accommodate different training strategies, such as full fine-tuning or parameter-efficient methods, depending on resource constraints. At the analytical level, clear evaluation metrics must be defined for assessing model performance on time-series prediction tasks. The system must ensure reproducibility by maintaining consistent data splits, version control, and logging mechanisms. It must also support the tracking of training efficiency and model behavior to maintain transparency and traceability throughout the development process.

The third task area, Concept, first evaluates different approaches and techniques for adapting time-series data to LLMs. An evaluation matrix based on the requirements defined in the second block will be used to select the final concept. This block also depends on the chosen task or use case, which will guide the focus and scope of the implementation in the next phase.

The fourth task area, Implementation, begins with the selection of driving scenario classification as the target task. Based on the selected task, the implementation process follows the principles of the Cross-Industry Standard Process for Machine Learning with Quality (CRISPMQ)¹⁴ framework, which includes iterative steps such as business and data understanding, data preparation, modeling, training, and evaluation. The evaluation phase itself is treated as a separate block. After each evaluation cycle, the process returns to the business and data understanding phase, following a cyclical refinement approach until the model achieves satisfactory performance. This block concludes with the transition toward deployment. However, due to time constraints, full deployment is not covered within the scope of this master thesis, though its role is acknowledged in the overall process.

The fifth task area, Evaluation, starts by defining appropriate metrics based on the task chosen in the Implementation block. Following the experiments and simulations, results are analyzed, interpreted, and assessed. A deeper evaluation is then conducted focusing on model quality, robustness, and scalability. Once this evaluation is complete, the process loops back to the business and data understanding phase, enabling iterative refinement until the desired performance and reliability criteria are met.

¹⁴ Studer, S. et al.: Towards CRISP-ML(Q) (2021), pp. 392-413.

2 Electric Truck Range Prediction

This chapter presents the state of the art in range prediction for ETs. The first section introduces the key factors influencing driving range, followed by a review of current data-driven methods. Scenario-based evaluations are also discussed, providing a broader context for range estimation approaches. The focus then shifts to deep learning, with a brief introduction to main architectures and relevant models. In the following section, time-series classification and evaluation metrics are introduced, as these are central to scenario-based prediction tasks. Finally, the challenges faced by original equipment manufacturers (OEMs) in implementing these approaches are addressed. Throughout this chapter the mathematical notation adheres to the conventions established in Goodfellow et al.¹⁵, unless defined otherwise.

2.1 Overview of Range Determinants

Electrifying road freight transport is a key strategy to reduce greenhouse gas emissions and mitigate climate change. According to Qiu et al., Class 8 ETs consume three to six times less energy than equivalent diesel trucks, indicating higher energy efficiency under the investigated driving cycles.¹⁶ Range prediction of Electric Vehicles (EVs) and, by extension, of ETs, is contingent on vehicle design, driver behavior, and environmental conditions.^{17a} The goal of range prediction is to estimate the remaining driving distance of an ET based on the residual energy available in the traction battery at a given time.¹⁸

Vehicle design is rooted in physical relations, as exemplified by Newton's second law of motion.^{17a} For instance, a low aerodynamic drag coefficient reduces air resistance and increases vehicle range.¹⁹ The design objectives of ETs should align with safety requirements for electrical systems, as specified in Society of Automotive Engineers (SAE) Standard J2910_201404²⁰ and International Organization for Standardization (ISO) Standard 6469-1:2019-04²¹. While many aspects of vehicle design, such as mass, auxiliary systems, and the frontal area, influence range^{22a}, the traction battery is the primary determinant of available energy for vehicle operation and receives specialized focus in research and development by Original Equipment Manufacturers (OEMs)²³. The functional state of a battery is typically characterized by the SoC, denoted as *SOC*, which quantifies the energy available at a given time.^{22b} It is generally defined as the ratio of the current battery capacity $Q_{\text{bat}}(t)$ to the manufacturer-specified nominal capacity $Q_{\text{bat,nom}}$.²⁴ The current capacity, based on the charge or discharge current $I_{\text{bat}}(t)$, is given by

$$Q_{\text{bat}}(t) = \int_0^t I_{\text{bat}}(\tau) d\tau, \quad (2-1)$$

¹⁵ Goodfellow, I. et al.: Deep Learning (2016).

¹⁶ Qiu, Y. et al.: Medium- and Heavy-Duty EVs (2023), p. 1 f.

¹⁷ Varga, B. O. et al.: Prediction of EV Range (2019), a: p. 3 ff; b: p. 6 f.

¹⁸ Smuts, M. et al.: Factors influencing remaining driving range (2017), p. 196.

¹⁹ Mruzek, M. et al.: Parameters Influencing EV Range (2016), p. 172.

²⁰ SAE International: J2910_201404 - Design and Test of ETs (2014).

²¹ ISO: 6469-1:2019-04 - Electrically propelled road vehicles (2019).

²² Smuts, M. et al.: Factors influencing remaining driving range (2017). a: p. 198, tab. 1; b: p. 200; c: p. 199.

²³ Smodej, P. et al.: Mercedes-Benz Battery Technology Center (2024).

²⁴ Chang, W.-Y.: Battery SoC Estimating Methods (2013), p. 1 f.

and the SoC reads²⁴

$$SOC(t) = \frac{Q_{bat}(t)}{Q_{bat,nom}}. \quad (2-2)$$

In addition, the state of health (SoH), denoted as SOH , is used to assess electrochemical degradation, which diminishes both charge and discharge capacity as well as the total available energy.^{17b} It can be defined similarly to Equ. 2-2, using the current maximum capacity $Q_{bat,max}$ as the numerator²⁵

$$SOH = \frac{Q_{bat,max}}{Q_{bat,nom}}, \quad (2-3)$$

Although both metrics are relevant, ET range prediction commonly addresses either the estimation of the SoC for a known travel distance or the prediction of the remaining driving range for a given SoC.^{22b} Consequently, SoC estimation can indirectly provide insights into the SoH of a battery. For example, an experienced driver on a familiar route, may notice that the estimated SoC may be higher than usual, indicating potential battery degradation.

Current SoC estimation algorithms can be classified into physics-based, data-driven, and hybrid methods. Physics-based methods include direct measurement techniques such as impedance spectroscopy, which rely on battery quantities like voltage and impedance, and book-keeping methods such as Coulomb counting, which integrate the discharge current over time to obtain SOC .²⁴ Data-driven methods generally rely on AI, specifically ML algorithms, which are self-designing and can automatically adjust SOC under different discharge conditions.²⁴ An introduction to ML, particularly deep learning, is provided in Chap. 2.2. Hybrid methods combine physics-based approaches, which may be limited in adapting to variations in driving behavior or environmental conditions, with data-driven models, which can capture these variations effectively but often lack interpretability as well as robustness under unseen or extreme conditions.^{26a} Dang et al. presents a framework for ET SoC estimation that balances physical constraints and empirical data through a time-varying parameter, emphasizing physics early in training and gradually shifting toward data-driven refinement.^{26b} The proposed physical model captures energy consumption by integrating the electrical power

$$P_{el} = \begin{cases} \frac{P_{mech}}{\eta}, & P_{mech} \geq 0, \\ \eta P_{mech}, & P_{mech} < 0, \end{cases} \quad (2-4)$$

over time, where P_{mech} is the mechanical power demand determined from resistive forces, vehicle speed, and acceleration.^{26c} The efficiency η accounts for electrical, mechanical, and thermal losses and differs between motor operation and regenerative braking.

Driver behavior, as a key determinant of range prediction, enables the construction of profiles for range estimation and alert generation.^{22c} Driver behavior varies across individuals and can be classified into conservative, moderate, and aggressive driving styles, with the latter leading to increased energy use and reduced accuracy of remaining range estimation.^{22c} Dollinger et al. mentions that an aggressive

²⁵ Berecibar, M. et al.: Review of SoH estimation methods (2016), p. 576.

²⁶ Dang, C. et al.: Robust Electric Truck Range Estimation (2025). a: p. 2396; b: pp. 2396-2401; c: p.2398.

driving style can reduce ET range by 17 %.^{27a} and that the Further influencing factors include traffic flow, traffic regulations, vehicle design, and environmental conditions.^{22c} Behavioral indicators include throttle position, brake pressure, and vehicle speed, with throttle position exhibiting the largest variation between conservative, moderate, and aggressive drivers.^{22c} ETs are operated across a wide spectrum of duty profiles, including regional delivery, drayage, and long-haul applications. Within these operating contexts, driver-specific control strategies can induce substantial variability in energy demand and overall vehicle efficiency. Practically, driver behavior can be predicted using data collected from ETs during trips. The data is largely unlabeled, hence not directly useful. Data-driven approaches from unsupervised machine learning can be applied to identify behavioral patterns, for example by using clustering algorithms such as K-means. Given a dataset $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^n$ and a set of cluster centers $A = \{a_1, \dots, a_c\}$, each data point is assigned to the nearest cluster with

$$z_{ik} = \begin{cases} 1 & \text{if } k = \arg \min_j \|\mathbf{x}_i - a_j\|^2, \\ 0 & \text{otherwise.} \end{cases} \quad (2-5)$$

The objective is to minimize the sum of squared distances and reads

$$J = \sum_{i=1}^n \sum_{k=1}^c z_{ik} \|\mathbf{x}_i - a_k\|^2. \quad (2-6)$$

Cluster centers are updated as

$$a_k = \frac{\sum_{i=1}^n z_{ik} \mathbf{x}_i}{\sum_{i=1}^n z_{ik}}. \quad (2-7)$$

The algorithm iterates between the assignment step Equ. 2-5 and the update step Equ. 2-7 until convergence. The resulting clusters can then be interpreted to derive driver behavior patterns and assign labels.

Environmental conditions, as the third key factor influencing range prediction, encompass weather, road surface characteristics, and road types. For instance, elevated battery temperatures, whether due to high ambient temperatures or internal heat generation, have been shown to accelerate anodic side reactions, resulting in increased capacity degradation and reduced lifespan.²⁸ Moderate ambient temperatures yield the lowest energy consumption for ETs, while both cold and hot conditions increase consumption due to the energy demands of battery thermal management and cabin heating or cooling.²⁹ Heavy precipitation and higher wind speeds both exhibit clear positive correlations with energy consumption.²⁹ A plausible interpretation is that precipitation increases rolling resistance²⁹ while wind increases aerodynamic drag forces acting on the large surface area of trucks and trailers. Furthermore, the greatest driving range for ETs is observed on summer trips over flat terrain with passive driving style, whereas the lowest range occurs on hilly terrain with aggressive driving in winter.^{27b} Substantial differences in energy consumption also exist across road types, being highest in urban traffic and lower on rural and motorway routes.³⁰ With the increasing deployment of ETs, large volumes of driving data are collected continuously. These data

²⁷ Dollinger, M.; Fischerauer, G.: Range Prediction for ETs (2021). a: p. 14; b: p. 11 ff.

²⁸ Bamdezh, M. et al.: Climate condition effects on EV performance (2025), p. 11 f.

²⁹ Carlsson, R.; Nordgren, E.: Masterthesis, Weather Impact on ET Energy Consumption (2024), p. 40 f.

³⁰ Corguill , J. L. et al.: Analysis of battery electric trucks (2025), p. 7 f.

only become valuable when engineers can efficiently identify, access, and retrieve relevant subsets for specific analyses. For instance, optimizing an ET for low ambient temperatures to increase range, requires access to data representing those conditions. This further enables identifying underrepresented scenarios and planning targeted data collection trips.

Scenario classification addresses this by assigning labels to trips or trip segments based on task-relevant environmental and operational factors. In the context of range prediction, scenario classification enables the grouping of driving data into comparable operational contexts, thereby allowing prediction models to explicitly account for systematic variations in energy consumption across different scenarios. According to Dollinger et al., a scenario spans a certain amount of time and represents the temporal development between several scenes, with each scene constituting a snapshot of the environment including static elements, dynamic components, actors, and observers.³¹ Scenario labels can be obtained from multiple sources including vehicle signals collected by the CAN-bus system, digital maps, and external data sources such as weather services. The main limitation of this approach lies in the availability of data with consistent and reliable scenario labels. Current approaches rely on manual scenario annotation performed either during test drives by a co-driver or during offline post-processing.³² This scales poorly with data volume and is limited by human attention, particularly when multiple events occur simultaneously.³² In addition, introduces inconsistencies across annotators and increases labeling cost and labor.³² Data-driven methods such as ML algorithms can automate labeling systems and process large datasets efficiently with consistent rules and reduced processing time.

2.2 Deep Learning

Modern ETs continuously record large volumes of data during operation through onboard sensors. This data, often referred to as *Big Data*, enables the use of data-driven methods such as ML, where algorithms solve tasks by learning from experience rather than relying on fixed, human-designed programs.^{32a} A fundamental subdomain of ML is supervised learning, wherein a model is trained on labeled samples $(\mathbf{x}, y)$, where $\mathbf{x} \in \mathbb{R}^n$ denotes the input feature vector and y the associated label. From a statistical perspective, supervised learning aims to infer the conditional distribution $p(y | \mathbf{x})$ ^{32a}, enabling the prediction of y for unseen inputs $\mathbf{x}$. Of particular relevance to the present work is scenario classification, where the goal is to learn a function $f : \mathbb{R}^n \rightarrow \{1, \dots, C\}$ ^{32a} that assigns each input $\mathbf{x}$ to one of C class labels. Analogously, SoC estimation constitutes a regression task, where the learned function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ produces a continuous numerical output^{32a} rather than a categorical assignment.

Deep learning extends supervised learning through the use of Deep Neural Networks (DNNs), which learn hierarchical representations^{32b} by composing multiple layers of computational units, referred to as neurons, across an input layer, one or more hidden layers, and an output layer. The output of a DNN is called logits. Mathematically, a DNN with L layers is defined as a composition of functions $f_\ell : \mathbb{R}^{n_\ell} \rightarrow \mathbb{R}^{n_{\ell+1}}$, $\ell \in \{1, \dots, L\}$, given by

$$f(\mathbf{x}; \boldsymbol{\theta}) = (f_L \circ f_{L-1} \circ \dots \circ f_1)(\mathbf{x}), \quad (2-8)$$

³¹ Gruner, R. et al.: Driving scenarios and classification (2017), p. 1783 f.

³² Goodfellow, I. et al.: Deep Learning (2016). a: pp. 96-105; b: p. 5 f; c: p. 272 f; d: p. 149 f.

where $f(\mathbf{x}; \boldsymbol{\theta}) \in \mathbb{R}^{n_{L+1}}$ and $\boldsymbol{\theta}$ denotes the set of learnable parameters.^{33a} The computation defined in Equ. 2-8 is also referred to as *forward propagation*. The objective of $f(\mathbf{x}; \boldsymbol{\theta})$ is to minimize the expected cost function J^* over the data-generating distribution p_{data} defined as

$$J^*(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{data}}} [\mathcal{L}(f(\mathbf{x}; \boldsymbol{\theta}), y)], \quad (2-9)$$

where $\mathcal{L}$ denotes the loss function.^{32c} In practice, the expected loss is minimized over a training dataset with m examples using its empirical distribution $\hat{p}_{\text{data}}$ ^{32c}

$$J(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{x}, y) \sim \hat{p}_{\text{data}}} [\mathcal{L}(f(\mathbf{x}; \boldsymbol{\theta}), y)] = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)}). \quad (2-10)$$

This procedure is called *empirical risk minimization*.^{32c} Minimizing J requires computing its gradient

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)}), \quad (2-11)$$

which scales as $\mathcal{O}(m)$ and becomes computationally expensive for large training datasets, despite their necessity for adequate generalization.^{32d} The stochastic gradient descent (SGD) algorithm reduces this cost by treating the gradient in Equ. 2-11 as an expectation, which can be approximated using a minibatch $\mathcal{B} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m')}\}$ of $m' \ll m$ examples drawn uniformly from the training dataset, yielding the gradient estimate^{32d}

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \approx \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)}). \quad (2-12)$$

The SGD algorithm then updates the parameters by stepping in the direction of the negative gradient

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \quad (2-13)$$

where η is the learning rate.^{32d} The gradient $\nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\mathbf{x}; \boldsymbol{\theta}), y)$ is computed using the *backpropagation* algorithm, which recursively applies the chain rule across the layers of the DNN.

2.2.1 Feedforward Neural Networks

Feedforward Neural Networks (FNNs), also referred to as multilayer perceptrons, constitute a foundational architecture in deep learning. Each layer $\ell \in \{1, \dots, L\}$ is fully connected, meaning every neuron receives input from all neurons in the previous layer. Information propagates strictly from the input through successive hidden layers to the output, with no feedback connections.^{34a} Furthermore, the parameters of each layer are independent of those of all other layers.^{33a} Each layer passes information from the previous layer by computing a new representation $\mathbf{h}^{(\ell+1)} \in \mathbb{R}^{n_{\ell+1}}$, where deeper layers capture increasingly abstract concepts.^{34b} Given the current representation $\mathbf{h}^{(\ell)} \in \mathbb{R}^{n_{\ell}}$, each layer applies a linear transformation

³³ Caterini, A. L.; Chang, D. E.: Deep Neural Networks (2018). a: p. 24; b: p. 36 f.

³⁴ Goodfellow, I. et al.: Deep Learning (2016). a: p. 164; b: p. 5; c: pp. 326-329; d: p. 335 f.

followed by an elementwise nonlinear activation function $\phi^{(\ell)}$

$$\mathbf{h}^{(\ell+1)} = f^{(\ell)}(\mathbf{h}^{(\ell)}; \mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}) = \phi^{(\ell)}(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell)} + \mathbf{b}^{(\ell)}), \quad (2-14)$$

where $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell+1} \times n_{\ell}}$ and $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_{\ell+1}}$ are the learnable weight matrix and bias vector of layer ℓ , respectively.^{33b} Relevant activation functions for this work are presented in Tab. 2-1 and include the Rectified Linear Unit (ReLU)³⁵, Leaky ReLU³⁶, Gaussian Error Linear Unit (GELU)³⁷, Sigmoid Linear Unit (SiLU)³⁸ and Softmax³⁹. The softmax function is particularly relevant for the output layer of a

Table 2-1: Relevant activation functions.

Name	Formula
Leaky ReLU	$\phi(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x \leq 0 \end{cases}$
ReLU	$\phi(x) = \max(0, x)$
GELU	$\phi(x) = \frac{x}{2} [1 + \text{erf}(x/\sqrt{2})]$
SiLU	$\phi(x) = \frac{x}{1 + e^{-x}}$
Softmax	$\phi(x_i) = \frac{e^{x_i}}{\sum_{j=1}^C e^{x_j}}$

classification model, which produces a vector of unnormalized scores, referred to as logits. For example, given the logits of a FNN $\mathbf{z} = \mathbf{h}^{(L+1)} \in \mathbb{R}^C$, softmax normalizes them into a valid probability distribution over C classes⁴⁰

$$p(C_k | \mathbf{z}) = \frac{e^{z_k}}{\sum_{j=1}^C e^{z_j}}, \quad (2-15)$$

where $k = 1, \dots, C$ and $\sum_{k=1}^C p(C_k | \mathbf{z}) = 1$. FNNs are not well suited for capturing temporal dependencies and do not inherently model relationships across time steps.⁴¹

2.2.2 Convolutional Neural Networks

According to Goodfellow et al.^{34c}: „Convolutional Neural Networks (CNNs) are a specialized kind of neural network for processing data that has a known grid-like topology. Examples include time-series data, which can be thought of as a 1-D grid taking samples at regular time intervals, and image data, which can be thought of as a 2-D grid of pixels.“

CNNs replace general matrix multiplication with the convolution operation in at least one of their layers.^{34c} The convolution operation is commutative and typically denoted with an asterisk.^{34c} In its

³⁵ Nair, V.; Hinton, G. E.: Rectified linear units (2010), p. 808 f.

³⁶ Maas, A. et al.: Rectifier Nonlinearities Improve Neural Network Acoustic Models (2013), p. 3 f, equ. 3.

³⁷ Hendrycks, D.; Gimpel, K.: Gaussian Error Linear Units (2016), p. 1 f.

³⁸ Elfving, S. et al.: Sigmoid-weighted linear units (2018), p. 4, equ. 9.

³⁹ Bridle, J.: Training Stochastic Model Recognition Algorithms (1989), pp. 211-217.

⁴⁰ Bishop, C. M.: Pattern recognition and machine learning (2006), p. 209.

⁴¹ Mohammadi Foumani, N. et al.: Deep Learning for Time Series (2024), p. 5.

general continuous form, it is defined as

$$s(t) = (x * w)(t) = \int x(a) w(t - a) da, \quad (2-16)$$

where x denotes the input, w the kernel, and the resulting output s is referred to as the feature map.^{34c} In practice, the discrete form of convolution is used, where the input is a multidimensional array of data and the kernel is a multidimensional array of parameters adjusted by the learning algorithm.^{34c} Many ML libraries implement cross-correlation but refer to it as convolution. Since the kernel is learned, this distinction is inconsequential.^{34c} For a 1D input x and kernel k , the discrete convolution operation reads^{34c}

$$s(i) = (k * x)(i) = \sum_a x(i + a) k(a), \quad (2-17)$$

and for a 2D input X and kernel K it is defined as^{34c}

$$S(i, j) = (K * X)(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n). \quad (2-18)$$

A convolutional layer is composed of three successive stages.^{34d} The layer first computes a set of linear activations through parallel convolutions, which are subsequently passed through a nonlinear activation function in the detector stage.^{34d} Finally, a pooling operation aggregates the activations spatially, reducing the dimensionality of the feature maps.^{34d} An example is max pooling, which retains the maximum activation within a local rectangular neighborhood.^{34d} The output of the last convolutional layer is typically flattened and passed to an FNN to perform the target task. CNNs have demonstrated strong performance on time series data, as exemplified by InceptionTime⁴², which is introduced in Chap. 2.3.1.

2.2.3 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) extend FNNs by introducing feedback connections, enabling the network to retain information across time steps.^{43a} While an FNN maps an input $\mathbf{x}$ to an output solely based on the current input, an RNN maintains a hidden state denoted as $\mathbf{h}^{(t)} \in \mathbb{R}^h$ where h is the number of hidden units, that accumulates information from all previous inputs ($\mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(1)}$), making them well-suited for sequential data such as time series.^{43a} At each time step t , an RNN computes a hidden state $\mathbf{h}^{(t)} \in \mathbb{R}^h$ from the current input $\mathbf{x}^{(t)}$ and the preceding hidden state $\mathbf{h}^{(t-1)}$

$$\mathbf{h}^{(t)} = \phi_{\mathbf{h}}(\mathbf{W}_{xh}\mathbf{x}^{(t)} + \mathbf{W}_{hh}\mathbf{h}^{(t-1)} + \mathbf{b}_h), \quad (2-19)$$

where $\mathbf{W}^{xh}$ is the input-to-hidden weight matrix, $\mathbf{W}^{hh}$ the hidden-to-hidden weight matrix, $\mathbf{b}_h$ the bias vector, and ϕ a nonlinear activation function.^{43a} The output state $\mathbf{o}^{(t)}$ is then given by^{43a}

$$\mathbf{o}^{(t)} = \phi_{\mathbf{o}}(\mathbf{h}^{(t)}\mathbf{W}_{xo} + \mathbf{b}_o). \quad (2-20)$$

⁴² Ismail Fawaz, H. et al.: AlexNet for time series classification (2020).

⁴³ Schmidt, R. M.: Introduction to RNNs (2019). a: p. 1 f; b: p. 3 f.

Since $\mathbf{h}^{(t)}$ recursively depends on $\mathbf{h}^{(t-1)}$, the hidden state implicitly encodes information from all preceding time steps.^{??a}

RNNs suffer from vanishing and exploding gradients because repeated matrix multiplications over long sequences can shrink gradients limiting the influence of earlier states or grow gradients destabilizing training.^{43b} Long Short-Term Memory Networks (LSTMs) were introduced to address the vanishing gradient problem, enabling better learning over long sequences and outperforming traditional RNNs on many tasks.^{43b}

2.2.4 Large Language Models

Language modeling is usually defined as an unsupervised estimation of the distribution over symbol sequences.⁴⁴ Given a sequence x of m symbols $(s_1, \dots, s_m)$, the joint probability is factorized as the product of conditional probabilities⁴⁴

$$p(x) = \prod_{i=1}^m p(s_i \mid s_1, \dots, s_{i-1}). \quad (2-21)$$

This enables tractable sampling and estimation of $p(x)$, as well as conditional probabilities of the form⁴⁴

$$p(s_{m-k}, \dots, s_m \mid s_1, \dots, s_{m-k-1}). \quad (2-22)$$

Important advances in computing these conditional probabilities have been achieved through self-attention architectures, most notably the Transformer.⁴⁴ Building on this, Radford et al. introduced the first version of Generative Pre-trained Transformer (GPT)⁴⁵ in 2018, followed by GPT-2⁴⁶ in 2019, which is widely regarded as the starting point of the LLM era. The GPT-2 architecture shown Fig. 2-1 will serve as the basis to explain LLM concepts.

Token Embedding

Traditionally, NLP divided text into sentences and space-separated words called tokens.^{48a} These tokens served as practical approximations of linguistic word-forms, even though true word boundaries are often ambiguous due to contractions, compounds, and related linguistic phenomena.^{48a} In modern NLP, particularly with neural language models, tokens are not full words but smaller subword units. This shift is mainly motivated by the need to maintain a fixed and limited vocabulary size while reducing sparsity and out-of-vocabulary issues.^{48a} Tokenization therefore refers to the task of segmenting text into these model-specific subword units rather than into space-separated words.^a LLMs typically use Byte Pair Encoding (BPE), a compression-based tokenization algorithm that iteratively merges the most frequent pairs of adjacent symbols to form subword units.^{48b} A key variant, Byte-Level BPE, operates directly on raw bytes instead of characters and is the algorithm used in GPT-2.^{48b} A vocabulary is constructed

⁴⁴ Radford, A. et al.: Language Models are Unsupervised Multitask Learners (2019), p. 2.

⁴⁵ Radford, A. et al.: Improving Language Understanding by Generative Pre-Training (2018).

⁴⁶ Radford, A. et al.: Language Models are Unsupervised Multitask Learners (2019).

⁴⁷ Raschka, S.: Build A Large Language Model (2024), p. 90, fig. 4.15, fig. 4.13.

⁴⁸ Mielke, S. J. et al.: Tokenization in NLP (2021). a: p. 2 f; b: p. 10.

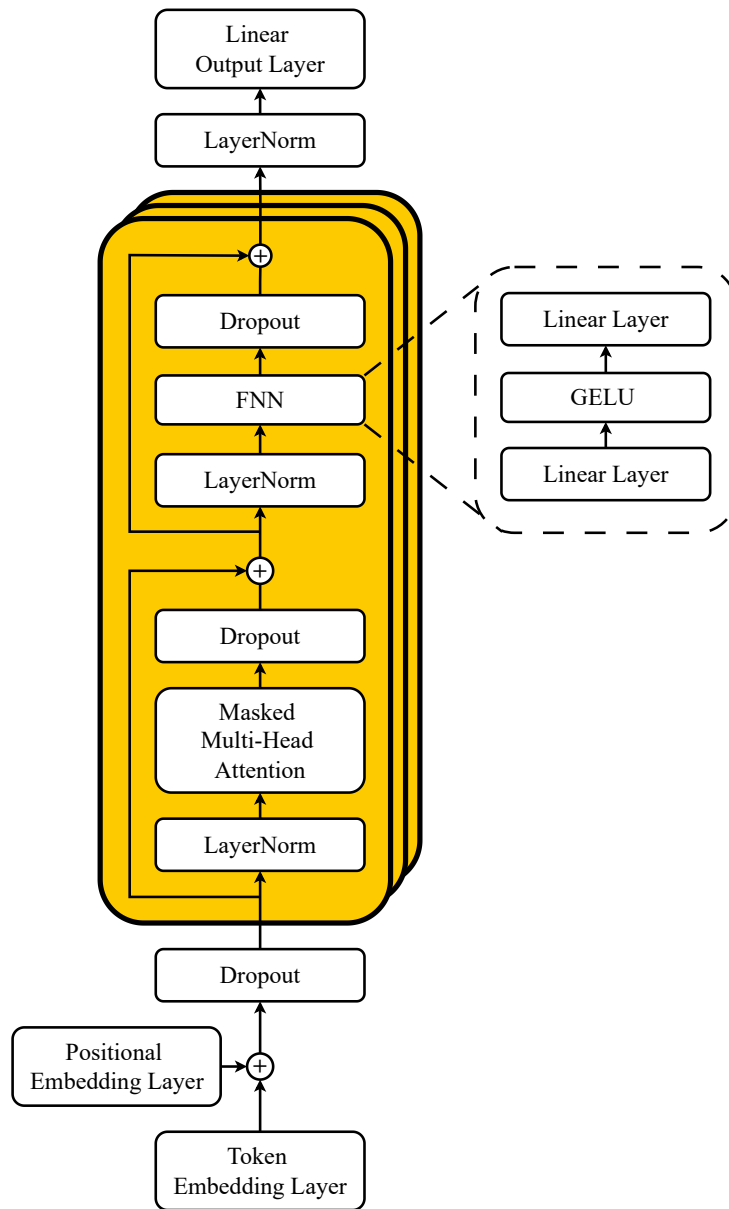


Figure 2-1: GPT-2 model architecture. The orange box represents a Transformer block.⁴⁷

by tokenizing a training corpus into individual tokens.^{49a} Each unique token is then mapped to a unique integer identifier, referred to as a token ID.^{49a} Since LLMs require numeric representations to process text, an embedding layer maps each token ID to a corresponding dense vector^{49a}, acting as a lookup table. While pre-trained embeddings such as Word2Vec⁵⁰ can be used, LLMs typically learn their own embeddings during training.^{49a} They are optimized jointly with the model parameters for the specific data and task.^{49a} The embedding dimension for GPT-2 corresponds to its hidden size d_{model} .

⁴⁹ Raschka, S.: Build A Large Language Model (2024). a: pp. 31-35; b: pp. 45-47.

⁵⁰ Mikolov, T. et al.: Word Representations in Vector Space (2013).

Positional Embedding

Since the self-attention mechanism is position-agnostic, information about the relative or absolute position of each token in a sequence must be injected into the input representation.^{49b} Two categories exist, absolute positional embeddings, which assign a unique embedding to each token position, and relative positional embeddings, which encode the distance between tokens, offering better generalization to variable-length sequences.^{49b} Positional embeddings share the same dimension as the token embeddings and are added to them element-wise.^{49b} GPT-2 uses absolute positional embeddings that are learned during training.⁵¹ In contrast, the original Transformer uses fixed sinusoidal positional encodings, where each position pos in a sequence is encoded as^{52a}

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad (2-23)$$

$$PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad (2-24)$$

where i denotes the dimension index.

Multi-Head Attention

Attention originates from the goal to improve RNNs for handling longer sequences.⁵³ An attention function maps a query and a set of key-value pairs to an output vector, computed as a weighted sum of the values. Each weight reflects the compatibility between the query and the corresponding key.^{52b} The process of *Masked Multi-Head Attention* is shown in Fig. 2-2

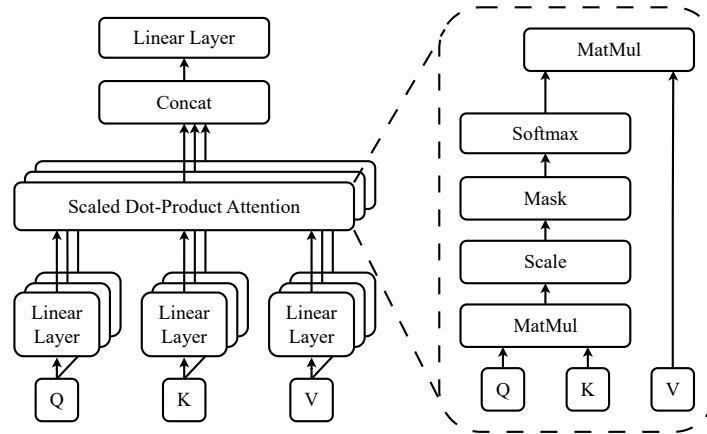


Figure 2-2: Overview of Masked Multi-Head Attention.⁵⁴

Let $Q \in \mathbb{R}^{n \times d_k}$ denote the queries, $K \in \mathbb{R}^{n \times d_k}$ the keys, and $V \in \mathbb{R}^{n \times d_v}$ the values. *Scaled dot-product attention* computes the dot products of the queries with all keys, scales them by $\frac{1}{\sqrt{d_k}}$, and applies a softmax to obtain the weights on the values.^{52b} The scaling counteracts the effect of large dot product magnitudes

⁵¹ Radford, A. et al.: Language Models are Unsupervised Multitask Learners (2019).

⁵² Vaswani, A. et al.: Attention is all you need (2017). a: p. 6; b: pp. 3-5.

⁵³ Bahdanau, D. et al.: Neural Machine Translation by Jointly Learning to Align and Translate (2016).

⁵⁴ Vaswani, A. et al.: Attention is all you need (2017), p. 4, fig. 2.

for high d_k , which would otherwise push the softmax into regions of extremely small gradients. The matrix of outputs is then computed as^{52b}

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \quad (2-25)$$

To capture information from multiple representation subspaces, the *multi-head attention* mechanism applies h parallel attention operations with learned linear projections.^{52b} For each head $i = 1, \dots, h$, let $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}, W^O \in \mathbb{R}^{h \cdot d_v \times d_{\text{model}}}$ denote the projection matrices.^{52b} Then each head is computed as^{52b}

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V), \quad (2-26)$$

and the final multi-head output is

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O. \quad (2-27)$$

2.3 Multivariate Time Series Classification

Let $\mathcal{T} \subseteq \mathbb{R}$ be an ordered index set and $N \in \mathbb{N}$ the number of observed features. A time series is defined as a function $\mathbf{x} : \mathcal{T} \rightarrow \mathbb{R}^N, t \mapsto \mathbf{x}^{(t)}$, where $\mathbf{x}^{(t)} \in \mathbb{R}^N$ denotes the observation at time point t .^{55a} Different types of time series are distinguished in Tab. 2-2, where $a = \min(\mathcal{T})$ and $b = \max(\mathcal{T})$.

Table 2-2: Types of time series.^{55b}

Property	Terminology
$N = 1$	Univariate time series
$N \geq 2$	Multivariate time series
$\mathcal{T} \subset [a, b] \subset \mathbb{R}$, finite	Discrete time
$\exists u \in \mathbb{R}_+$ s.t. $t_{j+1} - t_j = u$	Equidistant time
$\mathcal{T} = [a, b], \mathbb{R}_+$ or $\mathbb{R}$	Continuous time

ET data is primarily collected from the CAN-bus system at fixed frequencies, and thus constitutes a *multivariate, discrete, and equidistant time series*, thereafter referred to simply as *multivariate time series*.

ML algorithms are trained with batches of multivariate time series data is represented as $\mathbf{X} \in \mathbb{R}^{B \times T \times N}$, where B denotes the batch size, T the sequence length, and N the number of features, often referred to as channels in the time series literature.

⁵⁵ Beran, J.: Mathematical Foundations of Time Series (2018). a: p. 1; b: p. 2, tab. 1.1.

2.3.1 Models

Two established Multivariate Time Series Classification (MTSC) models are introduced here which include InceptionTime^{58a} from the deep learning domain, ROCKET⁵⁶ and MiniRocket⁵⁷ as a kernel-based model, and more recent approaches using LLMs. extracting features from randomly generated kernels,

InceptionTime

InceptionTime employs an ensemble of five randomly initialized Inception networks.^{58a} As illustrated in Fig. 2-3, each Inception network consists of two residual blocks, each comprising three Inception modules.^{58a} The residual connections between blocks serve to mitigate the vanishing gradient problem^{58a} introduced in Chap. 2.2.3.

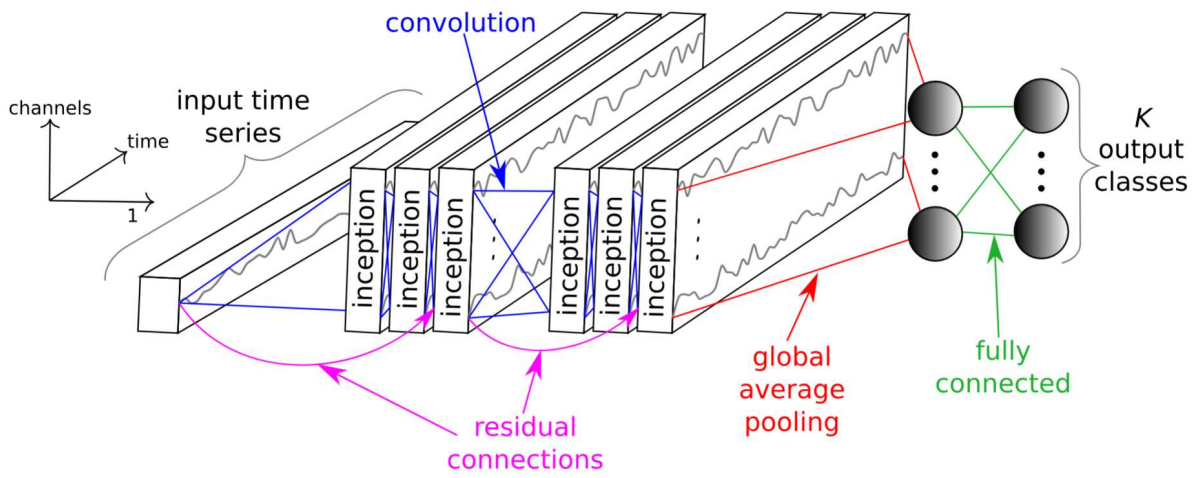


Figure 2-3: Inception network.^{58b}

After the residual blocks, a global average pooling layer aggregates the temporal dimension, after which a fully connected classification head maps the resulting representation to the C output classes.^{58a} Each Inception module consists of three parallel components as shown Fig. 2-4. A bottleneck layer applies m filters of length 1 to reduce the input dimensionality from N to $m \ll N$ channels, reducing model complexity and mitigating overfitting.^{58a} Next, three parallel convolutions with different filter lengths are applied to the bottleneck output, capturing temporal patterns at multiple scales.^{58a} Last, a parallel max pooling operation followed by a bottleneck layer provides invariance to local perturbations.^{58a} The outputs of all parallel branches are concatenated to form the module output.^{58a}

⁵⁶ Dempster, A. et al.: TSC using random convolutional kernels (2020), pp. 1454-1495.

⁵⁷ Dempster, A. et al.: Fast Transform for TSC (2021), pp. 248-257.

⁵⁸ Ismail Fawaz, H. et al.: AlexNet for time series classification (2020). a: p. 1941 f; b: p. 1941, fig. 1; c: -.

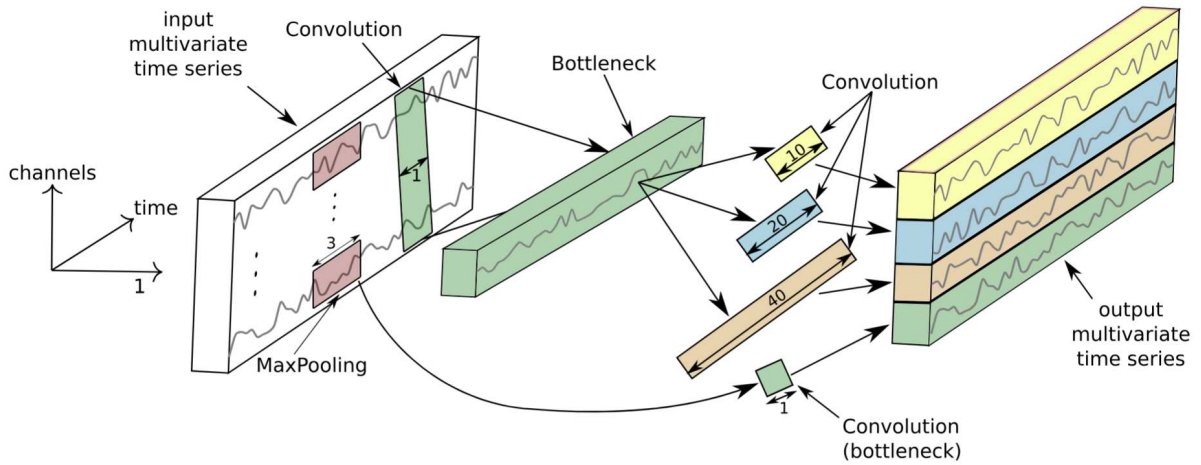


Figure 2-4: Inception module with a bottleneck layer.⁵⁹

ROCKET

ROCKET transforms time series by applying a large number of randomly generated convolutional kernels with random length, weights, bias, dilation, and padding.^{60a} A kernel is a vector of weights with an associated bias term.^{60b} In contrast to InceptionTime or generally the convolutional neural networks described in Chap. 2.2.2, these kernel parameters are not learned but randomly initialized, hence very large number of kernels is used.^{60a} Dilation spreads the kernel over the input at regular intervals.^{60b} Dilation is sampled per kernel, allowing patterns to be captured across multiple scales. Two features are extracted from each kernel: the maximum value and the proportion of positive values which is the most important factor for accuracy.

Padding appends values, typically zero, to the start and end of the input such that the central kernel weight aligns with the first element at the start of the convolution operation.^{60b} The resulting features are not used to train the kernels themselves but instead serve as input to a simple linear classifier. The only hyperparameter is the number of kernels k , and computational complexity scales linearly with k . A more efficient version MiniRocket will be used in this work.

2.3.2 Evaluation Metrics

The performance of a trained classifier is evaluated and visualized using a confusion matrix. Let $n_{i,j}$ be the entry of a $C \times C$ confusion matrix for a C -class classification problem, representing the number of samples from class i predicted as class j , for $i, j = 1, \dots, C$.⁶¹ By convention, rows correspond to true classes and columns to predicted classes. The total number of samples belonging to class i is

$$n_i = \sum_{j=1}^C n_{i,j}, \quad (2-28)$$

⁵⁹ Ismail Fawaz, H. et al.: AlexNet for time series classification (2020), p. 1942, fig. 2.

⁶⁰ Dempster, A. et al.: TSC using random convolutional kernels (2020). a: p. 1460 f; b: p. 1458.

⁶¹ Rainio, O. et al.: Evaluation metrics for machine learning (2024), p. 3.

and the total number of all class samples is

$$N = \sum_{i=1}^C n_i = \sum_{i=1}^C \sum_{j=1}^C n_{i,j}. \quad (2-29)$$

Given a C -class classification problem with $C \geq 3$, the true positive count for class i corresponds to the diagonal entry of the confusion matrix, representing samples of class i that are correctly classified⁶². The confusion matrix entries are used to define four fundamental counts for each class i . Let TP_i , FN_i , FP_i , and TN_i denote the true positives, false negatives, false positives, and true negatives for class i , respectively, defined as

$$TP_i = n_{i,i}, \quad (2-30)$$

$$FN_i = \sum_{j \neq i} n_{i,j}, \quad (2-31)$$

$$FP_i = \sum_{j \neq i} n_{j,i}, \quad (2-32)$$

$$TN_i = \sum_{\substack{j \neq i \\ h \neq i}} n_{j,h}. \quad (2-33)$$

Using these definitions, the per-class accuracy, precision, recall, and F1-score are

$$ACC_i = \frac{TP_i + TN_i}{TP_i + TN_i + FP_i + FN_i}, \quad (2-34)$$

$$Pr_i = \frac{TP_i}{TP_i + FP_i}, \quad (2-35)$$

$$R_i = \frac{TP_i}{TP_i + FN_i}, \quad (2-36)$$

$$F_{1,i} = 2 \cdot \frac{Pr_i \cdot R_i}{Pr_i + R_i}. \quad (2-37)$$

Micro-averaged metrics are given as⁶³

$$ACC = P_{\text{mic}} = R_{\text{mic}} = F_{1,\text{mic}}, \quad (2-38)$$

$$ACC_{\text{bal}} = R_{\text{mac}}. \quad (2-39)$$

⁶² Rainio, O. et al.: Evaluation metrics for machine learning (2024), p. 3.

⁶³ Grandini, M. et al.: Metrics for Multi-Class Classification (2020).

Macro-averaging computes the metric independently for each class and then takes the unweighted mean across classes⁶³

$$P_{\text{mac}} = \frac{1}{C} \sum_{i=1}^C P_i, \quad (2-40)$$

$$R_{\text{mac}} = \frac{1}{C} \sum_{i=1}^C R_i, \quad (2-41)$$

$$F_{1,\text{mac}} = \frac{1}{C} \sum_{i=1}^C F_{1,i}. \quad (2-42)$$

As noted above, in single-label multiclass classification macro-averaged recall equals balanced accuracy, i.e. $Acc_{\text{bal}} = R_{\text{mac}}$. Weighted-averaging computes the metric for each class and averages them using class sample counts as weights⁶³

$$P_{\text{wgt}} = \frac{1}{N} \sum_{i=1}^C n_i P_i, \quad (2-43)$$

$$R_{\text{wgt}} = \frac{1}{N} \sum_{i=1}^C n_i R_i, \quad (2-44)$$

$$F_{1,\text{wgt}} = \frac{1}{N} \sum_{i=1}^C n_i F_{1,i}. \quad (2-45)$$

The Matthews Correlation Coefficient metric introduced by Brian W. Matthews⁶⁴ is „a contingency matrix method of calculating the Pearson product moment correlation coefficient between actual and predicted values and is defines as⁶⁵

$$MCC = \frac{\sum_k TP_k \cdot TN_k - FP_k \cdot FN_k}{\sqrt{(TP_k + FP_k)(TP_k + FN_k)(TN_k + FP_k)(TN_k + FN_k)}} \quad (2-46)$$

⁶⁴ Matthews, B. W.: Comparison of T4 phage lysozyme (1975).

⁶⁵ Chicco, D.; Jurman, G.: Advantages of Matthews correlation coefficient (2020).

3 Large Language Models for Time Series Analysis

This chapter presents the fundamentals and applications of LLMs for multivariate time series data, focusing on preprocessing and model architectures. While current research emphasizes time series forecasting, the main goal in any task is to learn representations. These representations support other tasks such as classification, anomaly detection, and regression. The initial stage constructs data embeddings that captures temporal patterns, forming the basis for downstream tasks. The field is progressing rapidly, making it impractical to cover every recent development. Instead, this chapter highlights foundational approaches and key architectural component.

3.1 Background

Originally developed for NLP tasks, transformer-based architectures have been extended to other data modalities by modeling non-text inputs as token sequences. For example, Dosovitskiy et al. introduced the Vision Transformer, where images are divided into fixed-size patches that are embedded and processed in the same manner as textual tokens.⁶⁶ Similarly, Borsos et al. proposed AudioLM, a framework for audio generation that combines discrete semantic and acoustic tokens in a hierarchical structure.⁶⁷ In the time series domain, transformer-based architectures have been predominantly applied to forecasting. Notable models include Informer⁶⁸, Autoformer⁶⁹, and FEDformer⁷⁰.^{71a} However, the findings of Zeng et al.⁷² challenge this trajectory, as simple linear models can outperform them on established benchmarks.^{71a} This can be attributed to the use of point-wise attention, which processes each time step independently and fails to capture the local semantic information inherent to temporal segments.^{71b} Motivated by these limitations, Nie et al.^{71c} propose two central design principles to improve transformer-based architectures for multivariate time series forecasting. First, a channel-independent modeling strategy is employed, whereby each feature of a multivariate time series is treated as an independent univariate series, with all features sharing the same transformer backbone while their forward passes remain independent.^{71b} Given a multivariate time series $\mathbf{X} \in \mathbb{R}^{T \times N}$, each univariate series corresponding to feature $i \in \{1, \dots, N\}$ is denoted as $\mathbf{x}^{(i)} \in \mathbb{R}^T$. Next, each univariate time series is partitioned into a sequence of patches, which may be overlapping or non-overlapping^{71b} as shown in Fig. 3-1. Let T_{pat} denote the patch length and S the stride between two consecutive patches, which in Fig. 3-1 are set equal in length. The patching process produces a sequence of N_{pat} patches $\mathbf{X}_{\text{pat}}^{(i)} \in \mathbb{R}^{T_{\text{pat}} \times N_{\text{pat}}}$, where^{71c}

$$N_{\text{pat}} = \left(\frac{T - T_{\text{pat}}}{S} \right) + 2. \quad (3-1)$$

Before patching, $\mathbf{x}^{(i)}$ is padded by repeating the last observed value $x_T^{(i)} \in \mathbb{R}$ exactly S times at the end of the sequence.^{71b}

⁶⁶ Dosovitskiy, A. et al.: Transformers for Image Recognition (2021).

⁶⁷ Borsos, Z. et al.: Language Modeling Approach to Audio Generation (2023), pp. 2523-2533.

⁶⁸ Zhou, H. et al.: Transformer for Long Sequence Time-Series (2021), pp. 11106-11115.

⁶⁹ Wu, H. et al.: Decomposition Transformers with Auto-Correlation (2021), pp. 22419-22430.

⁷⁰ Zhou, T. et al.: Frequency Enhanced Transformer for Time Series (2022), pp. 27268-27286.

⁷¹ Nie, Y. et al.: Long-term Forecasting with Transformers (2023). a: p. 1 f; b: p. 3 f; c: -.

⁷² Zeng, A. et al.: Are Transformers Effective for Time Series? (2023), pp. 11121-11128.

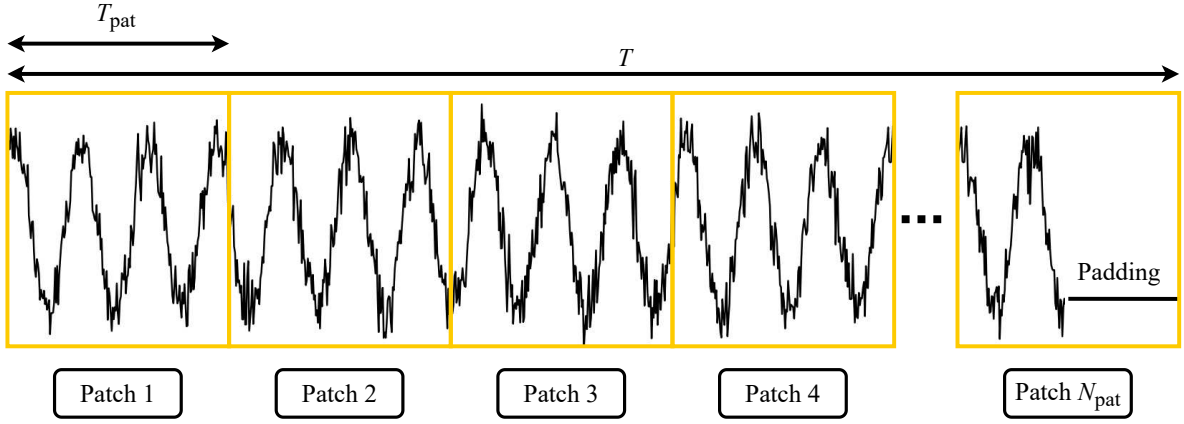


Figure 3-1: Patching process of a univariate time series.

The idea of patching is to capture local semantic information by aggregating consecutive time steps into subseries-level patches, since a single time step lacks the semantic meaning necessary to capture meaningful temporal dependencies.^{71a} Patching reduces the number of input tokens from T to approximately T/S .^{71c} Since the memory usage and computational complexity of self-attention scale quadratically with the number of tokens, this reduces both by approximately a factor of S^2 .^{71b} Consequently, for fixed training time and memory constraints, patching allows a model to process longer sequences lengths, which can improve forecasting performance.^{71b}

A third optional design choice addresses distribution shift, whereby the statistical properties of the time series, such as mean and variance, vary over time, causing discrepancies between training and test data distributions.^{73a} To mitigate this, Reversible Instance Normalization (RevIN) normalizes each input sequence before the transformer backbone and symmetrically denormalizes the output, preserving the original distributional information.^{73a} Given the μ and σ of $\mathbf{x}^{(i)}$ computed over $t \in 1, \dots, T$, the normalized input reads^{73b}

$$\hat{x}_t^{(i)} = \gamma \left(\frac{x_t^{(i)} - \mathbb{E}[x^{(i)}]}{\sigma[x^{(i)}] + \epsilon} \right) + \beta, \quad (3-2)$$

where γ, β are learnable affine parameters and $\epsilon > 0$ ensures numerical stability. The denormalized output is then obtained by reversing this transformation^{73c}

$$\hat{y}^{(i)} = \sqrt{\sigma^2[x^{(i)}] + \epsilon} \left(\frac{\tilde{y}^{(i)} - \beta}{\gamma} \right) + \mathbb{E}[x^{(i)}], \quad (3-3)$$

where $\tilde{y}^{(i)}$ is the normalized model output and $\hat{y}^{(i)}$ the final prediction in the original data scale. While RevIN has been shown to improve performance in time series forecasting^{73d}, it does not appear to provide benefits for time series classification⁷⁴ tasks. Additionally, transformers have been investigated in a self-supervised context for learning general time series representations, with the objective of developing foundation models^{71b} analogous to LLMs in NLP. Although attempts have been made for example by

⁷³ Kim, T. et al.: Reversible Instance Normalization (2022). a: p. 1; b: p. 4, equ. 2; c: p. 5, equ. 3; d: p. 6 ff.

⁷⁴ Chen, X. et al.: Learning Time Series Classification (2025), p.3 f.

Zerveas et al.⁷⁵, the potential for universal time series representation learning remains not fully realized.^{71b} Traditional ML models e.g. DNN trained solely on time series data are constrained by limited dataset scale and diversity, and lack the reasoning and interpretability capabilities of LLMs. Motivated by their remarkable generalization capacity, zero-shot learning, and inherent reasoning mechanisms, recent research has explored adapting LLMs for time series analysis. Two paradigms have emerged embedding-visible adaptation, where time series are aligned with the LLM latent space at the embedding level, and text-visible adaptation, where time series are transformed into textual representations.⁷² This work focuses on embedding-visible adaptation, which has garnered substantial research attention and demonstrates considerable promise for practical applications.

3.2 Time Series Embeddings

A time series embedding is obtained by mapping the temporal or feature dimensions of a time series sequence into a higher-dimensional representation space.^{76a} This representation space corresponds to the hidden size d_{LLM} of the LLM. Analogous to token embeddings in NLP, time series embeddings aim to represent temporal data as dense vector representations that capture relevant temporal dependencies, reduce dimensionality, and learn transferable features that generalize across heterogeneous downstream tasks.⁷⁷ Relevant embedding strategies are value, patch, and positional embeddings. Given a multivariate time series $\mathbf{X} \in \mathbb{R}^{T \times N}$, a value embedding projects the feature dimension N into the hidden size d_{LLM} . Two approaches are commonly employed. A token-based convolutional embedding applies Equ. 2-17 along the temporal dimension, generalized to N input channels and d_{LLM} output channels, yielding $\mathbf{E}_{\text{conv}} \in \mathbb{R}^{T \times d_{\text{LLM}}}$. A simpler alternative is a linear projection applied independently at each time step $t \in 1, \dots, T$. Each input feature vector $\mathbf{x}^{(t)} \in \mathbb{R}^N$ is mapped to an embedding vector $\mathbf{e}^{(t)} \in \mathbb{R}^{d_{\text{LLM}}}$ with a single-layer FNN, giving

$$\mathbf{E}_{\text{lin}} = f(\mathbf{X}; \mathbf{W}, \mathbf{b}) = \mathbf{X}\mathbf{W}^T + \mathbf{b}, \quad (3-4)$$

where $\mathbf{W} \in \mathbb{R}^{d_{\text{LLM}} \times N}$, $\mathbf{b} \in \mathbb{R}^{d_{\text{LLM}}}$, and $\mathbf{E}_{\text{lin}} \in \mathbb{R}^{T \times d_{\text{LLM}}}$. In contrast to both value embedding methods, a patch embedding creates patches of a multivariate time series resulting in $\mathbf{X}_{\text{pat}} \in \mathbb{R}^{N_{\text{pat}} \times N_{\text{pat}} \times L_{\text{pat}}}$. Each patch is then linearly projected into d_{LLM} using Equ. ??, where $\mathbf{W} \in \mathbb{R}^{T_{\text{pat}} \times d_{\text{LLM}}}$, $\mathbf{b} \in \mathbb{R}^{d_{\text{LLM}}}$, yielding $\mathbf{E}_{\text{pat}} \in \mathbb{R}^{N_{\text{pat}} \times N_{\text{pat}} \times d_{\text{LLM}}}$. Positional embedding injects information about the position of each time step in the sequence, which is not inherently modeled by components such as attention or FNNs. Unlike value embeddings, positional embeddings depend solely on the position index $t \in \{1, \dots, T\}$ and not on the input content. This process applies Equ. 2-23 where $d_{\text{model}} = d_{\text{LLM}}$, $pos = t$ yielding $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{N \times d_{\text{LLM}}}$.

3.3 Model Architectures

The candidate model architectures presented in this section employ GPT-2 as the backbone LLM, as introduced in Chap. 2.2.4. The channel-independent modeling strategy introduced in Chap. 3.1 is assumed throughout, such that $\mathbf{x}^{(i)} \in \mathbb{R}^T$ denotes the univariate series of feature $i \in \{1, \dots, N\}$ of a multivariate dataset.

⁷⁵ Zerveas, G. et al.: Transformer for Time Series Representation (2021), pp. 2114-2124.

⁷⁶ Nematirad, R. et al.: Data Embeddings in Time Series Forecasting (2025). a: -.

⁷⁷ Irani, H. et al.: Time Series Embeddings for Classification (2025), p. 1 f.

3.3.1 Patch Embedding Layer

Zhou et al. introduced *One Fits All*, a model for time series classification, anomaly detection, imputation, and forecasting.^{78a} As illustrated in Fig. 3-2, each univariate series $\mathbf{x}^{(i)}$ is first normalized with RevIN, followed by a learnable patch embedding layer that projects the time series into the hidden size d_{model} .^{79b} The backbone remains largely frozen during training, with only the positional embedding and Layer Normalization (LayerNorm) layers fine-tuned.^{79b}

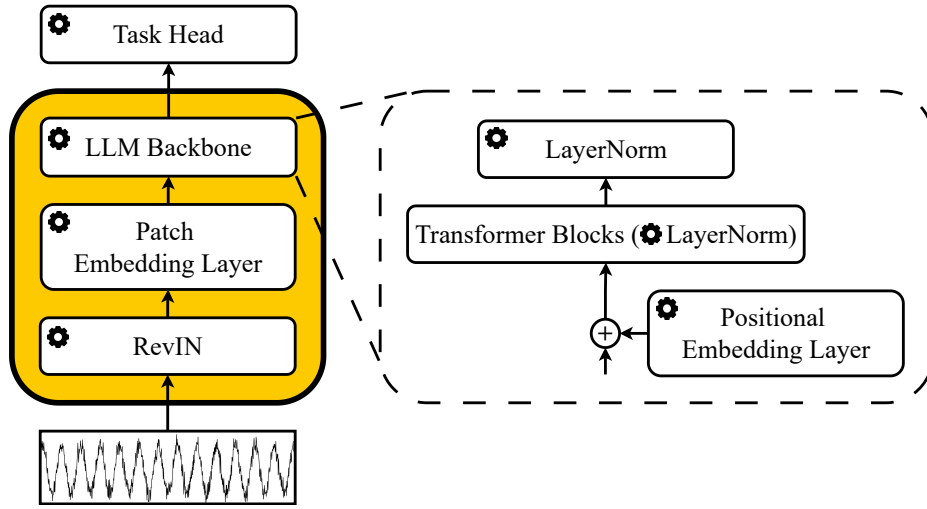


Figure 3-2: OneFitsAll model architecture.⁷⁹

3.3.2 Patch Reprogramming Layer

*Time-LLM*⁸⁰ is a LLM-based architecture for multivariate time series forecasting whose key component is a patch reprogramming layer. As presented in Fig. 3-3, each univariate input feature $\mathbf{x}^{(i)}$ first undergoes RevIN, after which it is divided into patches and embedded into patch representations $\mathbf{E}_{\text{pat}}^{(i)} \in \mathbb{R}^{N_{\text{pat}} \times d_{\text{pat}}}$. These are then reprogrammed into the natural language representation space with a multi-head cross-attention layer, aligning the time series and language modalities to activate the LLM backbone's reasoning capabilities.^{81b}

⁷⁸ Zhou, T. et al.: Time series analysis by pretrained LM (2023). a: -; b: p. 3 f.

⁷⁹ Zhou, T. et al.: Time series analysis by pretrained LM (2023), p. 4, fig. 2.

⁸⁰ Jin, M. et al.: Reprogramming Large Language Models (2024). a: -; b: 4 ff.

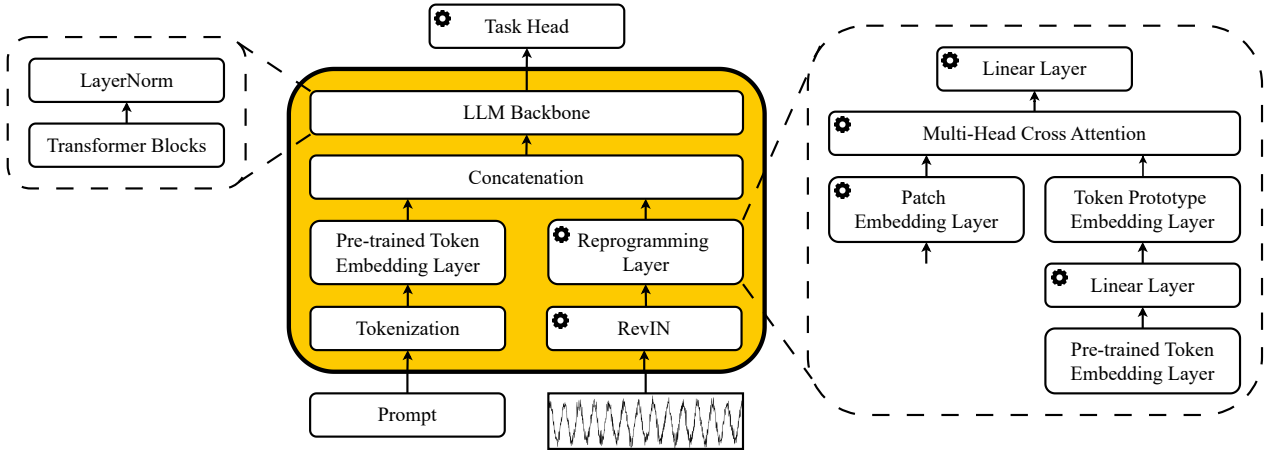


Figure 3-3: Time-LLM model architecture.⁸¹

To reprogram $\mathbf{E}_{\text{pat}}^{(i)}$, the pre-trained token embeddings $\mathbf{E}_{\text{tok}} \in \mathbb{R}^{V \times d_{\text{LLM}}}$ of the backbone are used, where V denotes the vocabulary size.^{81b} Since not all tokens are relevant, a reduced set of $V' \ll V$ token prototypes $\mathbf{E}'_{\text{tok}} \in \mathbb{R}^{V' \times d_{\text{LLM}}}$ is obtained with a learned linear projection of $\mathbf{E}_{\text{tok}}$.^{81b} Each token prototype learns to capture a linguistic cue describing local temporal patterns, such as trend direction or magnitude, allowing patch embeddings to be expressed as combinations of these cues without leaving the representation space of the pre-trained LLM.^{81b}

This is realized using multi-head cross-attention in which For each attention head $h \in \{1, \dots, H\}$, the query matrices $\mathbf{Q}_h^{(i)} = \mathbf{E}_{\text{pat}}^{(i)} \mathbf{W}_h^Q$, key matrices $\mathbf{K}_h^{(i)} = \mathbf{E}'_{\text{tok}} \mathbf{W}_h^K$, and value matrices $\mathbf{V}_h^{(i)} = \mathbf{E}'_{\text{tok}} \mathbf{W}_h^V$ with $\mathbf{W}_h^Q \in \mathbb{R}^{d_{\text{pat}} \times d_h}$ are defined.^{81b} The attention output for each head is computed as

$$\mathbf{Z}_h^{(i)} = \text{softmax} \left(\frac{\mathbf{Q}_h^{(i)} \mathbf{K}_h^{(i)\top}}{\sqrt{d_h}} \right) \mathbf{V}_h^{(i)}. \quad (3-5)$$

where $\mathbf{Z}_h^{(i)} \in \mathbb{R}^{P \times d_{\text{attn}}}$ where $\mathbf{W}_k^Q \in \mathbb{R}^{d \times d}$ and $\mathbf{W}_k^K, \mathbf{W}_k^V \in \mathbb{R}^{D \times d}$ are projection matrices with $d = \lfloor \frac{d_m}{K} \rfloor$.^{81b}

3.3.3 Patch-based Temporal Convolution Encoder

Fig. 3-4 shows the Patch-based Temporal Convolution Encoder (PTCEnc) model architecture. Convolutional Networks, PTCEnc stacks D residual causal convolution blocks, where each block captures multiscale temporal patterns with exponentially increasing dilation. The output at depth d is defined as⁸³

$$\mathbf{H}_e^{(d)} = \text{ConvBlock}^{(d)}(\mathbf{H}_e^{(d-1)}) + \mathbf{H}_e^{(d-1)}, \quad (3-6)$$

⁸¹ Jin, M. et al.: Reprogramming Large Language Models (2024), p. 4, fig. 2.

⁸² Chen, Y. et al.: LLMs are few-shot multivariate time series classifiers (2025), p. 7, fig. 2.

⁸³ Chen, Y. et al.: LLMs are few-shot multivariate time series classifiers (2025), p.8.

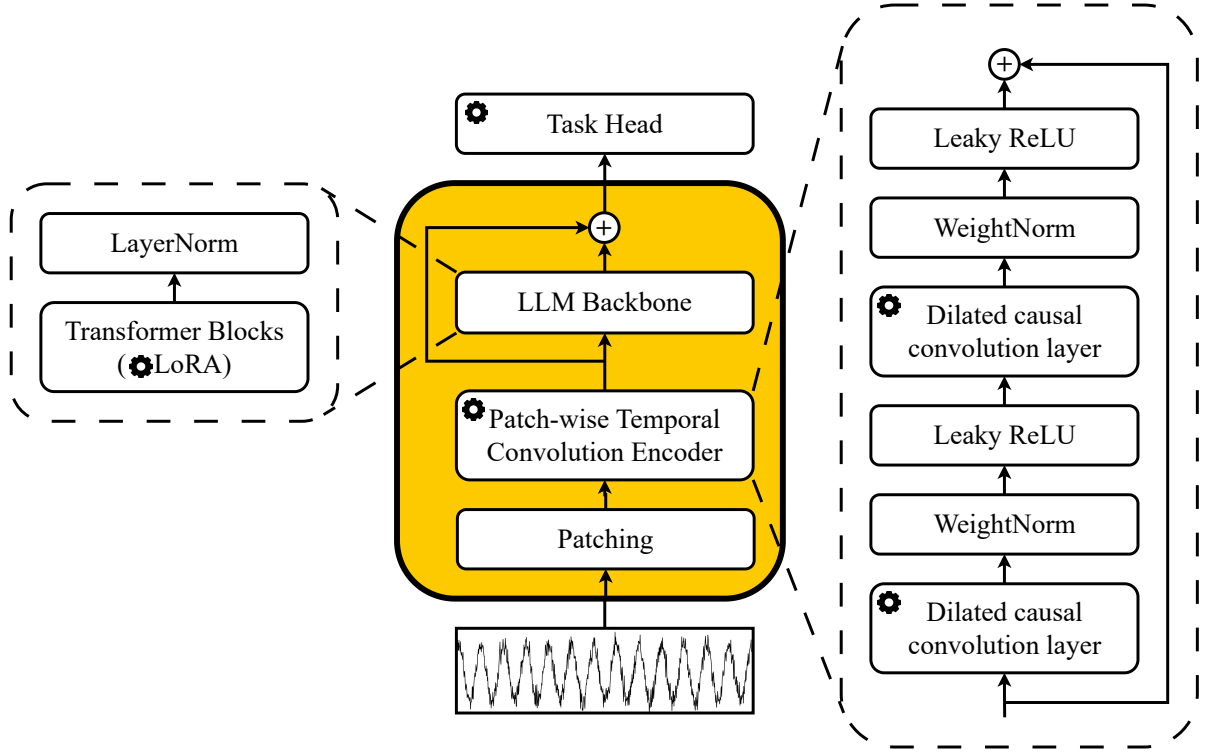


Figure 3-4: LLMFew model architecture.⁸¹

where $\mathbf{H}_e^{(0)} = \hat{\mathbf{X}}_p^{(i)}$ and each $\text{ConvBlock}^{(d)}(\cdot)$ consists of two sequential causal layers

$$\text{ConvBlock}^{(d)}(\cdot) = \text{CausalLayer}_2^{(d)}(\text{CausalLayer}_1^{(d)}(\cdot)), \quad (3-7)$$

$$\text{CausalLayer}_i^{(d)}(\cdot) = \sigma(\text{Norm}(\text{CausalConv}_i^{(d)}(\cdot))), \quad (3-8)$$

with dilation factor 2^d , weight normalization $\text{Norm}(\cdot)$, and LeakyReLU activation $\sigma(\cdot)$.⁸³ The final output is obtained as⁸³

$$\mathbf{H}_e = \text{Conv}(\mathbf{H}_e^{(D)}), \quad (3-9)$$

where a sampling convolution matches the dimension of $\mathbf{H}_e$ to the LLM hidden dimension d_{model} .

4 Definition of Requirements

Chap. 3 reveals that LLM-based architectures adapted for structured time series data have demonstrated strong sequential pattern recognition capabilities, yet their application to classification tasks remains largely unexplored in the literature. The objective of this thesis is to develop a LLM-based architecture for scenario classification, a MTSC task, in which an arbitrary ET trip must be correctly labelled across three scenario domains. These include ambient temperature, weather, and highway type. This thesis is situated in the pre-development phase of software and electronics, which has direct implications for data availability and model design. Production-scale labeled datasets are not yet available and must first be constructed through manual annotation. This process in itself motivates the need for automated ML-based labeling. The LLM-based architecture must therefore be designed to operate under conditions of limited data availability and early-stage label quality. Functional, architectural, data, label, training, and non-functional requirements are derived systematically to reflect these constraints, providing the foundation for all subsequent design and evaluation decisions.

Functional Requirements

Functional requirements specify the functions a system must perform and the input-output behavior it must exhibit under specified conditions.⁸⁴ The LLM-based architecture must accept batches of multivariate time series data of variable sequence length and variable number of features as input, without requiring architectural modifications for different input configurations. For each input sample, the model must produce a single predicted class label together with a probability distribution over all classes, where the number of classes is not fixed a priori. The predicted class label must correspond to the class with the highest predicted probability.

Architectural Requirements

Based on the categorization of Bass et al.⁸⁵, architectural requirements refer to structural constraints a system must satisfy in terms of its components and their relationships. They are closely linked to functional requirements, as the input-output behavior directly constrains how the architecture must be structured internally. Drawing on the work in Chap. 3, the LLM-based architecture must consist of a data representation layer, an interchangeable LLM backbone, and a classification head. The data representation layer must process each input feature independently, ensuring generalization across ET fleets with differing sensor configurations. Since most existing research on LLM-based architectures focuses on forecasting rather than classification, at least two candidate architectures with distinct data representation strategies must be systematically compared on neutral MTSC benchmarks. In accordance with CRISPMQ^{86a}, neutral benchmarks ensure that model selection remains free from domain-specific assumptions, so that subsequent adaptations to the ET context can be validated against an unbiased baseline. To ensure comparability, all candidates must share the same LLM backbone, classification head, and training settings. A novel architecture is subsequently derived from the most promising components identified. The detailed methodology for this selection process are outlined in Chap. 5.

⁸⁴ Glinz, M.: On Non-Functional Requirements (2007), p. 21.

⁸⁵ Bass, L. (et al.: Software architecture in practice (2013), pp. 63-78.

⁸⁶ Studer, S. et al.: Towards CRISP-ML(Q) (2021). a: p. 401; b: p. 398.

Data Requirements

Data requirements are defined at two levels.^{89b} At the meta-level, each ET trip must originate from a documented vehicle configuration with a known data dictionary specifying feature names, units, and sampling rates. The dataset must include trips from varying payload and route conditions to reflect the operational diversity of the ET fleet. At the data level, the requirements in Tab. 4-1 apply to the extraction of ET trips from the CAN-bus system to mitigate poor data quality early on and focus on operationally relevant trips. Trips that do not satisfy these conditions are excluded. Early detection of data errors reduces downstream debugging effort, improves model quality, and saves engineering hours.⁸⁷

Table 4-1: Requirements for extracting ET trips.¹⁰⁵

Requirement	Threshold
Trip length	> 600 s
Average speed	> 3 km/h
Maximum speed	> 5 km/h
Standstill share	< 60 %
NaN share (speed)	< 20 %
NaN share (SoC)	< 20 %
SoC difference	> 3 %

Prior to labeling, each ET trip must undergo two processes. First, a cleaning procedure addresses remaining quality issues such as missing values and outliers. Second, a data verification process checks schema conformity, data completeness, and temporal regularity, discarding any trip that does not meet these conditions. Schema conformity requires that each ET trip contains the same features and a minimum of three features to satisfy the MTSC task definition.

Label Requirements

Labeling requirements ensure the quality and consistency of class annotations. The set of scenario classes must be fixed prior to labeling, with each class assigned a unique identifier and a physically meaningful description. Labels must be consistently applied across all ET fleets, and all classes must be represented with a sufficient number of labeled samples to support model training. Each data point in a trip sequence must be assigned a label. A sequence is assigned a single class label only if all data points share the same class. Sequences containing missing or inconsistent labels are discarded. Each label must be traceable to its raw source, with clearly defined rules for handling ambiguous values. Any features used exclusively for labeling must be discarded prior to model training to prevent label leakage.

Training Requirements

The ET data must be randomly shuffled across all fleets and split into a training set for model optimization, a validation set for model selection and early stopping, and a test set for final evaluation. Each split must

⁸⁷ Breck, E. et al.: Data Validation for Machine Learning (2019), p. 1.

⁸⁸ Siegle, W.: Masterthesis, Range Prediction with PINNs (2025), p. 55 f.

preserve the class distribution of the full dataset, ensuring that the ratio of samples per class remains consistent across all three subsets. A fixed random seed must be used for data splitting, weight initialization, and all stochastic operations to ensure reproducibility. Training must be performed exclusively on the training split. The test set must be used exactly once for final evaluation after model training and selection is complete. Hyperparameter tuning and final model selection decisions are based on the validation set. Input features must be normalized using parameters derived only from the training set, which are subsequently applied to the validation and test sets. In cases of class imbalance, over-sampling of minority classes and under-sampling of majority classes should be considered as complementary strategies.^{89a} The $F_{1,mac}$ must be monitored on the validation set at each epoch and used as the model selection criterion, as it is robust to class imbalance. The model checkpoint achieving the highest validation $F_{1,mac}$ must be saved and used for final evaluation. Early stopping must be applied with a suitable patience and a defined maximum number of training epochs, whichever termination criterion is reached first. A learning rate scheduler must be employed throughout training. All hyperparameters and their final selected values must be logged to ensure reproducibility. Mixed-precision training should be used where supported to reduce memory footprint. All training must be executable on a single NVIDIA A100 80 GB Graphics Processing Unit (GPU).

Non-Functional Requirements

Non-functional requirements govern how well a system performs its functions rather than what it does.⁹⁰ A non-functional requirement is either an attribute pertaining to performance or quality concerns, or a constraint that restricts the solution space beyond what is necessary for meeting functional requirements.⁹⁰ CRISPMQ defines the success of a ML project from the business, ML, and economic perspective.^{89b} Since this work develops a LLM-based architecture rather than a production system, the focus lies on ML success criteria. These include classification performance, robustness, and scalability. All three must be assessed in comparison to the two established MTSC baselines InceptionTime and MiniRocket, introduced in Chap. 2.3.1. The evaluation covers all three scenario classification tasks which are highway type, ambient temperature, and weather.

Classification performance is evaluated using Acc , Acc_{bal} , $F_{1,mac}$, and MCC as defined in Chap. 2.3.2. Since scenario classes are not equally frequent in the ET dataset, Acc_{bal} and $F_{1,mac}$ are the primary performance indicators, as both are robust to class imbalance. Acc serves as a secondary overall performance metric, and MCC provides a holistic assessment of classification quality. Training time t_{train} is recorded for each experiment as a measure of computational cost. All metrics are evaluated on the held-out test set after model selection is complete.

Robustness is assessed along three axes. First, task robustness measures whether performance generalizes across all three scenario classification tasks, each presenting distinct class distributions, label characteristics, and a differing number of classes. Second, data robustness measures whether performance is maintained under data scarcity, using a reduced training set of 10 % of the full training data, referred to as the few-shot experiment. Third, sequence robustness measures whether performance is consistent across sequence lengths of 100 s and 500 s. Robustness is quantified across all experimental settings, covering all three tasks, both data regimes, and both sequence lengths, using $F_{1,mac}$ and MCC as primary metrics.

⁸⁹ Studer, S. et al.: Towards CRISP-ML(Q) (2021). a: p. 400; b: p. 396 f.

⁹⁰ Glinz, M.: On Non-Functional Requirements (2007), pp. 21-26.

Scalability is closely related to robustness. While robustness measures the consistency of performance, scalability measures how performance and computational cost change with increasing data availability and model size. Data scalability compares full-shot and few-shot settings across all tasks and sequence lengths. Computational scalability measures how training time t_{train} scales with model size, directly addressing whether the additional computational cost of a larger LLM backbone is justified by a corresponding gain in classification performance.

5 Methodology and Concept

This chapter presents the methodology used to derive DeepRange, a novel LLM-based architecture for MTSC. First, a brief introduction to Multiple-criteria decision analysis (MCDA) is provided, which forms the basis of the architecture selection process. Candidate LLM-based architectures are then systematically evaluated against defined evaluation criteria. Finally, the evaluation results are used to select the most promising components and derive two novel architecture concepts.

5.1 Multi-Criteria Decision Analysis

MCDA is a key discipline in operations research and management science, useful for addressing complex decision problems that involve conflicting objectives, heterogeneous data types, multiple stakeholders, and high uncertainty.⁹¹ It provides a systematic framework for selecting an appropriate policy, strategy, action, or choice from multiple alternatives based on quantitative and qualitative criteria.⁹¹ Decision problems fall into categories depending on the time horizon of the outcomes, spanning tactical, strategic and operational decisions as listed in Tab. 5-1.^{92a} MCDA can be applied in the ML domain, where model selection constitutes an operational decision problem. Model selection follows an iterative cycle with a

Table 5-1: Categories of decision problems.^{92a}

Decision	Time Horizon	Novelty	Degree of Structure	Automation
Strategic	Long term	New	Low	Low
Tactical	Medium term	Adaptive	Semi-structured	Middle
Operational	Short term	Every day	Well defined	High

high degree of automation. End-to-end process frameworks, such as CRISPMQ, define a well-defined structure for building machine learning systems. After candidate model architectures have been selected, the pipeline consisting of data preparation, model training, and evaluation can be highly automated using orchestration tools such as MLflow⁹³. The time horizon is determined mainly by the training phase, which can span from several hours to multiple days⁹⁴, depending on dataset and model size. Based on the model evaluation results, further hyperparameter tuning or the selection of a different model architecture may be required. Advances in AI occur rapidly, requiring continuous effort to stay up to date on new model architectures and maintain a short-term decision horizon. According to Tang et al., the number of AI preprints increased by approximately an order of magnitude every six years between 1993 and 2019.⁹⁵ For example, in 2019, the number of AI preprints submitted to arXiv, an open research-sharing platform funded by Cornell University, amounted to 3817 per hour.⁹⁵

The developed methodology for the model selection problem is oriented toward the simple multi-attribute rating technique (SMART)⁹⁶ as Fig. 5-1 outlines, which is a commonly used MCDA method in computer

⁹¹ Zyoud, S. H.; Fuchs-Hanusch, D.: AHP and TOPSIS techniques (2017), p. 158.

⁹² Ishizaka, A.; Nemery, P.: Multi-criteria Decision Analysis (2013). a: p. 2; b: p. 3 f.

⁹³ Zaharia, M. A. et al.: Accelerating Machine Learning with MLflow (2018).

⁹⁴ Hazelwood, K. et al.: Applied Machine Learning at Facebook (2018), p. 621.

⁹⁵ Tang, X. et al.: The pace of artificial intelligence (2020), p. 8.

⁹⁶ Olson, D. L.: Smart (1996), pp. 34-48.

science and mathematics^{97a}.

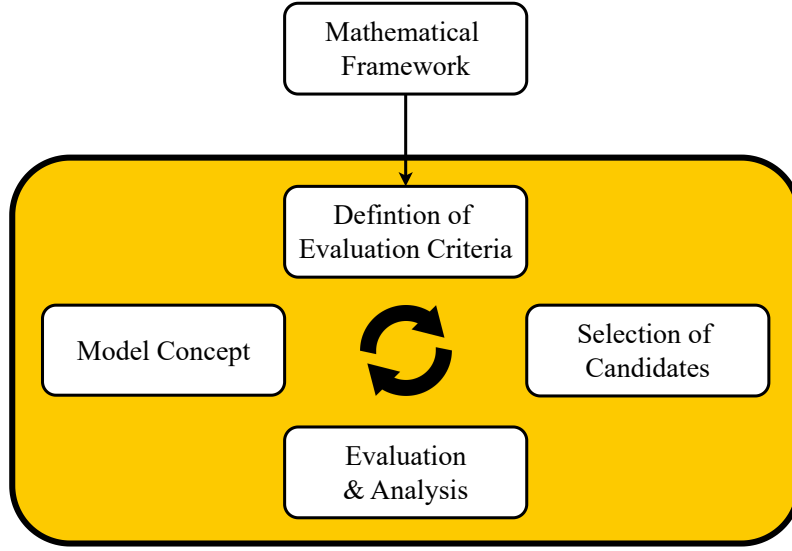


Figure 5-1: Developed methodology for the model selection problem.^{97b}

The objective is to derive an LLM-based architecture for MTSC. To identify the most suitable data representation and architecture, each candidate is systematically evaluated against a set of defined criteria, with weights assigned to reflect their relative importance. The evaluation results are organized into a decision matrix, from which a weighted sum is computed for each architecture, enabling a structured and objective comparison. Based on this analysis, the best-performing architectures are identified and their most promising components selected. In a subsequent phase, two novel component combinations are explored, drawing from the strengths of the top-performing architectures while compensating for their respective weaknesses. Given that multiple combinations may require re-evaluation within the decision matrix, the overall procedure is inherently iterative.

5.1.1 Mathematical Framework

Since LLM-based time series classification lacks established performance benchmarks, absolute scoring thresholds cannot be defined in a principled way. A rank-based scoring approach is therefore adopted, in which architectures are evaluated relative to one another rather than against fixed reference values. Let $m \in \mathbb{N}$ denote the number of candidate LLM-based architectures and $n \in \mathbb{N}$ the total number of evaluation criteria. For each criterion $j = 1, \dots, n$, each architecture $i = 1, \dots, m$ is assigned a rank $r_{ij} \in \{1, \dots, m\}$, where $r_{ij} = 1$ denotes the best-performing and $r_{ij} = m$ the worst-performing architecture with respect to criterion j . Each rank is then normalized to a score $x_{ij} \in [0, 1]$ with

$$x_{ij} = \frac{m - r_{ij}}{m - 1}, \quad (5-1)$$

so that $x_{ij} = 1$ corresponds to the top-ranked architecture and $x_{ij} = 0$ to the lowest-ranked. By construction, for all $i, k \in \{1, \dots, m\}$, if $r_{ij} < r_{kj}$ then $x_{ij} > x_{kj}$.

⁹⁷ Taherdoost, H.; Mohebi, A.: Using SMART Method for MCDA (2024). a: p. 191; b: p. 195, fig. 2.

These normalized scores are collected into the decision matrix $\mathbf{X} \in \mathbb{R}^{m \times n}$, defined as⁹⁸

$$\mathbf{X} = \begin{pmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{pmatrix}. \quad (5-2)$$

Each criterion j is assigned a weight $w_j \in [0, 1]$ reflecting its relative importance, subject to the constraint⁹⁸

$$\sum_{j=1}^n w_j = 1. \quad (5-3)$$

The final aggregated score for architecture i is then computed as the weighted sum over all criteria,

$$x_{i,\text{tot}} = \sum_{j=1}^n w_j x_{ij}. \quad (5-4)$$

Architectures are subsequently ranked by $x_{i,\text{tot}}$, with higher values indicating stronger overall performance.

5.1.2 Definition of Evaluation Criteria

The evaluation criteria applied to all candidate architectures are summarized in Tab. 5-2. The selection integrates classification performance with computational constraints. Predictive performance, as the most important criterion with assigned weight $w = 0.6$, is quantified using Acc , Acc_{bal} and $F_{1,\text{mac}}$.

Table 5-2: MCDA Evaluation Criteria.

Criterion	Description	w
Performance	Acc , Acc_{bal} , $F_{1,\text{mac}}$	0.6
Training time	t_{train} (min)	0.2
Model size	Number of parameters (million)	0.2

The selection is based on the ability to measure overall correctness and account for class imbalance. Training and evaluation are conducted on the UEA MTSC archive, a standardized benchmark collection of 30 datasets jointly developed by researchers at the UEA and the University of California Riverside (UCR).^{99a} The archive was specifically designed to address prior limitations in dataset availability and comparability for MTSC, thereby facilitating reproducible and robust statistical validation across models.^{99b} Detailed information is provided in Tab. A-1. Since evaluation is conducted across multiple UEA datasets, performance is summarized by the mean μ and standard deviation σ of each metric across datasets to capture absolute performance, and by a critical difference diagram to assess relative performance. The critical difference diagram visualizes the average rank of each model across all datasets on a number line, where models are grouped by solid lines into cliques.¹⁰⁰ Cliques are sets of models between which no

⁹⁸ Chaube, S. et al.: Multi-Criteria Decision Analysis (2024), p. 585 f.

⁹⁹ Bagnall, A. et al.: The UEA archive (2018), a: -; b: p. 1 f; c: p. 31 ff.

¹⁰⁰ Aeon: Plot critical difference (2025).

significant difference in performance exists and are determined using pairwise Wilcoxon signed-rank tests.¹⁰⁰ Accuracy is commonly used as the performance metric for critical difference diagrams, as adopted by the UEA archive.^{99c} The remaining two criteria, each assigned equal weight $w = 0.2$, capture computational cost and are measured by training time in minutes the averaged number of trainable parameters in millions, respectively. The parameter count is averaged across all UEA datasets, because the size of the data representation layer depends on the number of patches.

5.1.3 Selection of Candidate Architectures

The model architectures Time-LLM, One Fits All, and LLM-Few, introduced in Chap. 3.3, are selected as candidates with three main modifications. RevIN is not applied, as it has not demonstrated consistent benefits for Time Series Classification (TSC) tasks as discussed in Chap. 3.1. All candidates share Llama-3.2-1B as the LLM backbone, motivated by the relatively small scale of the UEA benchmark datasets and the associated risk of overfitting with larger models. This aligns with the CRISPMQ principle of starting with models of limited capacity and increasing capacity in subsequent steps as warranted by data properties and task requirements.¹⁰¹ All candidates further share a common classification head, shown in Fig. 5-2, inspired by Chen et al.¹⁰² The head flattens the LLM backbone output, applies LayerNorm and a ReLU activation, followed by dropout and a linear layer that maps to the class labels.

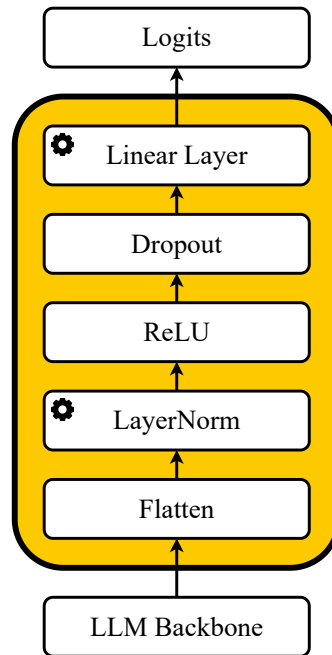


Figure 5-2: Architecture of the proposed classification head, adapted from Chen et al.¹⁰²

All datasets are normalized prior to training. The model is trained for 100 epochs on an NVIDIA A100 80 GB GPU. Since the UEA archive provides predefined train and test splits, no validation set is used. The remaining hyperparameters, which vary across datasets but are the same for all model architectures, are summarized in Tab. A-2.

¹⁰¹ Studer, S. et al.: Towards CRISP-ML(Q) (2021), p. 401.

¹⁰² Chen, Y. et al.: LLMs are few-shot multivariate time series classifiers (2025), p. 9.

5.1.4 Evaluation and Analysis

The classification performance across all candidate architectures on the UEA benchmark datasets is summarized in Tab. 5-3. LLM-Few achieves the best results across all metrics. One Fits All and Time-LLM perform comparably to each other, with Time-LLM marginally outperforming One Fits All in terms of Acc while showing slightly lower Acc_{bal} and $F_{1,mac}$.

Table 5-3: Classification performance on the UEA datasets reported as $\mu \pm \sigma$. Best μ per metric shown in bold.

Metric	One Fits All	Time-LLM	LLM-Few
Acc	0.676 ± 0.241	0.679 ± 0.244	0.702 ± 0.238
Acc_{bal}	0.665 ± 0.250	0.662 ± 0.260	0.690 ± 0.241
$F_{1,mac}$	0.667 ± 0.250	0.666 ± 0.259	0.698 ± 0.241
Detailed results	Tab. A-3	Tab. A-4	Tab. A-5

The consistency between Acc and Acc_{bal} across all three architectures suggests that class imbalance does not substantially distort the results. The high standard deviations across all models reflect the heterogeneity of the UEA benchmark, where performance varies considerably across datasets of different domains and complexity, rather than indicating instability of the models themselves. The critical difference diagram in Fig. 5-3 shows the mean ranks of the three candidate architectures across the UEA benchmark datasets, where rank 1 indicates best performance and rank 4 the worst. LLM-Few achieves the best mean rank, followed closely by One Fits All, while Time-LLM performs worst. The thick horizontal bar indicates a clique, which means that no architecture can be considered definitively superior to the others based on this evaluation alone. This is consistent with the results in Tab. 5-3. LLM-Few nonetheless consistently ranks highest and is the most promising candidate for further development.

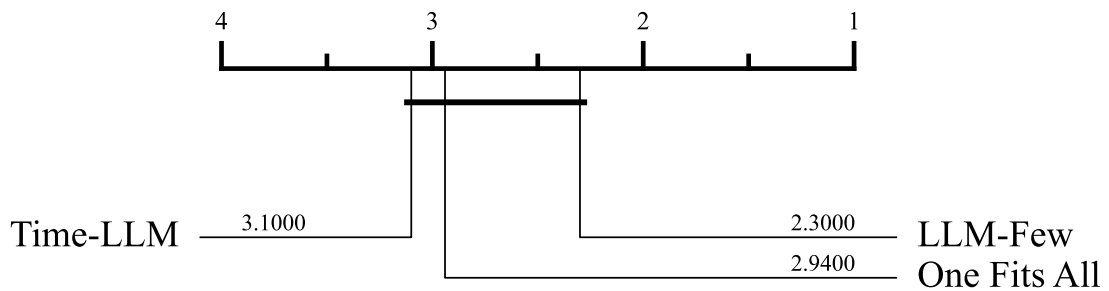


Figure 5-3: Critical difference diagram based on Acc across the UEA benchmark datasets, computed using the Aeon Python library.¹⁰³

The averaged number of model parameters and training time across the three candidate architectures are presented in Tab. 5-4. All architectures are dominated by the shared Llama-3.2-1B backbone, which

¹⁰³ Aeon: Plot critical difference (2025).

accounts for the vast majority of total parameters in each case. Time-LLM is the largest architecture overall, with its reprogramming layer contributing 164.411 million additional parameters and accounting for 11.764 % of trainable parameters. This also mirrors into training time, with Time-LLM requiring 264.539 min, more than three times that of One Fits All and more than twice that of LLM-Few. One Fits All and LLM-Few are considerably more parameter-efficient, with trainable parameters accounting for 0.554 % and 0.291 % of their respective totals. LLM-Few has the fewest trainable parameters overall despite a marginally larger backbone, attributable to its PTCEnc being substantially smaller than the patch embedding used in One Fits All.

Table 5-4: Model size (in million of parameters) and training time across the UEA datasets.

Component	One Fits All	Time-LLM	LLM-Few
Patch Embedding	6.458	–	–
PTCEnc	–	–	1.543
Reprogramming Layer	–	164.411	–
LLM Backbone (Llama 3.2 1B)	1235.814	1235.814	1237.518
Classification Head		0.354	
Total Parameters	1242.626	1400.579	1239.415
Trainable Parameters	6.880 (0.554 %)	164.765 (11.764 %)	3.601 (0.291 %)
t_{train}	79.225 min	264.539 min	119.093 min

The decision matrix in Tab. 5-5 reveals that no single candidate architecture dominates across all criteria. One Fits All and LLM-Few achieve equal total scores, yet with fundamentally different trade-off profiles.

Table 5-5: Decision matrix and final results.

Criterion	OneFitsAll		Time-LLM		LLM-Few	
	r	x	r	x	r	x
Performance	2	0.5	2	0.5	1	1
Model Size	1	1	3	0	2	0.5
Training Time	1	1	3	0.5	2	0
x_{tot}	0.7		0.4		0.7	

One Fits All achieves the highest computational efficiency but lowest classification performance, while LLM-Few achieves the highest classification performance at greater computational cost. Time-LLM scores lowest overall, as the computational overhead of its reprogramming layer and the prompt-as-prefix mechanism. Critically, the absolute performance differences between all three architectures remain small, motivating the derivation of a novel architecture that selectively retains the most effective components while discarding those that introduce unnecessary complexity.

5.2 Introducing DeepRange

The first proposed architecture, DeepRange V1, shown in Fig. 5-4, follows the reprogramming paradigm of Time-LLM but replaces its patch embedding layer with PTCEnc for the tokenization of the input time series. Unlike Time-LLM, which keeps the LLM backbone fully frozen, DeepRange V1 applies Low-Rank Adaptation (LoRA) fine-tuning to the backbone, consistent with the approach used in LLM-Few. The same classification head introduced in Fig. 5-2 is retained.

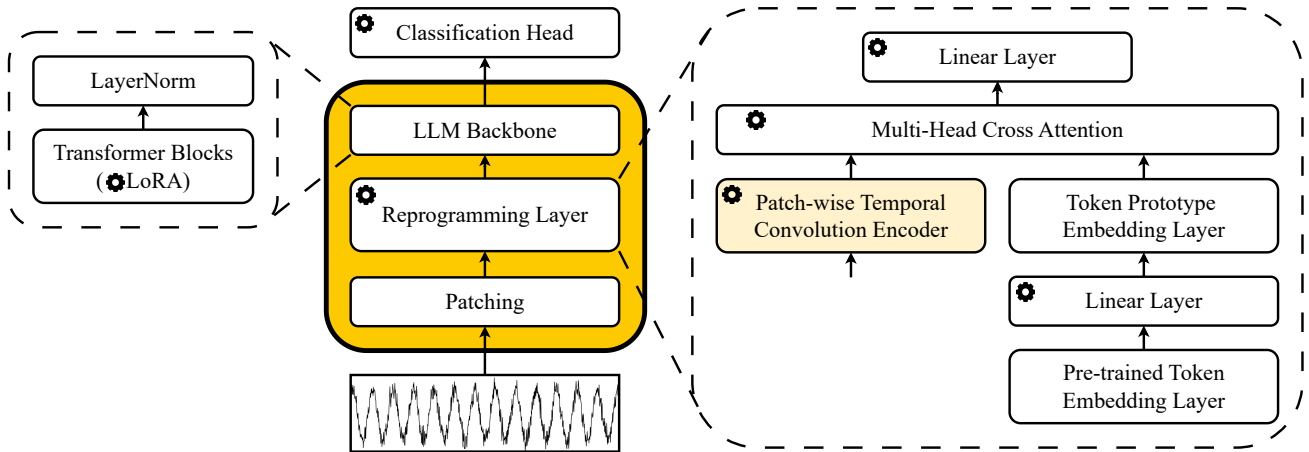


Figure 5-4: DeepRange V1 model architecture.

The second proposed architecture, DeepRange V2, shown in Fig. 5-5, retains the reprogramming layer, LoRA fine-tuning of the LLM backbone, and the shared classification head. The patch embedding layer is however redesigned. Instead of applying a linear projection directly after patching, a token-based convolutional layer followed by a positional embedding layer is applied.

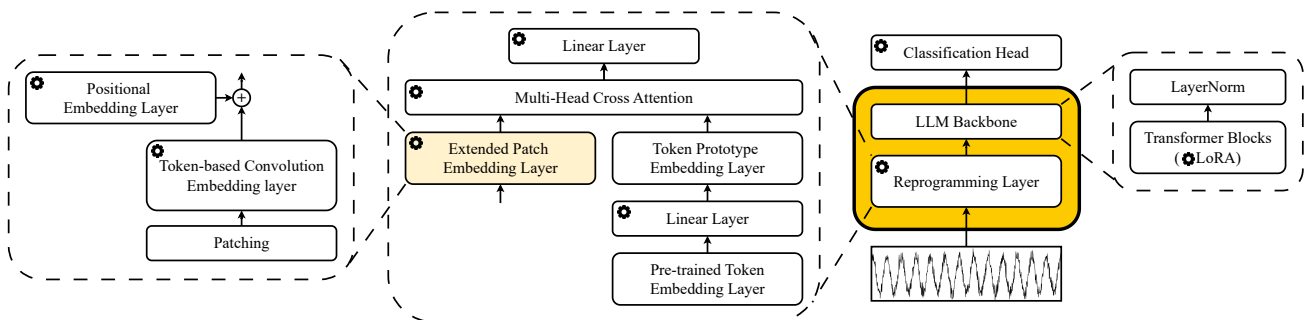


Figure 5-5: DeepRange V2 model architecture.

The performance of both DeepRange architectures across the UEA benchmark datasets is outlined in Tab. 5-6. DeepRange V2 outperforms DeepRange V1 across all three metrics by a consistent margin. Relative to the candidate architectures, DeepRange V2 performs better to One Fits All and Time-LLM across all metrics. However, the critical difference diagram in Fig. 5-6 confirms that these differences are not statistically significant among all architectures except DeepRange V1, which ranks last and appears to separate from the remaining clique. As shown in Tab. 5-7, DeepRange V2 trains in 98.065 min on average, which is faster than both Time-LLM and LLM-Few despite having more total parameters.

Table 5-6: DeepRange V1/V2 classification performance on the UEA datasets reported as $\mu \pm \sigma$. Best μ per metric shown in bold.

Metric	Deep Range V1	Deep Range V2
Acc	0.643 ± 0.248	0.683 ± 0.237
Acc_{bal}	0.625 ± 0.262	0.670 ± 0.245
$F_{1,mac}$	0.619 ± 0.278	0.677 ± 0.242
Detailed results	Tab. A-6	Tab. A-7

DeepRange V2 is selected as the final architecture. Its extended patch embedding produces representations structurally analogous to text tokens, and the reprogramming layer explicitly aligns these with the LLM backbone’s pretrained embedding space.

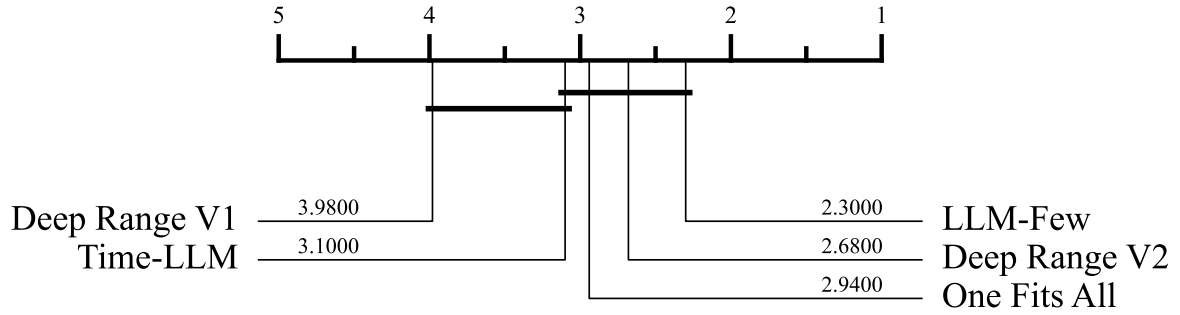


Figure 5-6: Critical difference diagram comparing candidate architecture accuracy across the UEA benchmark datasets and both DeepRange, computed using the Aeon Python library.¹⁰⁴

Combined with LoRA fine-tuning, DeepRange V2 achieves comparable classification performance to LLM-Few with lower training time and a modular design that provides clear extension points for adaptation to the ET scenario classification task.

Table 5-7: Model size (in million of parameters) and average training time across the UEA datasets.

Layers	Deep Range V1	Deep Range V2
Reprogramming Layer	146.586	292.668
LLM Backbone (Llama 3.2 1B)	1236.994	
Classification Head	0.354	
Total Parameters	1383.934	1530.016
Trainable Parameters	148.119 (10.703 %)	294.201 (19.229 %)
t_{train}	91.767 min	98.065 min

¹⁰⁴ Aeon: Plot critical difference (2025).

6 Implementation of Scenario Classification

This chapter presents the implementation of scenario classification of ETs. The setup is oriented towards CRISPMQ. fsirt a few stiats tidnetrsthte ET data is eoxianed thne the dat verification process folw sby the labelignrprocess after that

6.1 Electric Truck Data

The ET data is acquired from second-generation Mercedes-Benz eActros 300 and eActros 400 vehicles, provided in 14 distinct configurations¹⁰⁵ with battery capacities of 336 kWh and 448 kWh¹⁰⁶, respectively. These configurations differ in vehicle size, trailer or body type, number of axles, corresponding curb weight¹⁰⁵ and can be grouped into six¹⁰⁷ distinct vehicle groups. The ET data encompasses approximately 3000 raw trip files recorded from the CAN-bus system at frequencies between 0.1 Hz and 10 Hz. The data undergoes a preprocessing pipeline established by Siegle¹⁰⁵ and is downsampled to 1 Hz, yielding 12967 trip files comprising approximately 450 million rows in total. Each trip file consists of 46 features, described in Tab. A-8. Notable features include velocity, ambient temperature, Global Positioning System (GPS) coordinates, drivetrain variables, and battery state parameters. The trips are predominantly conducted in Germany, accounting for 82.471 % of all trips, followed by Turkey at 8.437 % and the Netherlands at 4.889 %. The remaining trips are distributed across Spain, Finland, Italy, Austria, Sweden, France, Serbia, Denmark, Bulgaria, Slovenia, and Croatia, each contributing a progressively smaller share. An example trip between Sweden and Denmark is shown in Fig. 6-1, displaying the GPS trajectory alongside the covered distance, average speed, and SoC.

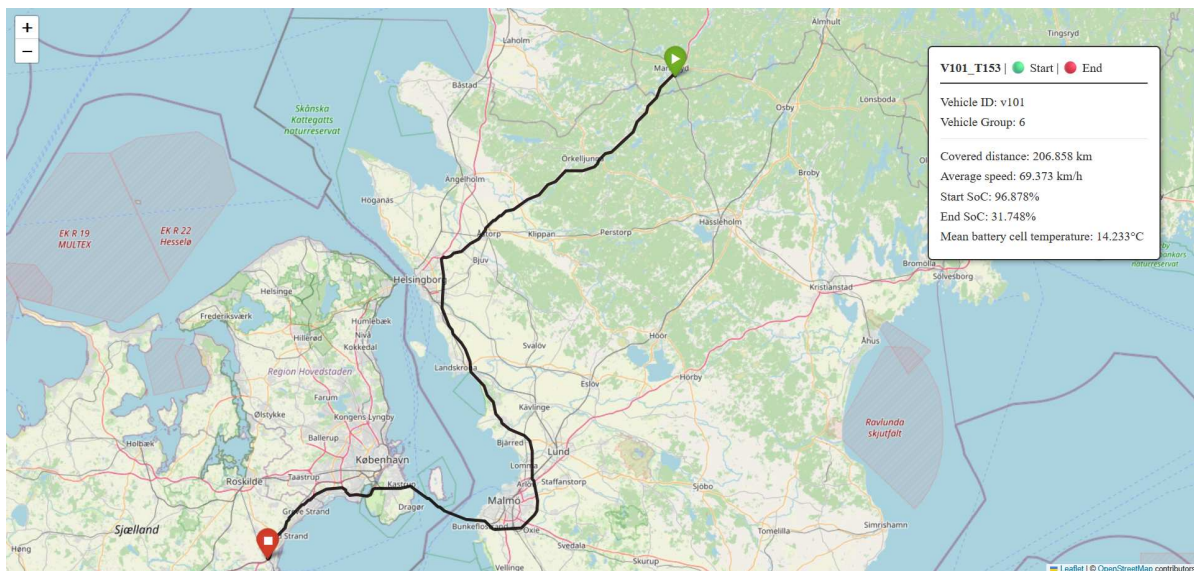


Figure 6-1: ET trip from Sweden to Denmark.

The descriptive statistics for all 12967 trips are summarized in Tab. 6-1. Trip distances range from 3.515 km to 343.120 km with a mean of 49.324 km and a high standard deviation. Trip durations follow a

¹⁰⁵Siegle, W.: Range Prediction with PINNs (2025), p. 54 ff.

¹⁰⁶Dang, C. et al.: Robust Electric Truck Range Estimation (2025), p. 2398.

¹⁰⁷Hecht, Y.: Filling Data Gaps with GenAI (2025), p. 25.

similar pattern, ranging from under 10 min to nearly 6 h. Payload varies between 8 t and 44 t with a mean of 27.623 t.

Table 6-1: Descriptive statistics for all ET trips across payload, distance, and duration.

Variable	min	max	μ	σ
Payload (t)	8.000	44.000	27.623	8.983
Distance (km)	3.515	343.120	49.324	56.625
Duration (min)	9.983	349.500	57.955	54.245

6.2 Data Labeling

The data labeling pipeline consists of two main processes. First, a data verification step ensures that each trip satisfies the data requirements defined in Chap. 4 and that all relevant features required for labeling are present. Second, the labeling procedure is applied across highway type, ambient temperature, and weather condition. The features used exclusively for label assignment are listed in Tab. 6-2 and are discarded prior to model training to prevent label leakage.

Table 6-2: Features used exclusively for label assignment and discarded prior to model training.

Feature	Highway Type	Ambient Temperature	Weather Condition
signal_time	×	×	✓
airtempoutsd_cval_cpc	×	✓	×
altitude_cval_ippc	✓	×	✓
latitude_cval_ippc	✓	×	✓
longitude_cval_ippc	✓	×	✓
vehspd_cval_cpc	✓	×	×

6.2.1 Data Verification

Each trip file is subject to a quality verification step that checks schema conformity, data completeness, and temporal regularity, with any failing files recorded in a structured report. Schema verification is applied by comparing the set of columns present in the loaded trip file against a predefined mapping that specifies all expected feature names. Any missing or unexpected columns raise an error immediately. Columns are additionally reordered to match the canonical ordering defined in the mapping, guaranteeing consistent feature indexing across all trip files. The format and data quality check performs two independent assessments. Any missing values across all columns are checked then the temporal regularity of the trip is verified by computing successive differences between consecutive timestamps. Since all data has been downsampled to 1 Hz, each time difference must equal exactly one second. Any deviation indicates gaps, duplicated timestamps, or irregular sampling.

6.2.2 Highway Labeling

Highway labelling is performed by spatially matching each GPS point of a trip against OpenStreetMap¹⁰⁸ (OSM) road network data. Two labelling approaches were evaluated prior to selecting the final implementation. The first approach uses the Overpass API¹⁰⁹ to fetch highway geometries online for each trip's bounding box, matching points with buffered spatial joins. The second approach uses the OSMnx¹¹⁰ library, either with feature extraction or graph-based nearest edge snapping. Both were discarded due to the cumulative processing time required for 12967 trips, which would have extended to multiple weeks. Offline labelling using pre-downloaded country-level maps from Geofabrik¹¹¹ was therefore adopted. The offline pipeline is implemented using Pyogrio¹¹², which loads OSM line geometries from national map files filtered directly by highway key, and caches the resulting GeoDataFrame as a parquet file per country upon first load. For each trip, the cached GeoDataFrame¹¹³ is spatially subset to the trip bounding box prior to the spatial join, avoiding the need to query the full national road network on every iteration. For trips spanning multiple countries, the respective national map files are loaded and concatenated prior to processing. The highway keys considered¹¹⁴ include motorway, residential, and living street, among others. For each trip, the GPS trajectory bounding box is expanded by a configurable buffer and used to spatially subset the cached highway GeoDataFrame. Both the GPS points and the filtered highway geometries are reprojected from WGS84 to a metric coordinate reference system (CRS) appropriate for the country or region, with EPSG:3035 applied for European trips and country-specific Universal Transverse Mercator (UTM) projections otherwise, enabling distance calculations in meters. Spatial matching assigns each GPS point its nearest highway segment within a defined search radius. Where multiple candidates fall within this radius, a weighted scoring function selects the most appropriate match by combining a distance score, a highway priority score reflecting road hierarchy, and a speed plausibility score checking whether the recorded vehicle speed falls within the expected range for the matched highway type, weighted at 0.4, 0.3, and 0.3 respectively. The highest-scoring candidate is retained per point. Points with no match within the search radius are left unlabeled. The labelled data is saved as a parquet file, and an interactive map colour-coded by highway type is generated per trip for visual verification, as shown in Fig. 6-2. Although OSM defines a larger set of highway types, six dominant classes are selected for this work based on their prevalence in the ET fleet data, namely Motorway, Primary, Residential, Secondary, Tertiary, and Trunk.

6.2.3 Weather and Ambient Temperature Labeling

Weather condition labelling is performed by fetching hourly meteorological data from external weather stations and assigning condition labels to each GPS point of a trip. Since fetching weather data for every individual GPS point at 1 Hz sampling rate would be computationally prohibitive across 12967 trips, the trip is first segmented into fixed-size time windows of configurable length. Within each window, the first, middle, and last serve as sequential fallback candidates for the weather data fetch. This windowing

¹⁰⁸ OpenStreetMap: OpenStreetMap (2025).

¹⁰⁹ OpenStreetMap: Overpass API (2025).

¹¹⁰ OSMnx: OSMnx (2025).

¹¹¹ Geofabrik: OpenStreetMap Data Extracts (2025).

¹¹² Pyogrio: About (2025).

¹¹³ GeoPandas: geopandas.GeoDataFrame (2025).

¹¹⁴ OpenStreetMap: Key:highway (2025).

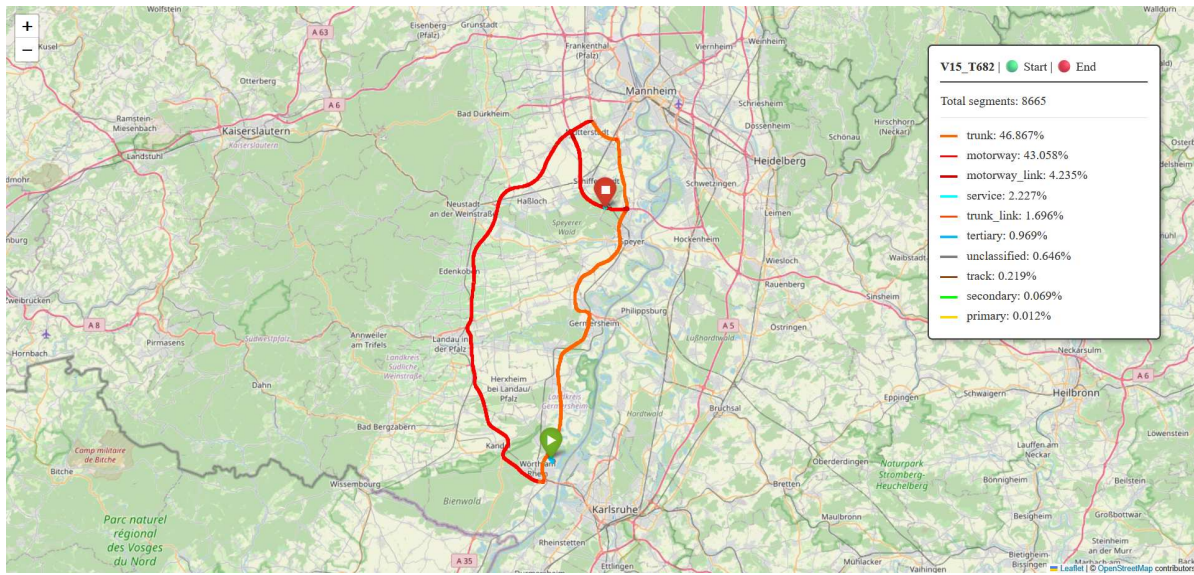


Figure 6-2: Highway labels for an electric truck trip.

strategy substantially reduces the number of API calls while preserving sufficient temporal resolution for weather labelling. Hourly meteorological data is retrieved using the Meteostat¹¹⁵ Python library, which queries the nearest available weather station to a given GPS coordinate. For each window, the middle row serves as the primary representative point. If the fetch fails or returns an empty result, the first and last rows are used as fallbacks in sequence. The queried time interval is centered on the rounded hour of the window timestamp and extended by a configurable delta in both directions to ensure data availability even at sparse station coverage. The retrieved variables include air temperature, dew point, relative humidity, precipitation, snow depth, wind speed, peak wind gust, sea-level pressure, sunshine duration, and the Meteostat present weather condition code. If temperature itself is missing, it is substituted by the vehicle's onboard exterior air temperature sensor signal. Atmospheric weather condition classification follows a strict priority hierarchy. Meteostat present weather condition codes are first mapped to predefined labels, as direct station observations carry the highest confidence. These codes define the label domain for the weather condition and comprise nine classes, namely Mild, Fog, Hail, Heavy Rain, Rain, Sleet, Heavy Sleet, Snowfall, and Storm. Where condition codes are absent, sensor-based rules derived from the remaining meteorological variables are applied in descending priority order. To handle gaps in the Meteostat condition codes, a conservative gap-filling procedure is applied to the weather condition series. A gap of up to five consecutive missing values is filled only if the immediately preceding and following non-missing values are identical, ensuring that labels are not extrapolated across condition transitions. Filled positions are tracked via a dedicated flag column for transparency and quality verification. Finally, the per-window labels are merged back onto the full trip dataframe. The label distribution per trip is saved as a JSON file for downstream quality verification.

Ambient temperature is classified into six categories based on OEM domain knowledge. Extreme cold covers temperatures below -10°C , freezing cold ranges from -10°C to 0°C , cold from 0°C to 10°C , moderate from 10°C to 20°C , warm from 20°C to 30°C , and hot above 30°C . The onboard exterior temperature sensor serves as the primary source for this classification, as it directly reflects the ambient

¹¹⁵ Meteostat: The Weather's Record Keeper (2025).

conditions experienced by the vehicle rather than the nearest weather station reading.

6.3 Data Preparation

The data preparation pipeline consolidates the highway, ambient temperature and weather condition labels with the preprocessed trip data, extracts fixed-length windows suitable for model training, and produces stratified, normalized train, validation, test, and few-shot splits for each label domain and window size configuration. The first step merges the preprocessed trip data, the highway, ambient temperature and weather labels into a single unified dataset per trip. For each trip file, the highway and weather label dataframes are loaded and joined onto the main trip dataframe. Following the merge, the combined dataframe is validated against a predefined signal mapping that specifies the expected set of features and their column ordering. Any deviation from the expected schema raises an error. The validated and merged dataframes are saved as Parquet files. The second step extracts non-overlapping fixed-length windows from each merged trip file, using sequence lengths of 100 s and 500 s. Windows are extracted independently for each label domain and each configured window size, expressed in seconds and equivalent to rows at 1 Hz sampling rate. For a given trip, all required feature columns and the target label column are loaded, and a sliding window view is computed over the label column. Only windows where all timesteps carry an identical label are retained, ensuring that each window is assigned an unambiguous single class label consistent with the sequence-to-label classification task definition. Non-overlapping windows are selected by stepping through valid window start indices at intervals equal to the window size. Windows containing any missing label values are discarded. All valid windows across all trips are stacked into a single array per label domain and window size combination and saved to disk. The third step filters, splits, and normalizes the windowed data. Configurable lists of signals and label classes to exclude are applied first. These include all features listed in Tab. 6-2 as well as the `v_id`, `signal_time`, and `signal_ts`. A train, validation, and test split is then performed with configurable proportions, applied independently per class to ensure balanced class representation across all splits. For each class, the available windows are randomly shuffled and divided according to the configured percentages, with any rounding remainder assigned to the test set. An additional few-shot training subset is constructed by sampling 10 % of the training set in a class-balanced manner, taking at minimum one sample per class. This subset supports evaluation of the model under limited data conditions. Normalization is applied after splitting to prevent data leakage.

7 Evaluation

This chapter evaluates DeepRange against the baseline models InceptionTime and MiniRocket on ET data from Mercedes-Benz eActros 300 and eActros 400 electric trucks, assessing classification performance, robustness, and scalability in accordance with the requirements defined in Chap. 4.

7.1 Highway Classification

Tab. 7-1 presents the highway classification results for a sequence length of 100 s. DeepRange 8B, DeepRange 1B, and InceptionTime achieve similar Acc of approximately 90 %, while MiniRocket lags behind by approximately 7 %.

Table 7-1: Highway classification results using a sequence length of 100 s. The best result per metric is highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Evaluation Metrics</i>				
Acc	0.897	0.899	0.898	0.832
Acc_{bal}	0.830	0.831	0.844	0.689
$F_{1,mac}$	0.838	0.841	0.847	0.703
MCC	0.863	0.866	0.865	0.776
t_{train}	128.681 min	278.915 min	215.462 min	171.375 min
<i>Number of Samples</i>				
Training		77572		
Validation		16621		
Testing		16629		
<i>Detailed Results</i>				
Per-class Metrics	Tab. A-9			
Confusion Matrices	Fig. A-1a	Fig. A-1b	Fig. A-1c	Fig. A-1d
Learning Curves	Fig. 7-2a	Fig. 7-2b	–	–
	Fig. 7-3a	Fig. 7-3b	–	–

InceptionTime achieves the highest Acc_{bal} and $F_{1,mac}$, closely followed by DeepRange 8B, indicating superior handling of minority classes. Both rank as the two strongest overall performers in this full-shot experiment. However, the gap between Acc and Acc_{bal} across all models confirms that class imbalance affects all architectures, with minority class recall being the primary bottleneck. As shown in Fig. 7-1, DeepRange 8B achieves considerably better recall on the minority classes *Tertiary* and *Primary* compared to MiniRocket. Despite an eightfold difference in backbone size, DeepRange 8B and DeepRange 1B achieve nearly identical metrics across all evaluation metrics., with no meaningful performance gain from the larger backbone in this experiment. DeepRange 1B completes training in $t_{train} = 128.681$ min, less than half the time of DeepRange 8B, due to its smaller LLM backbone and the memory constraints of a

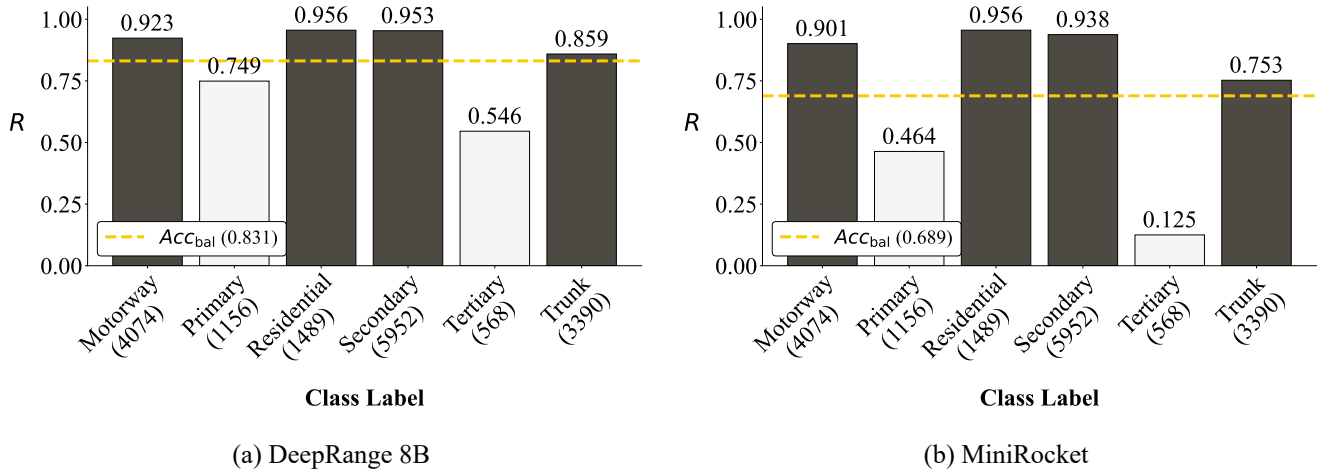


Figure 7-1: Per-class recall for full-shot highway scenario classification with sequence length 100 s. Support values are given in brackets.

single NVIDIA A100 80 GB GPU. The loss curves and learning rate schedules in Fig. 7-2 highlight that DeepRange 1B begins to overfit after epoch 10 and DeepRange 8B after epoch 8.

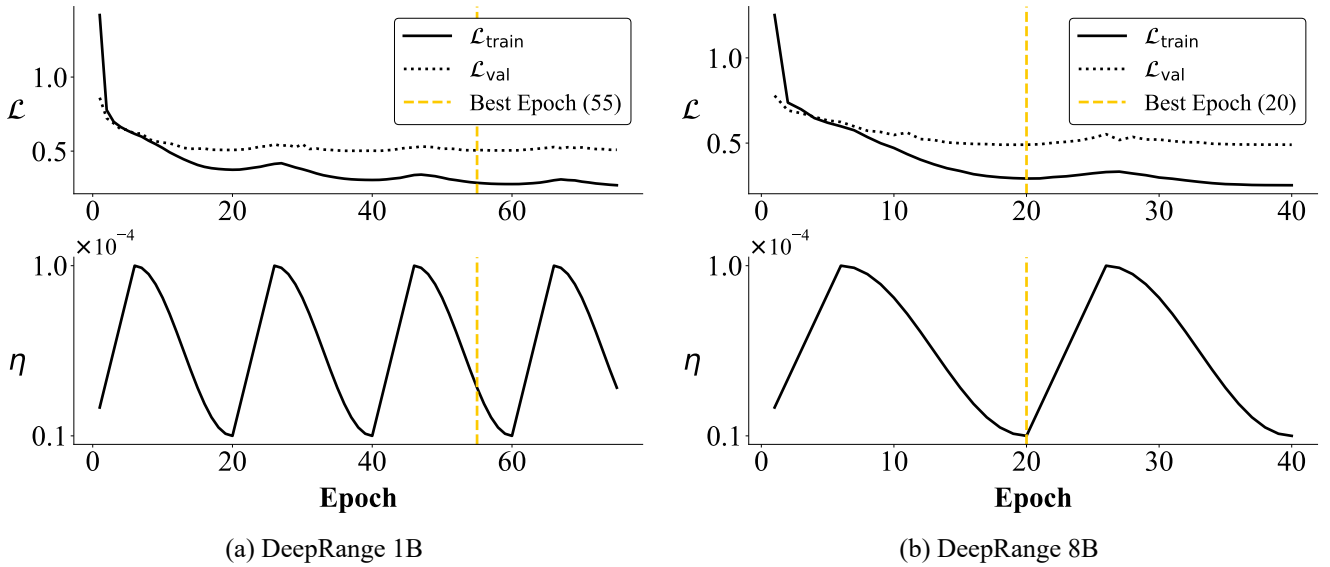


Figure 7-2: Loss and learning rate curves for full-shot highway scenario classification with sequence length 100 s.

Training beyond these points results in only marginal improvements in $F_{1,mac}$, which was used as the selection criterion for the best-performing epoch on the validation dataset as illustrated in Fig. 7-3. Both DeepRange models achieve a validation $F_{1,mac} \approx 0.6$ after only a single training epoch, indicating that the pre-trained LLM backbone already captures temporal features without requiring extensive fine-tuning. The majority of performance gains are therefore concentrated in the early training epochs. A similar effect was reported by Ouyang et al., where GPT-3 overfitted after a single epoch of supervised fine-tuning.¹¹⁶

¹¹⁶Ouyang, L. et al.: Training language models (2022), p. 6.

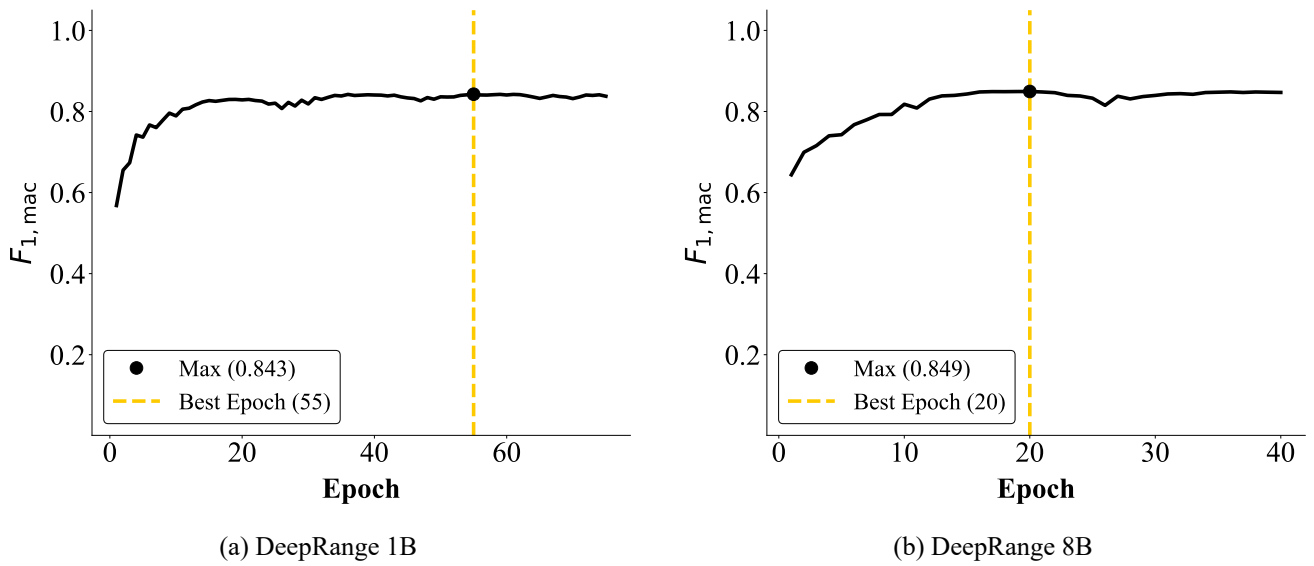


Figure 7-3: DeepRange 1B/8B validation curves for highway classification using a sequence length of 100 s.

Tab. 7-2 presents the results of the reduced training set experiment, in which only 10 % of the full training dataset is used, corresponding to 7755 training samples. MiniRocket achieves the highest Acc and MCC , a reversal from the full-shot experiment where it ranked last. InceptionTime retains the highest Acc_{bal} and $F_{1,mac}$, remaining the strongest performer on minority classes. The large performance drop of the

Table 7-2: Few-shot highway classification results using a sequence length of 100 s. The best result per metric is highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Metrics</i>				
ACC	0.767	0.767	0.790	0.800
ACC_{bal}	0.669	0.654	0.696	0.652
$F_{1,mac}$	0.670	0.658	0.699	0.662
MCC	0.692	0.691	0.722	0.733
t_{train}	42.931 min	182.154 min	24.548 min	3.370 min
<i>Number of Samples</i>				
Training		7755		
Validation		16621		
Testing		16629		
<i>Detailed Results</i>				
Per-class Metrics		Tab. A-10		
Confusion Matrices	Fig. A-2a	Fig. A-2b	Fig. A-2c	Fig. A-2d
Learning Curves	Fig. 7-4a	Fig. 7-4b	—	—
	Fig. 7-5a	Fig. 7-5b	—	—

DeepRange models compared to the full-shot experiment is attributable to the reduced training set size

leading in weaker generalization. In contrast, MiniRocket’s fixed random kernels require no gradient-based optimization beyond the final linear classifier. MiniRocket completes training in 3.370 min, over ten times faster than DeepRange 1B, making it an attractive choice in data-scarce and resource-constrained settings. Compared to the full-shot experiment in Fig. 7-2, the loss curves in Fig. 7-4 show a slower decrease during the first 20 epochs for both DeepRange models.

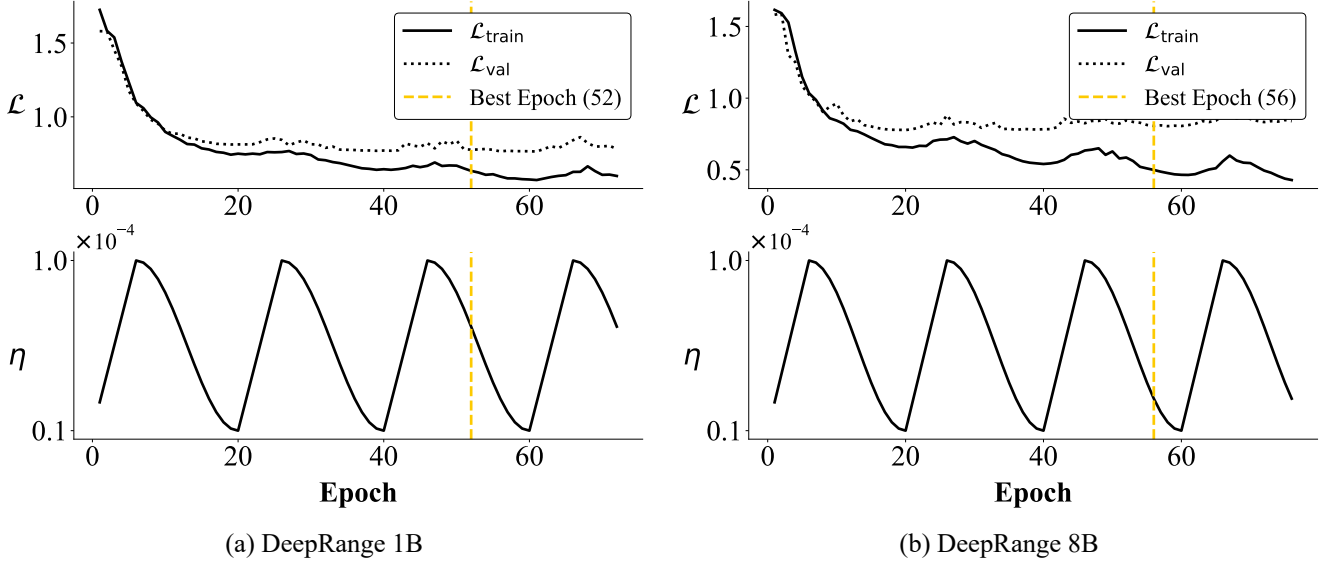


Figure 7-4: DeepRange 1B/8B loss curves and learning rate schedule for few-shot highway classification using a sequence length of 100 s.

Additionally, the validation curves in Fig. 7-5 show that both DeepRange models achieve $F_{1,\text{mac}} \approx 0.2$ after the first epoch and only reach $F_{1,\text{mac}} \approx 0.6$ after approximately 10 epochs, in contrast to the full-shot experiment.

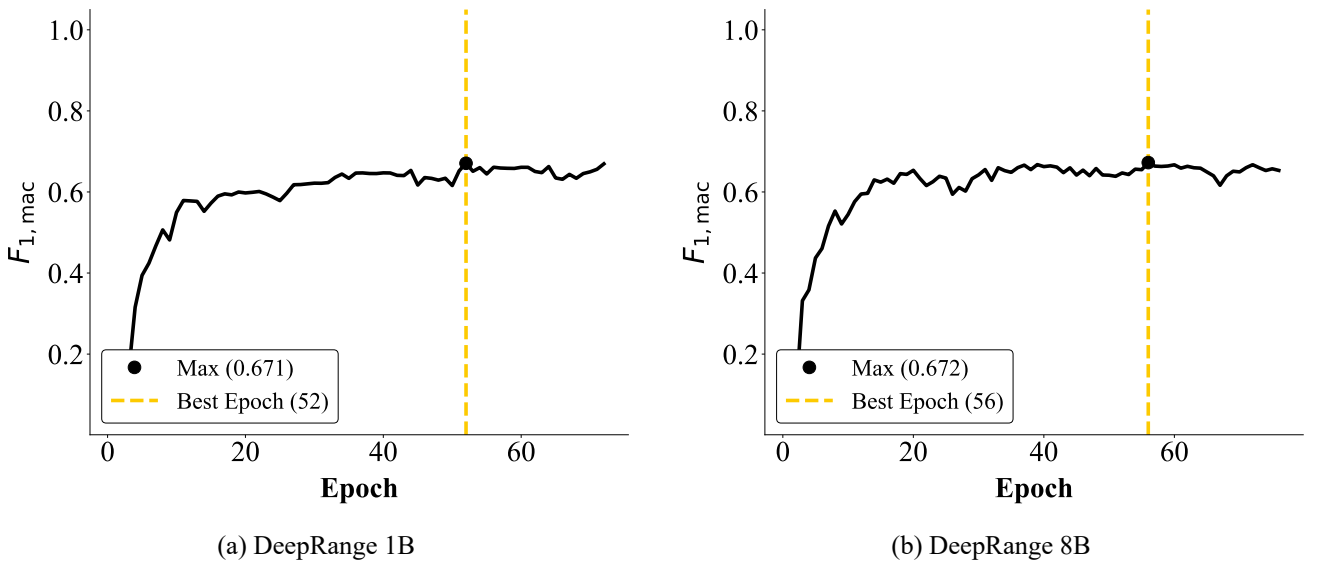


Figure 7-5: DeepRange 1B/8B validation curves for few-shot highway classification using a sequence length of 100 s.

Tab. 7-3 presents the results for a sequence length of 500 s, corresponding to only 1718 training samples

due to the longer sequence length. Despite having substantially fewer samples than both the 100 s full-shot and few-shot experiments, the results reveal that longer sequences partially compensate for reduced sample count, as DeepRange models achieve competitive Acc despite the severe data reduction. However, Acc_{bal} and $F_{1,mac}$ drop considerably across all models compared to the 100 s full-shot experiment. Class imbalance becomes more pronounced with fewer training samples. InceptionTime retains the highest Acc_{bal} and $F_{1,mac}$ so far and is the most consistent performer on minority classes. MiniRocket achieves the highest Acc and MCC , continuing the reversal first observed in the 100 s few-shot experiment. The DeepRange models show the largest performance degradation. DeepRange 1B $F_{1,mac}$ drops from 0.838 in the 100 s full-shot experiment to 0.615 here. Tab. 7-4 presents the most data-scarce setting, with only 169 training

Table 7-3: Highway classification results for input sequence length 500 s. Best metrics are highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Metrics</i>				
Acc	0.840	0.832	0.866	0.890
Acc_{bal}	0.607	0.577	0.657	0.617
$F_{1,mac}$	0.615	0.574	0.685	0.603
MCC	0.788	0.778	0.823	0.856
t_{train}	12.185 min	81.102 min	14.613 min	0.656 min
<i>Number of Samples</i>				
Training		1718		
Validation		366		
Testing		374		
<i>Detailed Results</i>				
Per-class Metrics		Tab. A-11		
Confusion Matrix	Fig. A-3a	Fig. A-3b	Fig. A-3c	Fig. A-3d
Learning Curves	—	—	—	—

samples. Both DeepRange models completely collapse, achieving $MCC = 0$ and $Acc = 0.307$, indicating degeneration to majority class prediction and a complete failure to learn discriminative features from such limited data. InceptionTime degrades more gracefully, retaining $F_{1,mac} = 0.441$ and $MCC = 0.526$. MiniRocket demonstrates the strongest data efficiency, achieving the highest results across all metrics including $MCC = 0.763$, confirming that its fixed random kernel design is highly robust to extreme data scarcity. These results establish a clear data requirement threshold for LLM-based architectures. Below approximately 1718 samples, DeepRange models fail to generalize, while MiniRocket remains effective even at 169 samples.

Table 7-4: Few-shot highway classification results for input sequence length 500 s. Best metrics are highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Metrics</i>				
Acc	0.307	0.307	0.615	0.821
Acc_{bal}	0.167	0.167	0.464	0.557
$F_{1,mac}$	0.078	0.078	0.441	0.543
MCC	0.000	0.000	0.526	0.763
t_{train}	1.153 min	5.084 min	1.561 min	0.077 min
<i>Number of Samples</i>				
Training		169		
Validation		366		
Testing		374		
<i>Detailed Results</i>				
Per-class Metrics		Tab. A-12		
Confusion Matrix	Fig. A-4a	Fig. A-4b	Fig. A-4c	Fig. A-4d
Learning Curves	—	—	—	—

7.2 Ambient Temperature Classification

Tab. 7-9 presents the full-shot ambient temperature classification results for a sequence length of 100 s with 288577 training samples. In contrast to all highway experiments, DeepRange 8B achieves the highest results across all metrics. In this experiment a larger LLM backbone benefits from additional training data. The smaller gap between Acc and Acc_{bal} compared to the highway task indicates that class imbalance is less severe in this dataset. Despite $Acc = 0.783$, MiniRocket yields $Acc_{bal} = 0.497$, near random on minority classes. DeepRange 8B requires $t_{train} = 689.006$ min, the longest training time across all experiments so far.

Table 7-5: Full-shot air temperature classification results for input sequence length 100 s. Best metrics are highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Metrics</i>				
Acc	0.924	0.933	0.904	0.783
Acc_{bal}	0.872	0.900	0.867	0.497
$F_{1,mac}$	0.892	0.912	0.868	0.525
MCC	0.887	0.902	0.859	0.676
t_{train}	107.554 min	689.006 min	234.539 min	92.591 min
<i>Number of Samples</i>				
Training		288577		
Validation		61835		
Testing		61845		
<i>Detailed Results</i>				
Per-class Metrics		Tab. A-13		
Confusion matrix	Fig. A-5a	Fig. A-5b	Fig. A-5c	Fig. A-5d
Learning curves		Fig. A-6	—	—

In the reduced training set in Tab. 7-10 with 28856 samples, InceptionTime achieves the highest results across all metrics with $Acc = 0.832$, $Acc_{bal} = 0.738$, $F_{1,mac} = 0.748$, and $MCC = 0.753$. Both DeepRange models degrade considerably. DeepRange 8B $F_{1,mac}$ drops from 0.912 to 0.735 and DeepRange 1B from 0.892 to 0.703. Notably, DeepRange 8B outperforms DeepRange 1B on Acc_{bal} and $F_{1,mac}$ in this setting, in contrast to the highway few-shot experiments where the two models performed identically. MiniRocket retains the fastest training time at 6.428 min but remains last on minority class metrics, confirming that its data efficiency advantage does not extend to this classification task.

Table 7-6: Few-shot air temperature classification results for input sequence length 100 s. Best metrics are highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Metrics</i>				
Acc	0.826	0.827	0.832	0.765
Acc_{bal}	0.662	0.704	0.738	0.473
$F_{1,mac}$	0.703	0.735	0.748	0.500
MCC	0.743	0.745	0.753	0.647
t_{train}	40.241 min	168.503 min	14.201 min	6.428 min
<i>Number of Samples</i>				
Training		28856		
Validation		61835		
Testing		61845		
<i>Detailed Results</i>				
Per-class Metrics		Tab. A-14		
Confusion matrix	Fig. A-7a	Fig. A-7b	Fig. A-7c	Fig. A-7d
Learning curves		Fig. A-8	—	—

Tab. 7-11 presents the full-shot results for a sequence length of 500 s with 49062 training samples. Despite having substantially fewer samples than the 100 s full-shot setting, the DeepRange models recover strongly, with DeepRange 1B achieving the highest $Acc = 0.906$ and $MCC = 0.860$, and DeepRange 8B the highest $F_{1,mac} = 0.857$. InceptionTime leads only on $Acc_{bal} = 0.840$, its narrowest margin of advantage across all settings. This confirms that longer sequences provide sufficient temporal context for DeepRange models to generalize effectively even with fewer samples. MiniRocket remains the fastest at 23.145 min but continues to underperform on minority class metrics with $Acc_{bal} = 0.562$.

Table 7-7: Full-shot air temperature classification results for input sequence length 500 s. Best metrics are highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Metrics</i>				
Acc	0.906	0.905	0.890	0.842
Acc_{bal}	0.817	0.829	0.840	0.562
$F_{1,mac}$	0.850	0.857	0.831	0.595
MCC	0.860	0.859	0.838	0.763
t_{train}	43.158 min	211.974 min	163.712 min	23.145 min
<i>Number of Samples</i>				
Training		49062		
Validation		10510		
Testing		10521		
<i>Detailed Results</i>				
Per-class Metrics	Tab. A-15			
Confusion matrix	Fig. A-9a	Fig. A-9b	Fig. A-9c	Fig. A-9d
Learning curves	Fig. A-10		—	—

The most striking result in this task appears in the 500 s few-shot setting in Tab. 7-12 with only 4904 training samples. DeepRange 8B completely collapses with $MCC = -0.004$, $Acc_{bal} = 0.166$, and $F_{1,mac} = 0.097$, performing below random on all minority class metrics. In stark contrast, DeepRange 1B achieves the strongest results across all metrics with $F_{1,mac} = 0.720$, $Acc_{bal} = 0.703$, and $MCC = 0.734$. This divergence between the two backbone sizes under identical data conditions is unique across all experiments and suggests a critical interaction between model capacity and data availability. At 4904 samples the 8B backbone is severely underconstrained, while the 1B backbone finds a generalizable solution. MiniRocket ranks second with $MCC = 0.693$, confirming its robustness under data scarcity, while InceptionTime achieves $MCC = 0.648$, placing third.

Table 7-8: Few-shot air temperature classification results for input sequence length 500 s. Best metrics are highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Metrics</i>				
Acc	0.821	0.397	0.758	0.797
Acc_{bal}	0.703	0.166	0.692	0.498
$F_{1,mac}$	0.720	0.097	0.607	0.532
MCC	0.734	-0.004	0.648	0.693
t_{train}	17.038 min	34.451 min	21.211 min	2.317 min
<i>Number of Samples</i>				
Training		4904		
Validation		10510		
Testing		10521		
<i>Detailed Results</i>				
Per-class Metrics	Tab. A-16			
Confusion matrix	Fig. A-11a	Fig. A-11b	Fig. A-11c	Fig. A-11d
Learning curves	Fig. A-20		—	—

7.3 Weather classification

Weather classification is additionally complicated by the low temporal resolution of weather labels, which are recorded only at hourly intervals. Since ET trip data is recorded at a higher frequency, weather labels are assigned by nearest timestamp mapping, introducing label noise that likely contributes to the substantially lower $F_{1,mac}$ values observed across all models and settings compared to the highway and temperature tasks. Tab. 7-9 presents the full-shot weather classification results for a sequence length of 100 s with 288715 training samples. Weather classification proves to be the most challenging task across all experiments, with $F_{1,mac}$ values substantially lower than both highway and temperature classification across all models and settings. DeepRange 1B achieves the highest $Acc = 0.910$ and $MCC = 0.814$, while InceptionTime leads on $Acc_{bal} = 0.700$ and $F_{1,mac} = 0.754$. DeepRange 8B performs poorly relative to DeepRange 1B despite identical training data, achieving only $Acc_{bal} = 0.387$ and $F_{1,mac} = 0.446$. This divergence indicates that weather classification patterns are more complex and less separable, and that the larger backbone does not generalize better on this task even with abundant data. MiniRocket achieves $Acc = 0.794$ but $Acc_{bal} = 0.211$, the lowest minority class performance observed across all experiments, confirming that fixed random kernels are fundamentally inadequate for this task. DeepRange 8B requires $t_{train} = 717.885$ min, the longest training time across all experiments.

Table 7-9: Weather classification results using a sequence length of 100 s. Best metrics are highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Metrics</i>				
Acc	0.910	0.843	0.897	0.794
Acc_{bal}	0.634	0.387	0.700	0.211
$F_{1,mac}$	0.719	0.446	0.754	0.202
MCC	0.814	0.667	0.788	0.554
t_{train}	214.816 min	717.885 min	227.258 min	27.361 min
<i>Number of Samples</i>				
Training		288715		
Validation		61863		
Testing		61878		
<i>Detailed Results</i>				
Per-class Metrics		Tab. A-17		
Confusion matrix	Fig. A-13a	Fig. A-13b	Fig. A-13c	Fig. A-13d
Learning curves	Fig. A-14		—	—

In the reduced training set setting in Tab. 7-10 with 28866 samples, all models degrade severely on minority class metrics. InceptionTime retains the highest $Acc_{bal} = 0.365$ and $F_{1,mac} = 0.406$, though these values indicate poor absolute performance. DeepRange 1B collapses on minority class metrics with $Acc_{bal} = 0.249$ and $F_{1,mac} = 0.268$, a dramatic drop from the full-shot setting. Notably, DeepRange 8B outperforms DeepRange 1B on Acc_{bal} and $F_{1,mac}$ in this setting despite underperforming in the full-shot experiment. MiniRocket achieves the highest $Acc = 0.791$ but $Acc_{bal} = 0.212$, confirming that it predicts majority classes almost exclusively regardless of data availability on this task.

Table 7-10: Few-shot weather classification results using a sequence length of 100 s. Best metrics are highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Metrics</i>				
Acc	0.783	0.774	0.774	0.791
Acc_{bal}	0.249	0.297	0.365	0.212
$F_{1,mac}$	0.268	0.333	0.406	0.206
MCC	0.519	0.512	0.540	0.539
t_{train}	50.665 min	178.142 min	24.865 min	4.059 min
<i>Number of Samples</i>				
Training		28866		
Validation		61863		
Testing		61878		
<i>Detailed Results</i>				
Per-class Metrics	Tab. A-18			
Confusion matrix	Fig. A-15a	Fig. A-15b	Fig. A-15c	Fig. A-15d
Learning curves	Fig. A-16		—	—

Tab. 7-11 presents the full-shot results for a sequence length of 500 s with 50989 training samples. InceptionTime achieves the highest results across all metrics with $Acc = 0.837$, $Acc_{bal} = 0.486$, $F_{1,mac} = 0.532$, and $MCC = 0.659$. The DeepRange models perform considerably worse than in the 100 s full-shot setting despite having comparable sample counts, with DeepRange 1B $F_{1,mac} = 0.304$ and DeepRange 8B $F_{1,mac} = 0.328$. This indicates that the DeepRange models do not benefit from longer sequences on the weather task, in contrast to the temperature and highway tasks. MiniRocket remains last on minority class metrics with $Acc_{bal} = 0.224$.

Table 7-11: Weather classification results using a sequence length of 500 s. Best metrics are highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Metrics</i>				
Acc	0.795	0.818	0.837	0.825
Acc_{bal}	0.282	0.300	0.486	0.224
$F_{1,mac}$	0.304	0.328	0.532	0.223
MCC	0.555	0.597	0.659	0.616
t_{train}	63.091 min	332.384 min	227.226 min	21.268 min
<i>Number of Samples</i>				
Training		50989		
Validation		10923		
Testing		10934		
<i>Detailed Results</i>				
Per-class Metrics	Tab. A-19			
Confusion matrix	Fig. A-17a	Fig. A-17b	Fig. A-17c	Fig. A-17d
Learning curves	Fig. A-18		—	—

The 500 s few-shot setting in Tab. 7-12 with 5098 training samples confirms the patterns observed across previous settings. InceptionTime achieves the highest $Acc_{bal} = 0.353$ and $F_{1,mac} = 0.354$, while MiniRocket leads on $Acc = 0.818$ and $MCC = 0.596$. All $F_{1,mac}$ values remain below 0.36 across all models, establishing that weather classification under data scarcity is not solved by any of the evaluated architectures. DeepRange 1B and DeepRange 8B perform nearly identically on all metrics, with no meaningful difference between backbone sizes at this sample count.

Table 7-12: Few-shot weather classification results using a sequence length of 500 s. Best metrics are highlighted in bold.

	DeepRange 1B	DeepRange 8B	InceptionTime	MiniRocket
<i>Metrics</i>				
Acc	0.762	0.768	0.781	0.818
Acc_{bal}	0.229	0.235	0.353	0.224
$F_{1,mac}$	0.231	0.236	0.354	0.226
MCC	0.471	0.477	0.524	0.596
t_{train}	19.955 min	75.719 min	22.290 min	2.657 min
<i>Number of Samples</i>				
Training		5098		
Validation		10923		
Testing		10934		
<i>Detailed Results</i>				
	Tab. A-20			
Confusion matrix	Fig. A-19a	Fig. A-19b	Fig. A-19c	Fig. A-19d
Learning curves	Fig. ??		–	–

7.4 Analysis of robustness and scalability

The results across all 12 experiments establish clear and size-dependent data requirement thresholds for the DeepRange models. Both models collapse completely at 169 samples, achieving $MCC = 0.000$ on the highway 500 s reduced training set, indicating degeneration to majority class prediction. DeepRange 8B exhibits a second collapse at 4904 samples on the temperature 500 s reduced training set where $MCC = -0.004$ and $F_{1,mac} = 0.097$, while DeepRange 1B achieves the strongest results across all models in the same setting with $F_{1,mac} = 0.720$ and $MCC = 0.734$. This establishes a size-dependent threshold. DeepRange 1B requires approximately 5000 samples to generalize effectively, while DeepRange 8B requires approximately 28000 to 50000 samples. Below these thresholds, InceptionTime and MiniRocket are the preferred architectures. InceptionTime due to its superior minority class performance and MiniRocket due to its extreme data efficiency and speed.

Above approximately 50000 samples on tasks with sufficient class separability, DeepRange 8B becomes competitive with InceptionTime on $F_{1,mac}$, and above approximately 288000 samples it outperforms all models. In the temperature full-shot 100 s setting with 288577 samples, DeepRange 8B achieves $F_{1,mac} = 0.912$ and $Acc_{bal} = 0.900$, surpassing InceptionTime.

Increasing the sequence length from 100 s to 500 s reduces training sample counts by factors of 4 to 6 across all tasks. The effect on $F_{1,mac}$ is strongly model and task dependent. DeepRange 1B degrades most severely on weather classification, while remaining competitive on temperature classification. InceptionTime degrades moderately across all tasks, with the largest drop on weather and smallest on temperature. MiniRocket is the only model that improves with longer sequences on temperature ($\Delta F_{1,mac} = +0.070$)

and weather, while all other models degrade. This indicates that longer sequences provide richer temporal context that fixed random kernels can exploit more effectively than gradient-based models, which require more samples per epoch to optimize their parameters.

InceptionTime is the most robust model across all tasks and settings, achieving the highest $F_{1,\text{mac}}$ in 8 out of 12 experiments and never collapsing below $F_{1,\text{mac}} = 0.354$. It is the only model that consistently handles minority classes across all three tasks and both sequence lengths, making it the most reliable architecture for ET scenario classification under varying conditions. DeepRange 1B is the second most robust model, outperforming DeepRange 8B in 6 out of 8 reduced training set experiments and never collapsing on tasks with more than 4904 samples. MiniRocket demonstrates strong robustness on Acc and MCC under data scarcity but fails consistently on $F_{1,\text{mac}}$ and Acc_{bal} across all tasks, confirming that its robustness is limited to majority class prediction. DeepRange 8B is the least robust model, collapsing in 2 out of 12 settings and underperforming DeepRange 1B in all reduced training set experiments except the temperature 100 s reduced training set.

MiniRocket is the most computationally efficient model by a substantial margin, completing training in under 5 min in all reduced training set settings and under 30 min in all full-shot settings. DeepRange 1B offers the best trade-off among LLM-based architectures, requiring less than half the training time of DeepRange 8B in all experiments while achieving superior $F_{1,\text{mac}}$ in 7 out of 12 settings. DeepRange 8B requires up to $t_{\text{train}} = 717.885$ min on the weather full-shot 100 s setting and $t_{\text{train}} = 689.006$ min on the temperature full-shot 100 s setting, making it computationally prohibitive for large datasets without demonstrably superior performance. InceptionTime provides a strong balance between performance and training time, training faster than both DeepRange models in the majority of settings while achieving the highest $F_{1,\text{mac}}$ in 8 out of 12 experiments. These results establish that DeepRange 1B is the most practical LLM-based architecture for ET scenario classification, offering competitive performance at substantially lower computational cost than DeepRange 8B, while InceptionTime remains the strongest performer on minority class metrics across all conditions.

8 Conclusion and Outlook

The findings of this thesis establish a foundation for the application of LLM-based architectures to electric truck multivariate time series classification, and raise several concrete directions for future research that address both the limitations identified in this work and the broader potential of the proposed approach. A central limitation revealed by the experimental evaluation is the data dependency of DeepRange. Bridging this gap is therefore a critical prerequisite for the practical deployment of LLM-based architectures in industrial settings where labeled data is costly to acquire. Parameter-efficient fine-tuning methods a promising direction in this regard. A systematic evaluation of parameter-efficient fine-tuning strategies on the electric truck scenario classification tasks defined in this work would directly address the identified data thresholds and could substantially extend the applicability of DeepRange to data-scarce settings.

The weather condition classification task proved to be the most challenging across all 12 experimental settings, with all models achieving substantially lower $F_{1,\text{mac}}$ compared to highway type and ambient temperature classification, regardless of data availability or sequence length. Two factors contribute to this increased difficulty. Weather condition classification involves nine classes compared to six for highway type and ambient temperature classification, increasing the complexity of the decision boundary and the minimum data requirements per class. Further condition labels are assigned at hourly resolution from external weather station data, introducing label noise at class boundaries that cannot be resolved through model scaling or additional training samples. The fundamental challenge lies not in the architecture or the quantity of training data, but in the number of classes and the quality of the labels themselves. Future work should focus on improving the labeling procedure for weather condition classification, for example by incorporating vehicle-mounted sensors such as wiper activity and road surface temperature to derive higher-resolution and more consistent labels directly from the vehicle data. A refined labeling strategy of this kind would reduce label noise, improve class separability, and benefit all evaluated architectures on this task.

The channel-independent processing design of DeepRange is a deliberate architectural choice that enables generalization across electric truck variants with differing sensor configurations without requiring architectural modifications. However, cross-vehicle transferability was not empirically evaluated in this work, as the available dataset did not provide sufficient coverage of distinct vehicle configurations to conduct a controlled transfer experiment. Evaluating whether a DeepRange model trained on one fleet configuration generalizes to another without full retraining is a practically highly relevant research direction, as it directly addresses the per-variant maintenance bottleneck that motivates this thesis. A positive result in this direction would constitute strong evidence for the scalability argument of LLM-based architectures in the electric truck context, and would represent a significant step toward a truly unified classification architecture across heterogeneous fleets.

The evaluation in this thesis focused on scenario classification as a foundational preprocessing task for range prediction pipelines. The extension of this work is to evaluate DeepRange across the full range of tasks that constitute the pipeline, including driving behavior classification, energy consumption prediction, and state of charge estimation. In particular, energy consumption prediction and state of charge estimation involve continuous output variables rather than discrete class labels, which would require extending the classification head of DeepRange to a regression formulation.

Finally, the LLM backbone comparison in this work was limited to two configurations of 1 billion and 8 billion parameters, motivated by the hardware constraints of a single NVIDIA A100 80 GB GPU. The results show that backbone size alone does not determine classification performance. Future work should investigate a broader range of backbone sizes, including sub-billion parameter configurations that may offer competitive performance at substantially lower computational cost, as well as more recent efficient backbone architectures that have emerged since the completion of this work. A finer-grained analysis of the relationship between backbone capacity, data requirements, and classification performance would provide more principled guidance for model selection in future deployments and contribute to a deeper theoretical understanding of how pre-trained language model representations transfer to structured time series data.

Annex

A.1 Methodology

Table A-1: Overview of UEA datasets, including data dimensions, training and test cases.¹¹⁷

Dataset	Train Samples	Test Samples	Dimensions	Length	Classes
ArticulatoryWordRecognition	275	300	9	144	25
AtrialFibrillation	15	15	2	640	3
BasicMotions	40	40	6	100	4
CharacterTrajectories	1422	1436	3	182	20
Cricket	108	72	6	1197	12
DuckDuckGeese	60	40	1345	270	5
EigenWorms	128	131	6	17984	5
Epilepsy	137	138	3	206	4
EthanolConcentration	261	263	3	1751	4
ERing	30	30	4	65	6
FaceDetection	5890	3524	144	62	2
FingerMovements	316	100	28	50	2
HandMovementDirection	320	147	10	400	4
Handwriting	150	850	3	152	26
Heartbeat	204	205	61	405	2
JapaneseVowels	270	370	12	29	9
Libras	180	180	2	45	15
LSST	2459	2466	6	36	14
InsectWingbeat	30000	20000	200	78	10
MotorImagery	278	100	64	3000	2
NATOPS	180	180	24	51	6
PenDigits	7494	3498	2	8	10
PEMS-SF	267	173	963	144	7
Phoneme	3315	3353	11	217	39
RacketSports	151	152	6	30	4
SelfRegulationSCP1	268	293	6	896	2
SelfRegulationSCP2	200	180	7	1152	2
SpokenArabicDigits	6599	2199	13	93	10
StandWalkJump	12	15	4	2500	3
UWaveGestureLibrary	120	320	3	315	8

¹¹⁷Bagnall, A. et al.: The UEA archive (2018), p. 3, tab. 3.

Table A-2: Defined model hyperparameters for each UEA dataset.

Dataset	Patch Length	Patch Stride	Batch Size
ArticularyWordRecognition	20	10	64
AtrialFibrillation	50	25	5
BasicMotions	16	8	10
CharacterTrajectories	128	64	256
Cricket	128	64	16
DuckDuckGeese	32	16	10
EigenWorms	256	128	2
Epilepsy	16	8	64
EthanolConcentration	128	64	16
ERing	8	4	8
FaceDetection	8	4	256
FingerMovements	16	8	64
HandMovementDirection	64	32	64
Handwriting	32	16	32
Heartbeat	64	32	32
JapaneseVowels	8	4	64
Libras	8	4	64
LSST	2	2	256
InsectWingbeat	16	8	128
MotorImagery	128	64	16
NATOPS	16	8	32
PenDigits	2	1	256
PEMS-SF	16	8	32
Phoneme	16	8	256
RacketSports	16	8	64
SelfRegulationSCP1	128	64	16
SelfRegulationSCP2	128	64	16
SpokenArabicDigits	16	8	256
StandWalkJump	128	64	1
UWaveGestureLibrary	32	16	32

Table A-3: OneFitsAll results on normalized UEA datasets.

Dataset	<i>Acc</i>	<i>Acc</i>_{bal}	<i>F</i>_{1,mac}
ArticularyWordRecognition	0.980	0.980	0.980
AtrialFibrillation	0.333	0.333	0.288
BasicMotions	1.000	1.000	1.000
CharacterTrajectories	—	—	—
Cricket	0.944	0.944	0.940
DuckDuckGeese	0.520	0.520	0.497
EigenWorms	0.473	0.449	0.478
Epilepsy	0.862	0.863	0.855
EthanolConcentration	0.297	0.297	0.297
ERing	0.937	0.937	0.937
FaceDetection	0.652	0.652	0.652
FingerMovements	0.580	0.581	0.579
HandMovementDirection	0.541	0.524	0.543
Handwriting	0.301	0.297	0.274
Heartbeat	0.717	0.599	0.699
JapaneseVowels	—	—	—
Libras	0.800	0.800	0.800
LSST	0.456	0.313	0.372
InsectWingbeat	—	—	—
MotorImagery	0.480	0.480	0.466
NATOPS	0.939	0.939	0.939
PenDigits	0.979	0.979	0.979
PEMS-SF	0.821	0.821	0.815
Phoneme	—	—	—
RacketSports	0.770	0.786	0.771
SelfRegulationSCP1	0.898	0.898	0.897
SelfRegulationSCP2	0.483	0.483	0.483
SpokenArabicDigits	—	—	—
StandWalkJump	0.267	0.267	0.265
UWaveGestureLibrary	0.875	0.875	0.875
$\mu \pm \sigma$	0.676 ± 0.241	0.665 ± 0.250	0.667 ± 0.250

Table A-4: Time-LLM results on normalized UEA datasets.

Dataset	Acc	Acc_{bal}	$F_{1,mac}$
ArticularyWordRecognition	0.983	0.983	0.984
AtrialFibrillation	0.200	0.200	0.172
BasicMotions	0.975	0.975	0.975
CharacterTrajectories	—	—	—
Cricket	0.958	0.958	0.959
DuckDuckGeese	0.580	0.580	0.567
EigenWorms	0.420	0.200	0.248
Epilepsy	0.899	0.899	0.896
EthanolConcentration	0.262	0.263	0.253
ERing	0.900	0.900	0.900
FaceDetection	0.651	0.651	0.651
FingerMovements	0.550	0.552	0.547
HandMovementDirection	0.514	0.523	0.513
Handwriting	0.305	0.301	0.280
Heartbeat	0.693	0.615	0.692
JapaneseVowels	—	—	—
Libras	0.844	0.844	0.839
LSST	0.655	0.497	0.630
InsectWingbeat	—	—	—
MotorImagery	0.590	0.590	0.563
NATOPS	0.939	0.939	0.939
PenDigits	0.982	0.982	0.982
PEMS-SF	0.861	0.861	0.862
Phoneme	—	—	—
RacketSports	0.750	0.767	0.746
SelfRegulationSCP1	0.788	0.789	0.783
SelfRegulationSCP2	0.439	0.439	0.438
SpokenArabicDigits	—	—	—
StandWalkJump	0.333	0.333	0.323
UWaveGestureLibrary	0.900	0.900	0.901
$\mu \pm \sigma$	0.679 ± 0.244	0.662 ± 0.260	0.666 ± 0.259

Table A-5: LLM-Few results on normalized UEA datasets.

Dataset	<i>Acc</i>	<i>Acc</i>_{bal}	<i>F</i>_{1,mac}
ArticularyWordRecognition	0.987	0.987	0.987
AtrialFibrillation	0.267	0.267	0.255
BasicMotions	1.000	1.000	1.000
CharacterTrajectories	—	—	—
Cricket	1.000	1.000	1.000
DuckDuckGeese	0.580	0.580	0.582
EigenWorms	0.710	0.596	0.686
Epilepsy	0.942	0.942	0.941
EthanolConcentration	0.262	0.262	0.259
ERing	0.915	0.915	0.915
FaceDetection	0.652	0.652	0.652
FingerMovements	0.480	0.482	0.476
HandMovementDirection	0.311	0.330	0.311
Handwriting	0.473	0.470	0.445
Heartbeat	0.693	0.620	0.693
JapaneseVowels	—	—	—
Libras	0.867	0.867	0.865
LSST	0.648	0.511	0.635
InsectWingbeat	—	—	—
MotorImagery	0.490	0.490	0.475
NATOPS	0.917	0.917	0.916
PenDigits	0.986	0.986	0.986
PEMS-SF	0.798	0.796	0.796
Phoneme	—	—	—
RacketSports	0.849	0.860	0.846
SelfRegulationSCP1	0.877	0.877	0.877
SelfRegulationSCP2	0.489	0.489	0.487
SpokenArabicDigits	—	—	—
StandWalkJump	0.467	0.467	0.463
UWaveGestureLibrary	0.891	0.891	0.891
$\mu \pm \sigma$	0.702 ± 0.238	0.690 ± 0.241	0.698 ± 0.241

Table A-6: Deep Range V1 results on normalized UEA datasets.

Dataset	Acc	Acc_{bal}	$F_{1,mac}$
ArticularyWordRecognition	0.937	0.937	0.938
AtrialFibrillation	0.267	0.267	0.272
BasicMotions	1.000	1.000	1.000
CharacterTrajectories	—	—	—
Cricket	0.847	0.847	0.846
DuckDuckGeese	0.460	0.460	0.451
EigenWorms	0.420	0.200	0.248
Epilepsy	0.913	0.913	0.910
EthanolConcentration	0.251	0.250	0.101
ERing	0.915	0.915	0.915
FaceDetection	0.627	0.627	0.626
FingerMovements	0.440	0.441	0.440
HandMovementDirection	0.243	0.245	0.244
Handwriting	0.345	0.339	0.328
Heartbeat	0.673	0.596	0.674
JapaneseVowels	—	—	—
LSST	0.645	0.475	0.621
InsectWingbeat	—	—	—
MotorImagery	0.460	0.460	0.389
NATOPS	0.889	0.889	0.889
PenDigits	0.985	0.985	0.985
PEMS-SF	0.809	0.814	0.811
Phoneme	—	—	—
RacketSports	0.842	0.853	0.841
SelfRegulationSCP1	0.857	0.857	0.856
SelfRegulationSCP2	0.467	0.467	0.467
SpokenArabicDigits	—	—	—
StandWalkJump	0.333	0.333	0.167
UWaveGestureLibrary	0.744	0.744	0.739
$\mu \pm \sigma$	0.643 ± 0.248	0.625 ± 0.262	0.619 ± 0.278

Table A-7: Deep Range V2 results on normalized UEA datasets.

Dataset	Acc	Acc_{bal}	$F_{1,mac}$
ArticularyWordRecognition	0.990	0.990	0.990
AtrialFibrillation	0.400	0.400	0.407
BasicMotions	0.875	0.875	0.861
CharacterTrajectories	—	—	—
Cricket	0.972	0.972	0.972
DuckDuckGeese	0.520	0.520	0.515
EigenWorms	0.466	0.378	0.449
Epilepsy	0.862	0.865	0.862
EthanolConcentration	0.270	0.270	0.271
ERing	0.948	0.948	0.948
FaceDetection	0.661	0.661	0.661
FingerMovements	0.480	0.481	0.479
HandMovementDirection	0.446	0.449	0.453
Handwriting	0.269	0.263	0.250
Heartbeat	0.688	0.568	0.670
JapaneseVowels	—	—	—
Libras	0.900	0.900	0.898
LSST	0.636	0.489	0.617
InsectWingbeat	—	—	—
MotorImagery	0.520	0.520	0.520
NATOPS	0.917	0.917	0.916
PenDigits	0.983	0.983	0.983
PEMS-SF	0.879	0.878	0.877
Phoneme	—	—	—
RacketSports	0.783	0.795	0.784
SelfRegulationSCP1	0.874	0.874	0.873
SelfRegulationSCP2	0.528	0.528	0.528
SpokenArabicDigits	—	—	—
StandWalkJump	0.333	0.333	0.262
UWaveGestureLibrary	0.881	0.881	0.881
$\mu \pm \sigma$	0.683 ± 0.237	0.670 ± 0.245	0.677 ± 0.242

A.2 Implementation

Table A-8: ET data features.

ID	Name	Unit	f_{sig}	Description
1	v_id	–	–	Vehicle identifier
2	signal_time	–	–	Signal time
3	signal_ts	ns	–	Unix timestamp
4	accelpdlposn_cval	%	10	Accelerator pedal position
5	actdrvtrnpwrprc_cval	%	10	Actual to max drivetrain power
6	actualdevoltage_pti1	V	1	DC link voltage of PTI1 inverter
7	actualspeed_pti1	rpm	10	Motor speed measured at PTI1 inverter
8	actualtorque_pti1	%	10	Motor torque output of PTI1 inverter
9	airtempinsd_cval_hvac	°C	1	Cabin air temperature
10	airtempinsd_rq	°C	1	Cabin air temperature setpoint
11	airtempoutsd_cval_cpc	°C	1	Ambient air temperature
12	altitude_cval_ippc	m	0.2	Vehicle altitude from GPS
13	brk_stat_brcl	–	–	Brake chopper BRC1 status
14	brktempra_cval	°C	0.1	Rear axle brake temperature
15	bs_brk_cval	%	1	Brake pedal position
16	currpwr_contendrnbrkresist_cval	kW	–	Power dissipated in brake resistor
17	elcomp_pwrcons_cval	kW	–	Power electric compressor
18	emot_pwr_cval	kW	10	Actual electrical power of all e-motors
19	epto_pwr_cval	kW	–	Power of the electric PTO
20	hirestotalvehdist_cval_icuc	km	1	High-resolution total vehicle distance
21	hv_bat_dc_momvolt_cval_bms1	V	10	Instantaneous HV battery voltage
22	hv_bat_soc_cval_bms1	%	1	HV battery state of charge
23	hv_batavcelltemp_cval_bms1	°C	1	Average HV battery cell temperature
24	hv_batcurr_cval_bms1	A	10	HV battery current
25	hv_batisores_cval_e2e	Ω	1	HV insulation resistance
26	hv_batmaxchrgpwrlim_cval_1	kW	1	Max HV battery charge power limit
27	hv_batmaxdischrgpwrlim_cval_1	kW	1	Max HV battery discharge power limit
28	hv_batmomavldischrgen_cval_1	kWh	0.1	HV battery discharge energy
29	hv_batpwr_cval_bms1	kW	10	HV battery power
30	hv_curr_cval_dcl1	A	0.1	DC/DC converter HV current
31	hv_dclink_volt_cval_dcl1	V	0.1	DC/DC converter HV link voltage
32	hv_ptc_cabin1_pwr_cval	kW	10	Cabin HV PTC 1 power
33	hv_pwr_cval_dcl1	kW	10	DC/DC converter HV power
34	latitude_cval_ippc	deg	0.2	Latitude
35	longitude_cval_ippc	deg	0.2	Longitude
36	lv_conv_pwr_cval_dcl1	kW	0.1	DC/DC converter LV power
37	maxrecupprc_cval	%	10	Actual to max regen braking power

Continued on next page

ID	Name	Unit	f_{sig}	Description
38	maxtracpwrpct_cval	%	10	Actual to max traction power
39	motortemperature_pti1	°C	1	PTI1 motor temperature
40	powerstagetemperature_pti1	°C	1	PTI1 inverter temperature
41	rmsmotorcurrent_pti1	A	1	PTI1 AC motor current RMS
42	roadgrad_cval_pt	%	1	Road grade
43	selgr_rq_pt	–	–	Selected gear
44	txoiltemp_cval_tcm	°C	10	Transmission oil temperature
45	vehspd_cval_cpc	km/h	1	Vehicle speed
46	vehweight_cval_pt	t	0.1	Vehicle weight

A.3 Highway Classification using a Sequence Length of 100 s

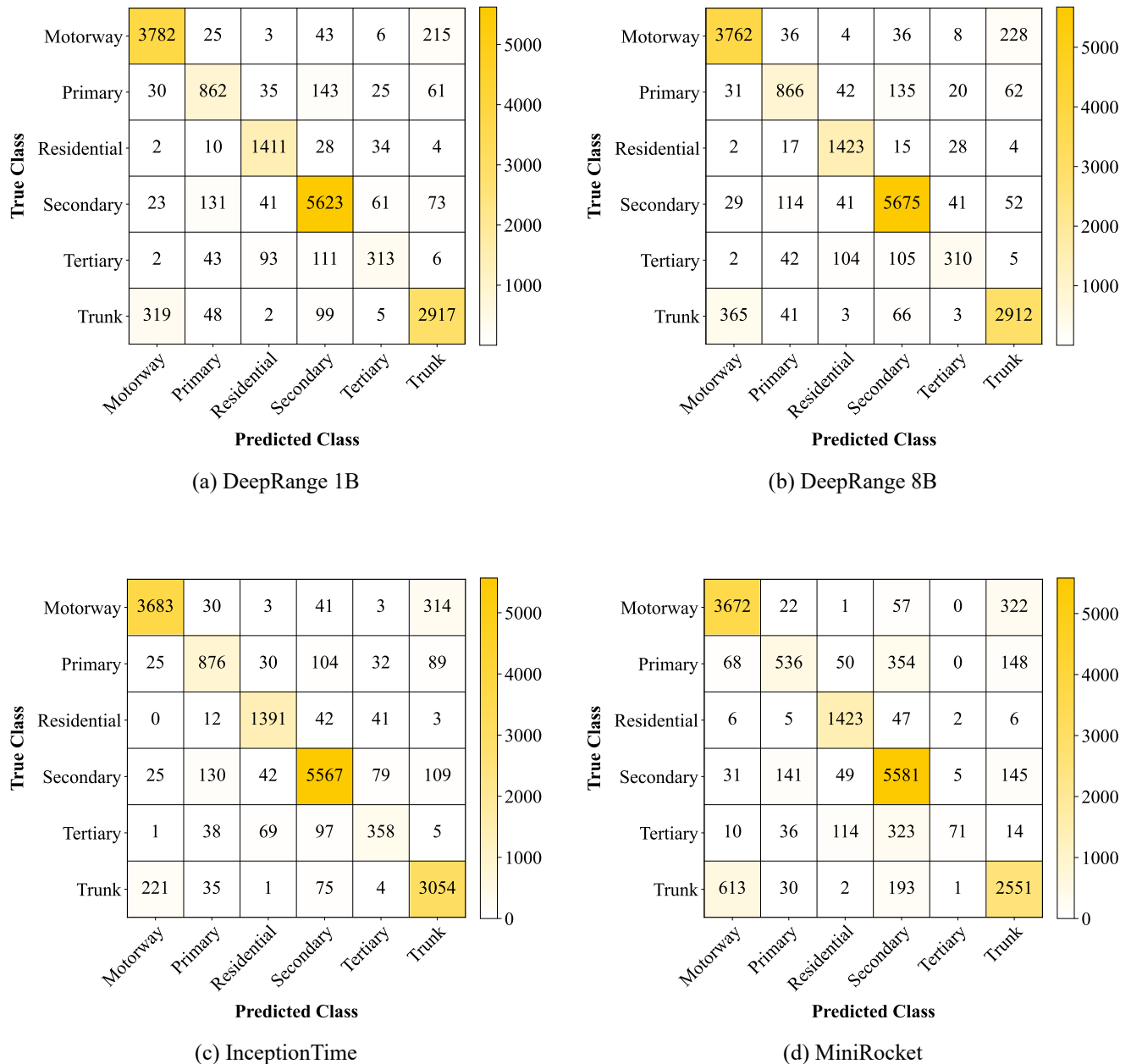




Table A-9: Per-class metrics for highway classification using a sequence length of 100 s.

	Motorway	Primary	Residential	Secondary	Tertiary	Trunk
<i>DeepRange 1B</i>						
<i>Acc</i>	0.928	0.746	0.948	0.945	0.551	0.860
<i>Pr</i>	0.910	0.770	0.890	0.930	0.705	0.890
<i>R</i>	0.928	0.746	0.948	0.945	0.551	0.860
<i>F₁</i>	0.919	0.758	0.918	0.937	0.619	0.875
<i>DeepRange 8B</i>						
<i>Acc</i>	0.923	0.749	0.956	0.953	0.546	0.859
<i>Pr</i>	0.898	0.776	0.880	0.941	0.756	0.892
<i>R</i>	0.923	0.749	0.956	0.953	0.546	0.859
<i>F₁</i>	0.910	0.762	0.916	0.947	0.634	0.875
<i>InceptionTime</i>						
<i>Acc</i>	0.904	0.758	0.934	0.935	0.630	0.901
<i>Pr</i>	0.931	0.781	0.906	0.939	0.692	0.855
<i>R</i>	0.904	0.758	0.934	0.935	0.630	0.901
<i>F₁</i>	0.917	0.769	0.920	0.937	0.660	0.877
<i>MiniRocket</i>						
<i>Acc</i>	0.901	0.464	0.956	0.938	0.125	0.753
<i>Pr</i>	0.835	0.696	0.868	0.851	0.899	0.801
<i>R</i>	0.901	0.464	0.956	0.938	0.125	0.753
<i>F₁</i>	0.867	0.557	0.910	0.892	0.219	0.776
Support	4074	1156	1489	5952	568	3390

A.4 Few-shot Highway Classification using a Sequence Length of 100 s

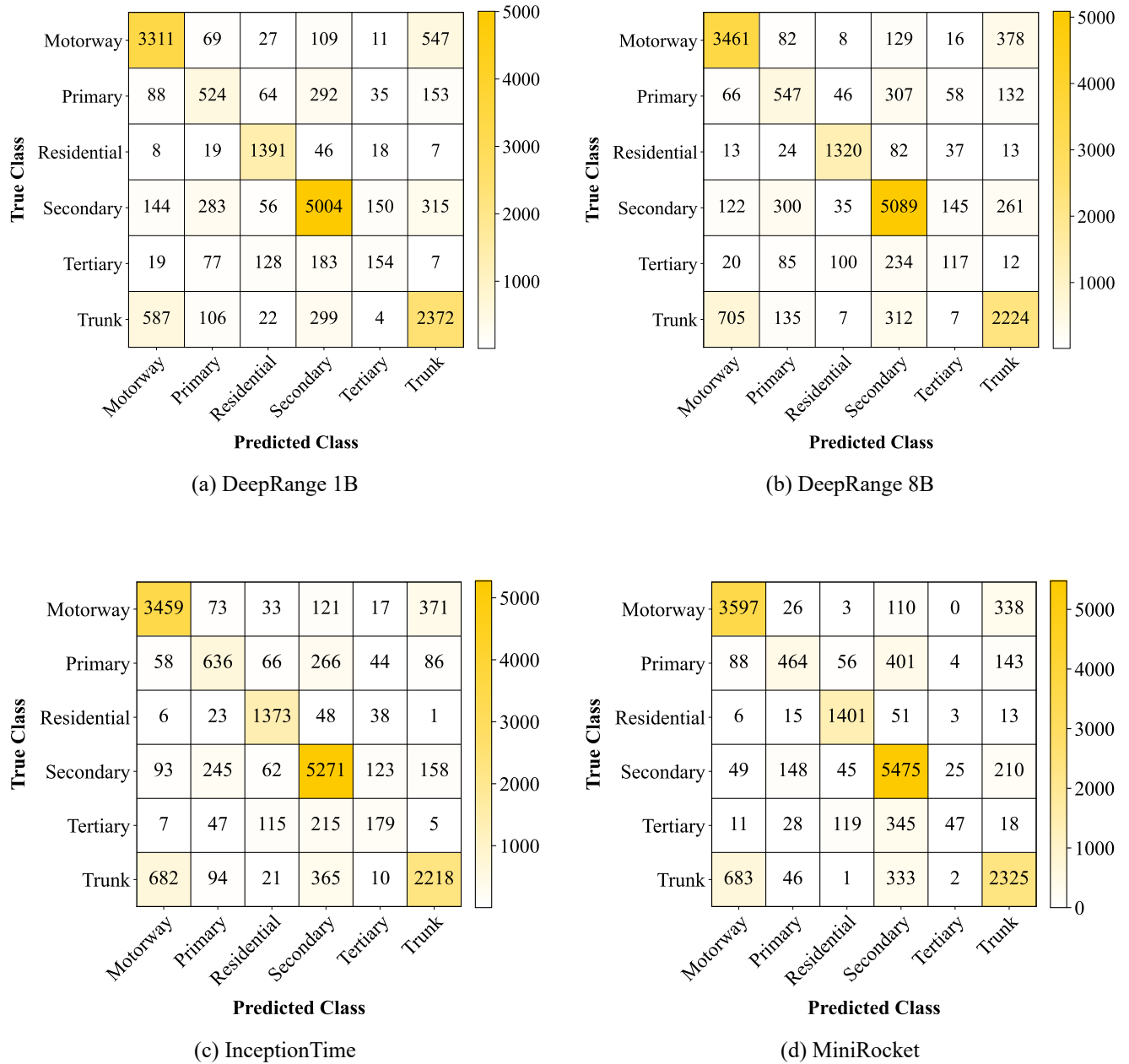
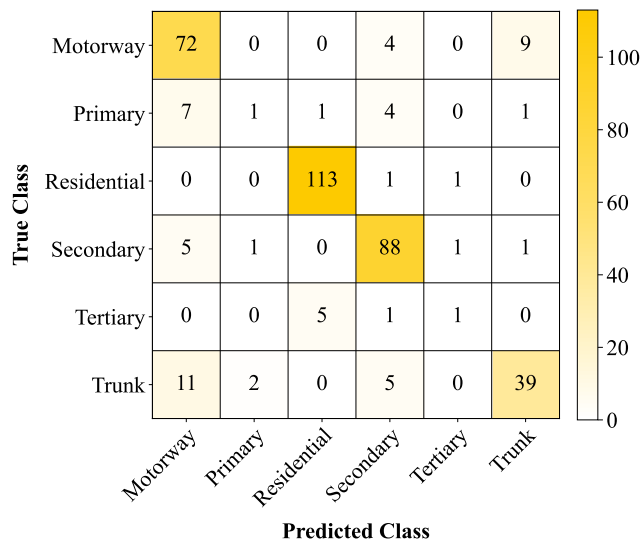


Figure A-2: Confusion matrices for few-shot highway classification using a sequence length of 100 s.

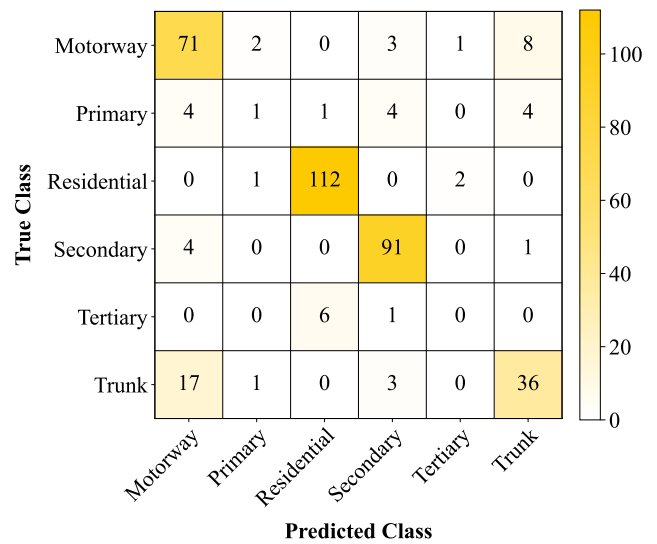
Table A-10: Per-class metrics for few-shot highway classification using a sequence length of 100 s.

	Motorway	Primary	Residential	Secondary	Tertiary	Trunk
<i>DeepRange 1B</i>						
<i>Acc</i>	0.813	0.453	0.934	0.841	0.271	0.700
<i>Pr</i>	0.796	0.486	0.824	0.843	0.414	0.697
<i>R</i>	0.813	0.453	0.934	0.841	0.271	0.700
<i>F₁</i>	0.805	0.469	0.876	0.842	0.328	0.699
<i>DeepRange 8B</i>						
<i>Acc</i>	0.850	0.473	0.887	0.855	0.206	0.656
<i>Pr</i>	0.789	0.466	0.871	0.827	0.308	0.736
<i>R</i>	0.850	0.473	0.887	0.855	0.206	0.656
<i>F₁</i>	0.818	0.470	0.879	0.841	0.247	0.694
<i>InceptionTime</i>						
<i>Acc</i>	0.849	0.550	0.922	0.886	0.315	0.654
<i>Pr</i>	0.803	0.569	0.822	0.839	0.436	0.781
<i>R</i>	0.849	0.550	0.922	0.886	0.315	0.654
<i>F₁</i>	0.826	0.559	0.869	0.861	0.366	0.712
<i>MiniRocket</i>						
<i>Acc</i>	0.883	0.401	0.941	0.920	0.083	0.686
<i>Pr</i>	0.811	0.638	0.862	0.815	0.580	0.763
<i>R</i>	0.883	0.401	0.941	0.920	0.083	0.686
<i>F₁</i>	0.846	0.493	0.900	0.864	0.145	0.722
Support	4074	1156	1489	5952	568	3390

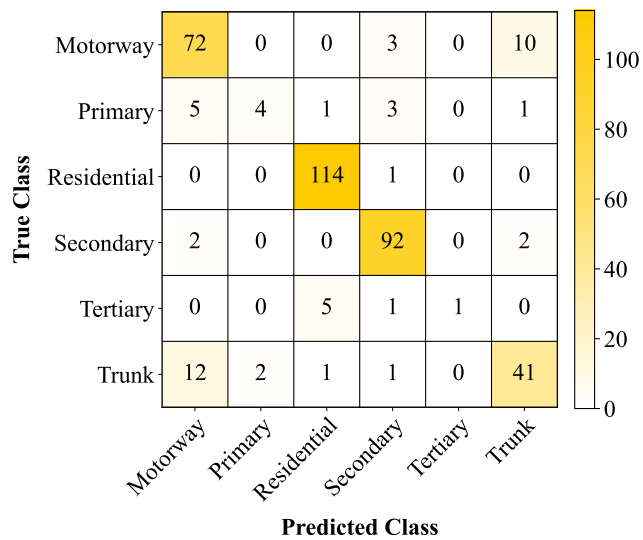
A.5 Highway Classification using a Sequence Length of 500 s



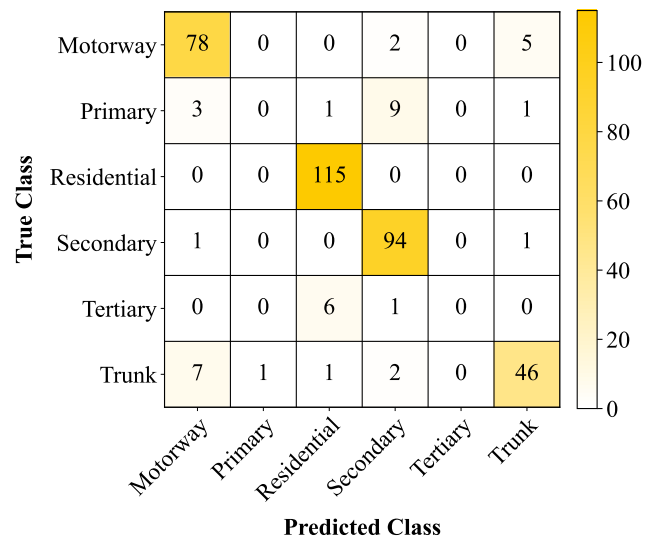
(a) DeepRange 1B



(b) DeepRange 8B



(c) InceptionTime



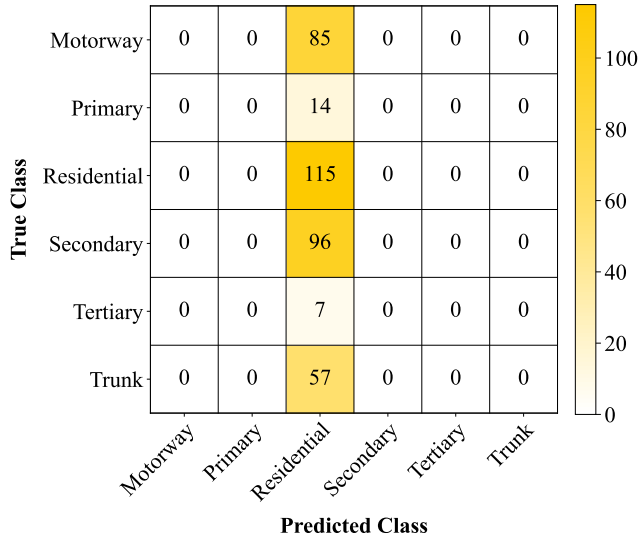
(d) MiniRocket

Figure A-3: Confusion matrices for highway classification using a sequence length of 500 s.

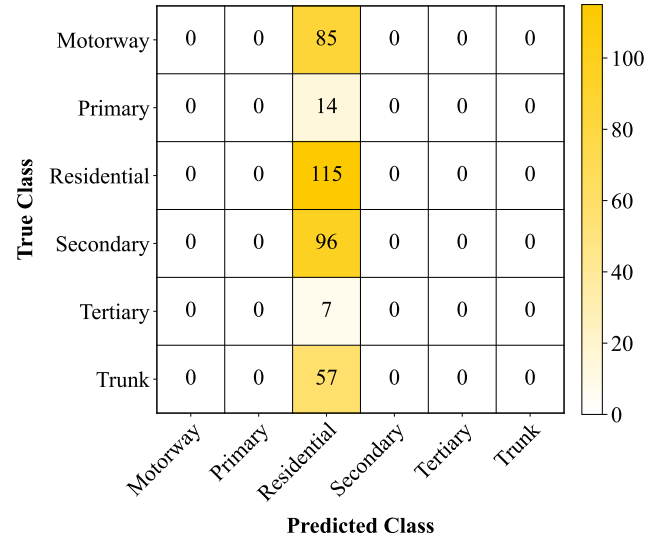
Table A-11: Per-class metrics for highway classification using a sequence length of 500 s.

	Motorway	Primary	Residential	Secondary	Tertiary	Trunk
<i>DeepRange 1B</i>						
<i>Acc</i>	0.847	0.071	0.983	0.917	0.143	0.684
<i>P</i>	0.758	0.250	0.950	0.854	0.333	0.780
<i>R</i>	0.847	0.071	0.983	0.917	0.143	0.684
<i>F₁</i>	0.800	0.111	0.966	0.884	0.200	0.729
<i>DeepRange 8B</i>						
<i>Acc</i>	0.835	0.071	0.974	0.948	0.000	0.632
<i>P</i>	0.740	0.200	0.941	0.892	0.000	0.735
<i>R</i>	0.835	0.071	0.974	0.948	0.000	0.632
<i>F₁</i>	0.785	0.105	0.957	0.919	0.000	0.679
<i>InceptionTime</i>						
<i>Acc</i>	0.847	0.286	0.991	0.958	0.143	0.719
<i>P</i>	0.791	0.667	0.942	0.911	1.000	0.759
<i>R</i>	0.847	0.286	0.991	0.958	0.143	0.719
<i>F₁</i>	0.818	0.400	0.966	0.934	0.250	0.739
<i>MiniRocket</i>						
<i>Acc</i>	0.918	0.000	1.000	0.979	0.000	0.807
<i>P</i>	0.876	0.000	0.935	0.870	0.000	0.868
<i>R</i>	0.918	0.000	1.000	0.979	0.000	0.807
<i>F₁</i>	0.897	0.000	0.966	0.922	0.000	0.836
Support	85	14	115	96	7	57

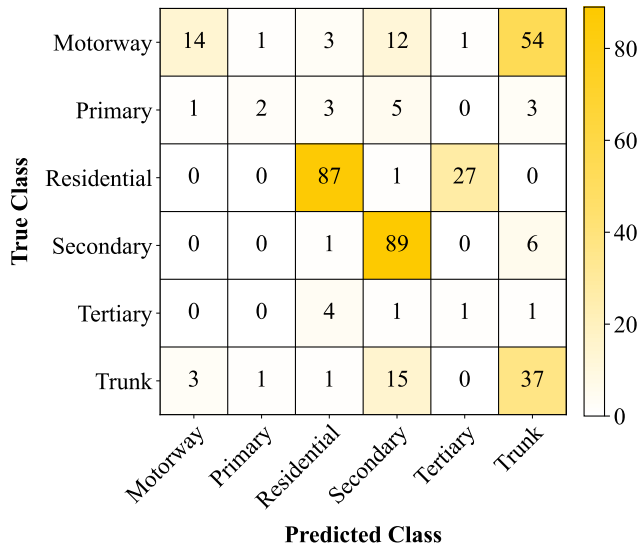
A.6 Few-shot Highway Classification using a Sequence Length of 500 s



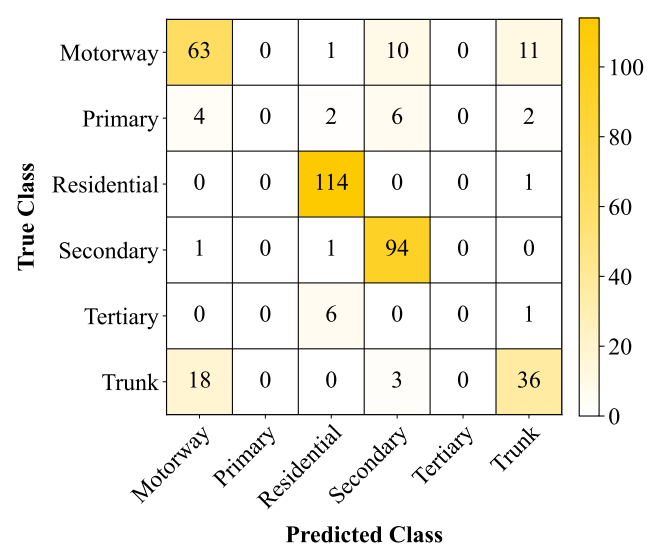
(a) DeepRange 1B



(b) DeepRange 8B



(c) InceptionTime



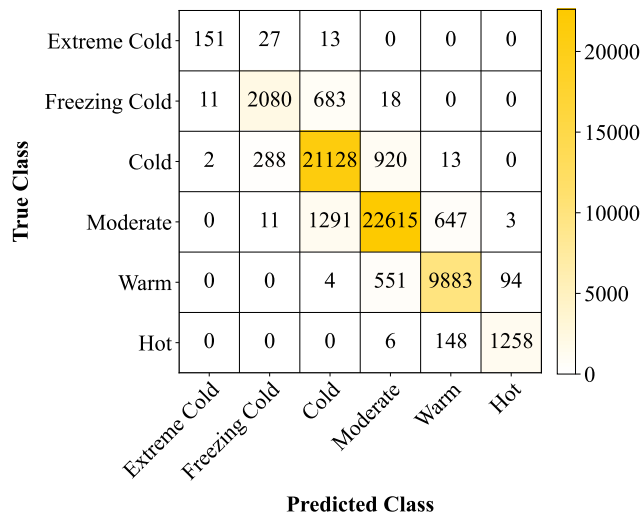
(d) MiniRocket

Figure A-4: Confusion matrices for few-shot highway classification using a sequence length of 500 s.

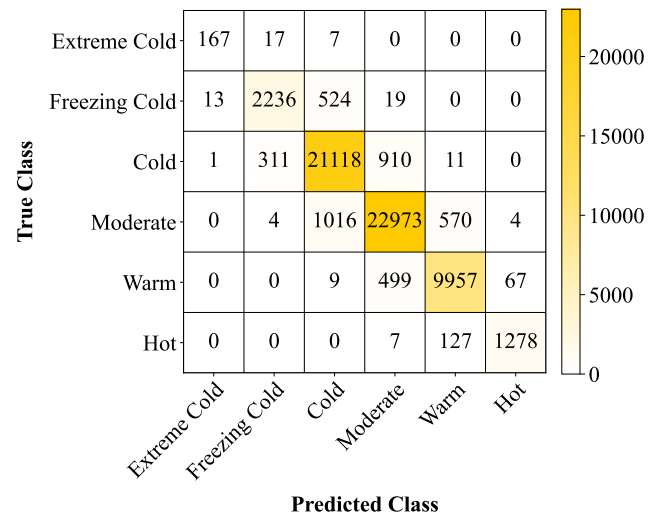
Table A-12: Per-class metrics for few-shot highway classification using a sequence length of 500 s.

	Motorway	Primary	Residential	Secondary	Tertiary	Trunk
<i>DeepRange 1B</i>						
<i>Acc</i>	0.000	0.000	1.000	0.000	0.000	0.000
<i>P</i>	0.000	0.000	0.307	0.000	0.000	0.000
<i>R</i>	0.000	0.000	1.000	0.000	0.000	0.000
<i>F₁</i>	0.000	0.000	0.470	0.000	0.000	0.000
<i>DeepRange 8B</i>						
<i>Acc</i>	0.000	0.000	1.000	0.000	0.000	0.000
<i>P</i>	0.000	0.000	0.307	0.000	0.000	0.000
<i>R</i>	0.000	0.000	1.000	0.000	0.000	0.000
<i>F₁</i>	0.000	0.000	0.470	0.000	0.000	0.000
<i>InceptionTime</i>						
<i>Acc</i>	0.165	0.143	0.757	0.927	0.143	0.649
<i>P</i>	0.778	0.500	0.879	0.724	0.034	0.366
<i>R</i>	0.165	0.143	0.757	0.927	0.143	0.649
<i>F₁</i>	0.272	0.222	0.813	0.813	0.056	0.468
<i>MiniRocket</i>						
<i>Acc</i>	0.741	0.000	0.991	0.979	0.000	0.632
<i>P</i>	0.733	0.000	0.919	0.832	0.000	0.706
<i>R</i>	0.741	0.000	0.991	0.979	0.000	0.632
<i>F₁</i>	0.737	0.000	0.954	0.900	0.000	0.667
Support	85	14	115	96	7	57

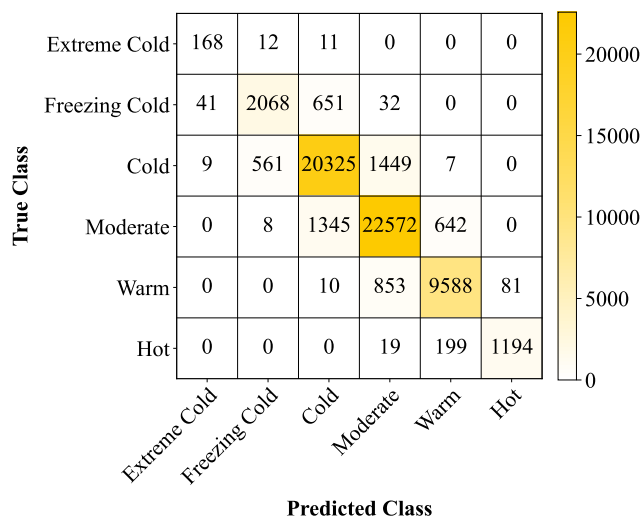
A.7 Ambient Temperature Classification using a Sequence Length of 100 s



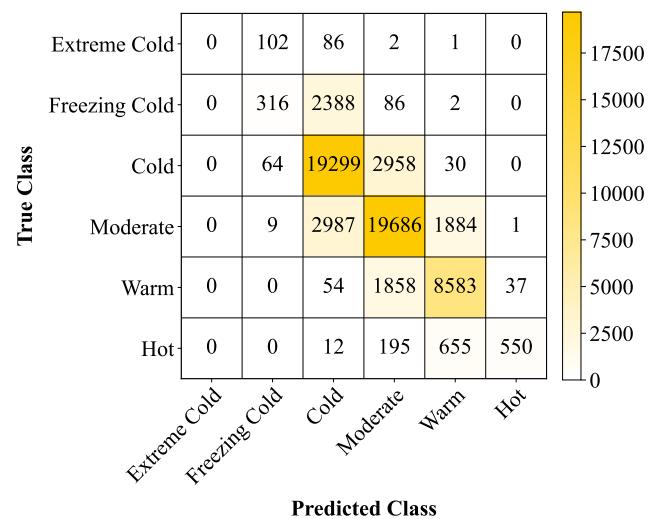
(a) DeepRange 1B



(b) DeepRange 8B



(c) InceptionTime

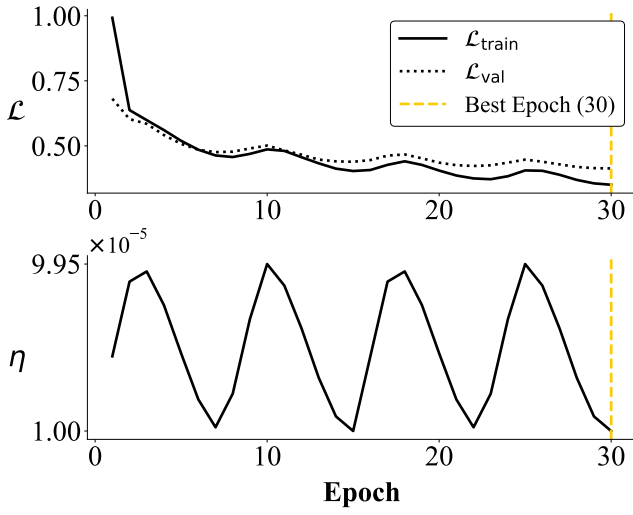


(d) MiniRocket

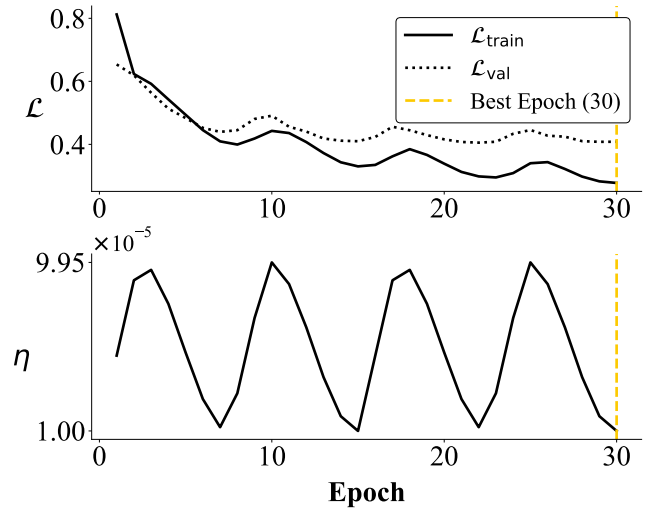
Figure A-5: Confusion matrices for ambient temperature classification using a sequence length of 100 s.

Table A-13: Per-class metrics for ambient temperature classification using a sequence length of 100 s.

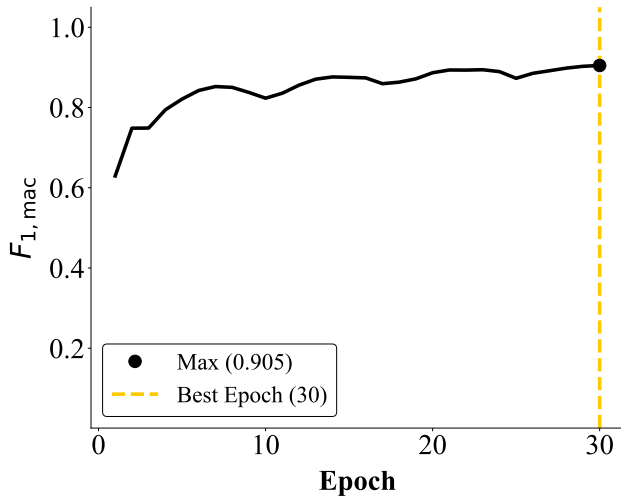
	Extreme Cold	Freezing Cold	Cold	Moderate	Warm	Hot
<i>DeepRange 1B</i>						
<i>Acc</i>	0.791	0.745	0.945	0.921	0.938	0.891
<i>Pr</i>	0.921	0.865	0.914	0.938	0.924	0.928
<i>R</i>	0.791	0.745	0.945	0.921	0.938	0.891
<i>F₁</i>	0.851	0.800	0.929	0.929	0.931	0.909
<i>DeepRange 8B</i>						
<i>Acc</i>	0.874	0.801	0.945	0.935	0.945	0.905
<i>Pr</i>	0.923	0.871	0.931	0.941	0.934	0.947
<i>R</i>	0.874	0.801	0.945	0.935	0.945	0.905
<i>F₁</i>	0.898	0.834	0.938	0.938	0.939	0.926
<i>InceptionTime</i>						
<i>Acc</i>	0.880	0.741	0.909	0.919	0.910	0.846
<i>Pr</i>	0.771	0.781	0.910	0.906	0.919	0.936
<i>R</i>	0.880	0.741	0.909	0.919	0.910	0.846
<i>F₁</i>	0.822	0.760	0.910	0.912	0.915	0.889
<i>MiniRocket</i>						
<i>Acc</i>	0.000	0.113	0.863	0.801	0.815	0.390
<i>Pr</i>	0.000	0.644	0.777	0.794	0.769	0.935
<i>R</i>	0.000	0.113	0.863	0.801	0.815	0.390
<i>F₁</i>	0.000	0.193	0.818	0.798	0.792	0.550
Support	191	2792	22351	24567	10532	1412



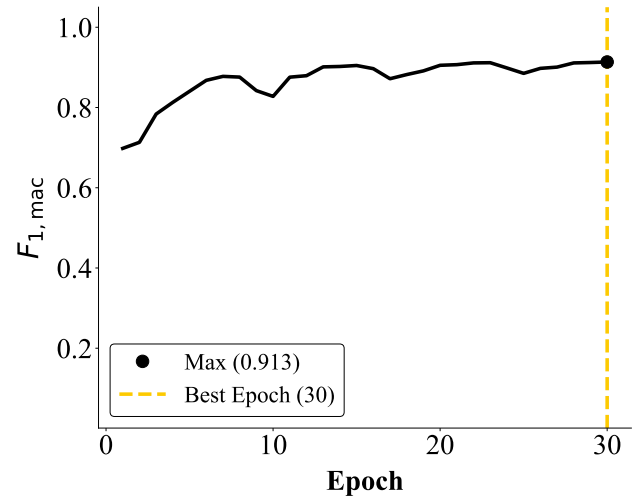
(a) DeepRange 1B - Loss and learning rate



(b) DeepRange 8B - Loss and learning rate



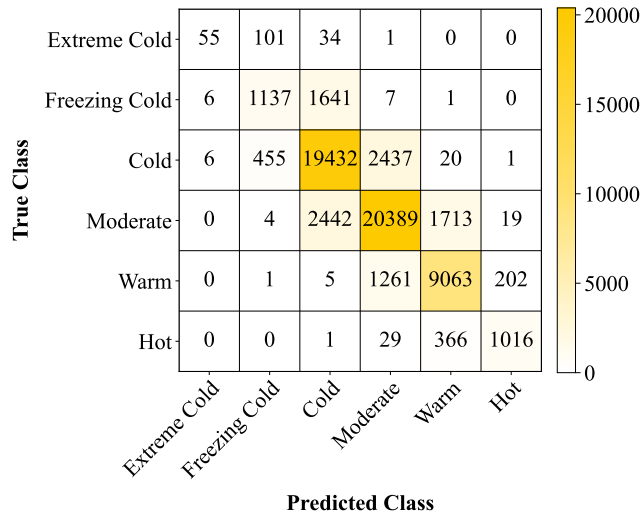
(c) DeepRange 1B - Validation curve



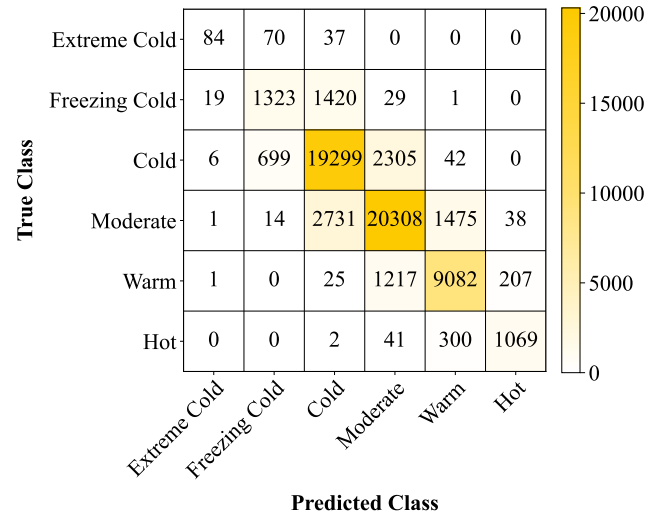
(d) DeepRange 8B - Validation curve

Figure A-6: DeepRange 1B/8B learning curves for ambient temperature classification using a sequence length of 100 s.

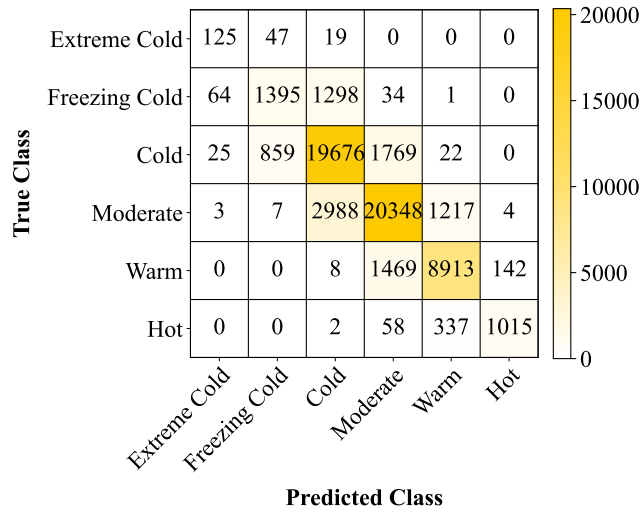
A.8 Few-shot Ambient Temperature Classification using a Sequence Length of 100 s



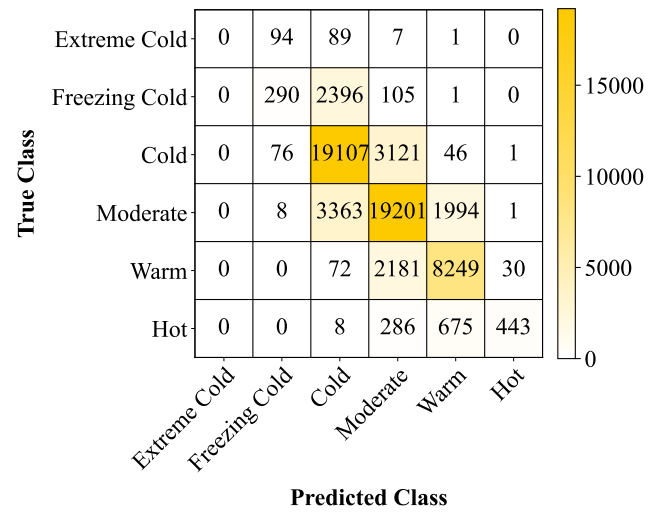
(a) DeepRange 1B



(b) DeepRange 8B



(c) InceptionTime

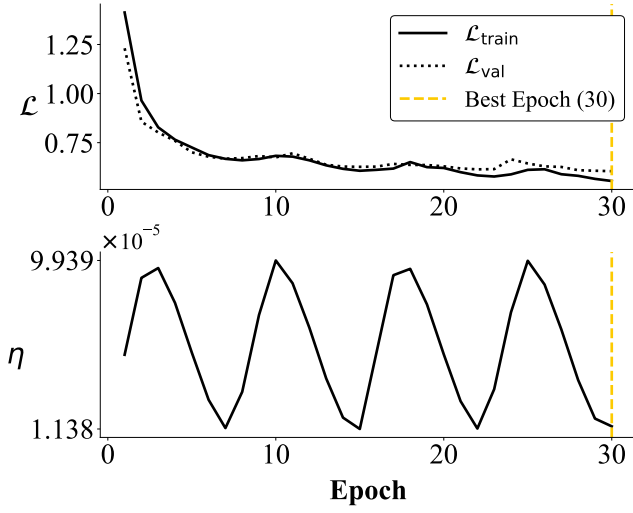


(d) MiniRocket

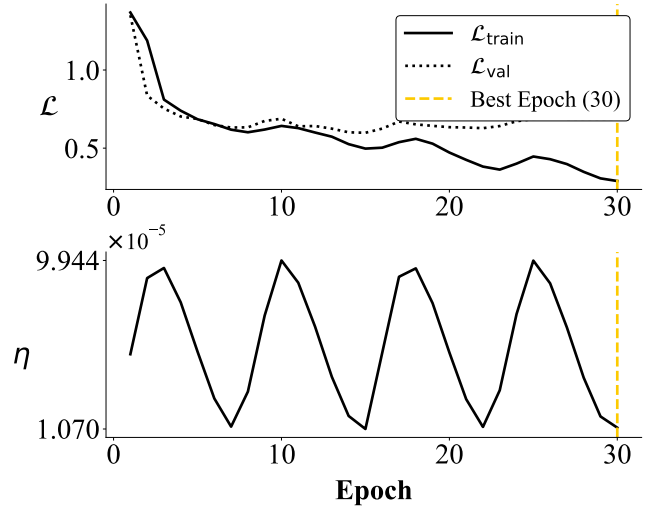
Figure A-7: Confusion matrices for few-shot ambient temperature classification using a sequence length of 100 s.

Table A-14: Per-class metrics for few-shot ambient temperature classification using a sequence length of 100 s.

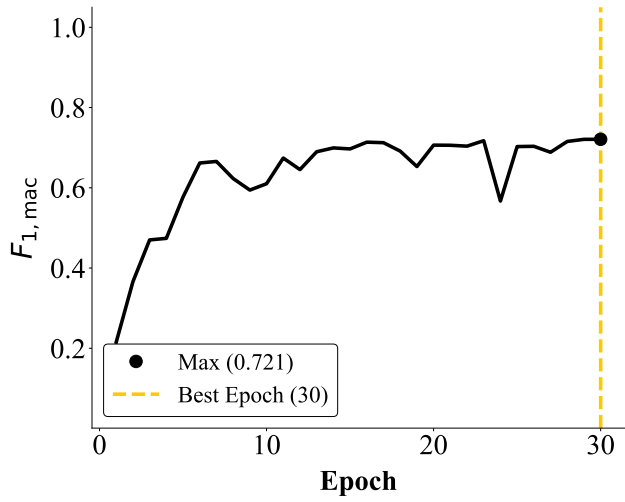
	Extreme Cold	Freezing Cold	Cold	Moderate	Warm	Hot
<i>DeepRange 1B</i>						
<i>Acc</i>	0.288	0.407	0.869	0.830	0.861	0.720
<i>Pr</i>	0.821	0.670	0.825	0.845	0.812	0.821
<i>R</i>	0.288	0.407	0.869	0.830	0.861	0.720
<i>F₁</i>	0.426	0.506	0.847	0.837	0.835	0.767
<i>DeepRange 8B</i>						
<i>Acc</i>	0.440	0.474	0.863	0.827	0.862	0.757
<i>Pr</i>	0.757	0.628	0.821	0.850	0.833	0.814
<i>R</i>	0.440	0.474	0.863	0.827	0.862	0.757
<i>F₁</i>	0.556	0.540	0.842	0.838	0.848	0.784
<i>InceptionTime</i>						
<i>Acc</i>	0.654	0.500	0.880	0.828	0.846	0.719
<i>Pr</i>	0.576	0.604	0.820	0.859	0.850	0.874
<i>R</i>	0.654	0.500	0.880	0.828	0.846	0.719
<i>F₁</i>	0.613	0.547	0.849	0.844	0.848	0.789
<i>MiniRocket</i>						
<i>Acc</i>	0.000	0.104	0.855	0.782	0.783	0.314
<i>Pr</i>	0.000	0.620	0.763	0.771	0.752	0.933
<i>R</i>	0.000	0.104	0.855	0.782	0.783	0.314
<i>F₁</i>	0.000	0.178	0.806	0.776	0.767	0.470
Support	191	2792	22351	24567	10532	1412



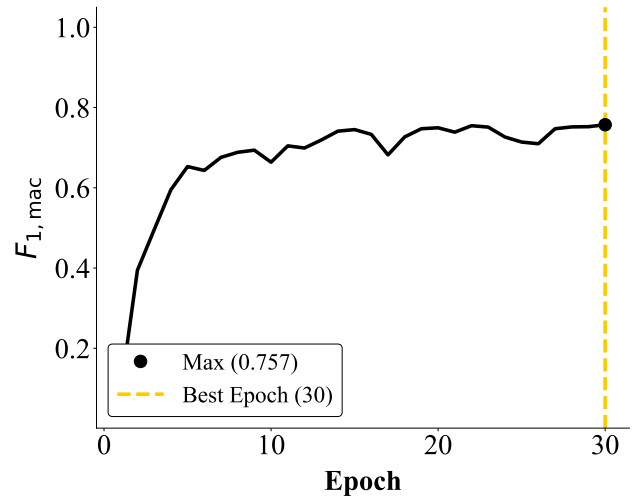
(a) DeepRange 1B - Loss and learning rate



(b) DeepRange 8B - Loss and learning rate



(c) DeepRange 1B - Validation curve



(d) DeepRange 8B - Validation curve

Figure A-8: DeepRange 1B/8B learning curves for few-shot ambient temperature classification using a sequence length of 100 s.

A.9 Ambient Temperature Classification using a Sequence Length of 500 s

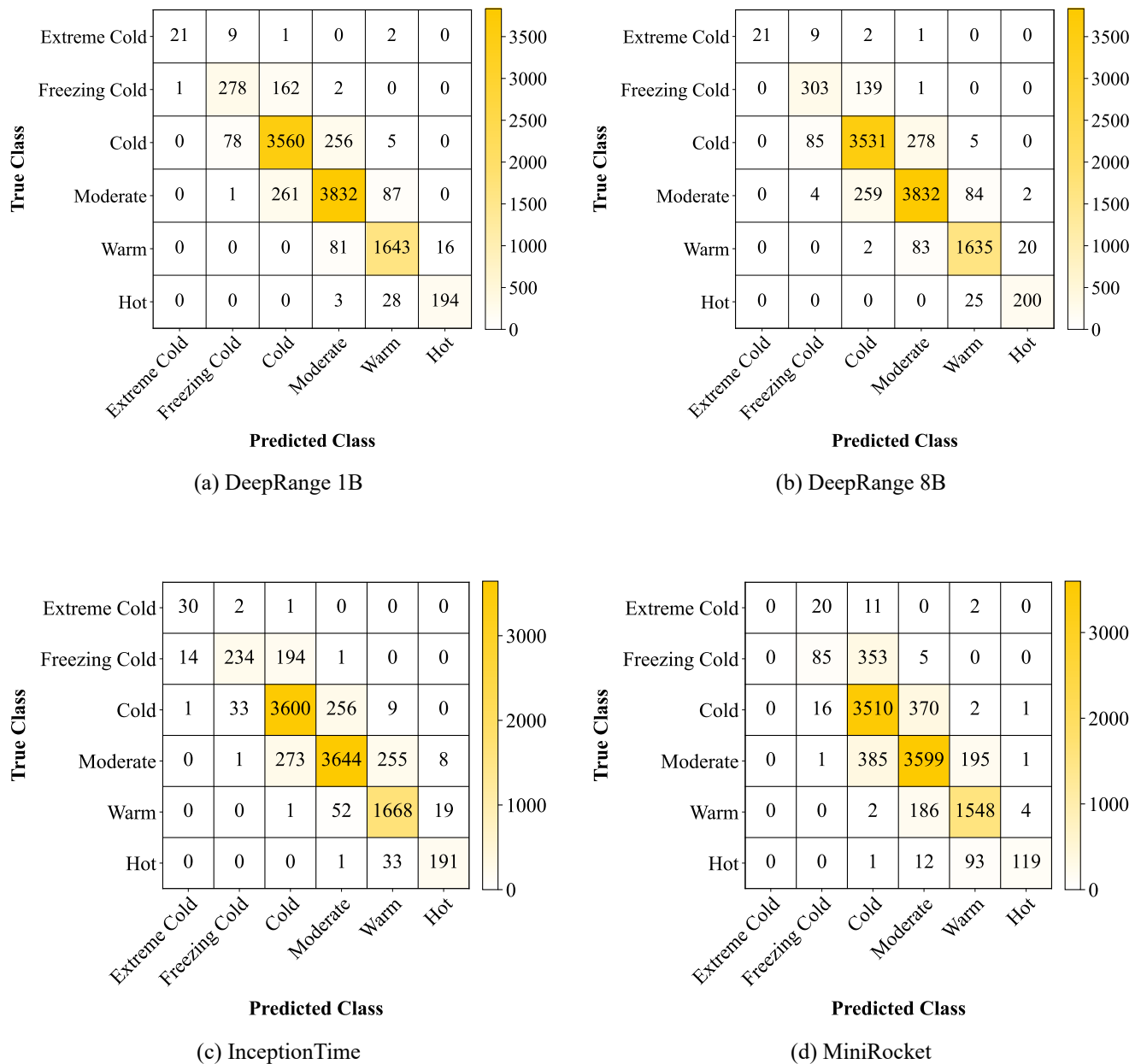
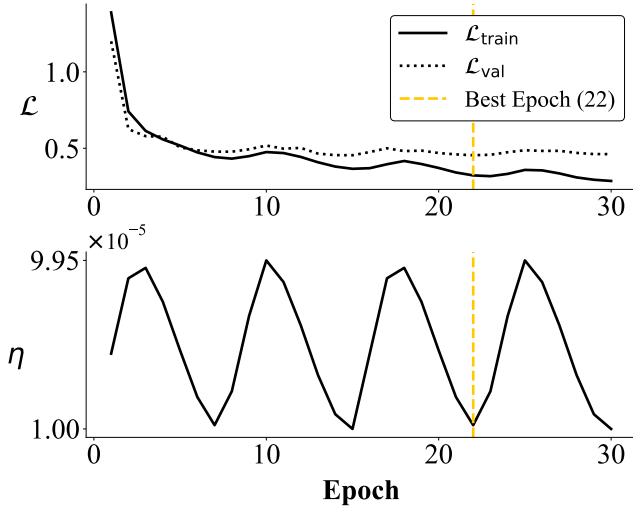


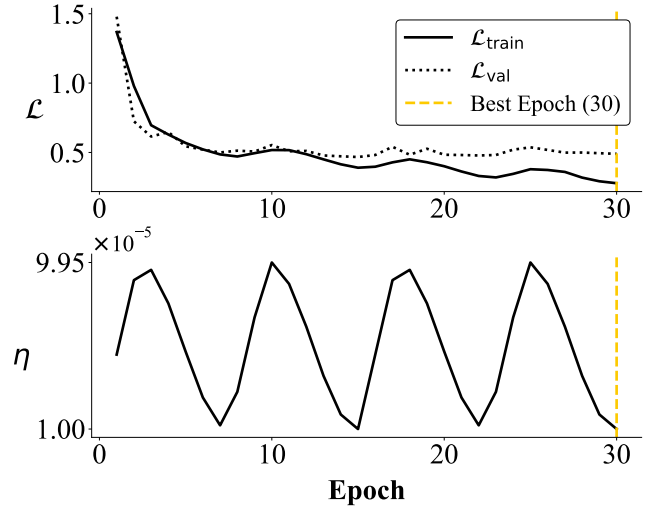
Figure A-9: Confusion matrices for ambient temperature classification using a sequence length of 500 s.

Table A-15: Per-class metrics for ambient temperature classification using a sequence length of 500 s.

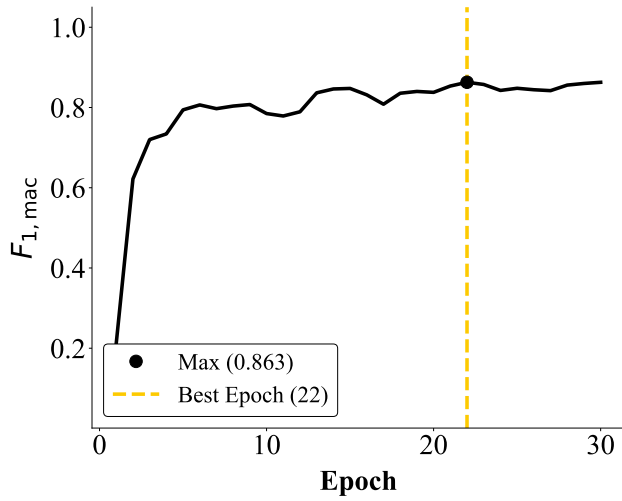
	Extreme Cold	Freezing Cold	Cold	Moderate	Warm	Hot
<i>DeepRange 1B</i>						
<i>Acc</i>	0.636	0.628	0.913	0.917	0.944	0.862
<i>Pr</i>	0.955	0.760	0.894	0.918	0.931	0.924
<i>R</i>	0.636	0.628	0.913	0.917	0.944	0.862
<i>F₁</i>	0.764	0.687	0.903	0.917	0.938	0.892
<i>DeepRange 8B</i>						
<i>Acc</i>	0.636	0.684	0.906	0.917	0.940	0.889
<i>Pr</i>	1.000	0.756	0.898	0.913	0.935	0.901
<i>R</i>	0.636	0.684	0.906	0.917	0.940	0.889
<i>F₁</i>	0.778	0.718	0.902	0.915	0.937	0.895
<i>InceptionTime</i>						
<i>Acc</i>	0.909	0.528	0.923	0.872	0.959	0.849
<i>Pr</i>	0.667	0.867	0.885	0.922	0.849	0.876
<i>R</i>	0.909	0.528	0.923	0.872	0.959	0.849
<i>F₁</i>	0.769	0.656	0.904	0.896	0.900	0.862
<i>MiniRocket</i>						
<i>Acc</i>	0.000	0.192	0.900	0.861	0.890	0.529
<i>Pr</i>	0.000	0.697	0.824	0.863	0.841	0.952
<i>R</i>	0.000	0.192	0.900	0.861	0.890	0.529
<i>F₁</i>	0.000	0.301	0.860	0.862	0.865	0.680
Support	33	443	3899	4181	1740	225



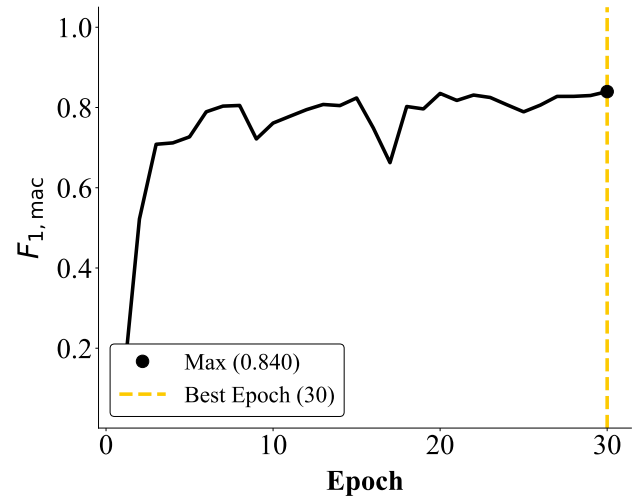
(a) DeepRange 1B - Loss and learning rate



(b) DeepRange 8B - Loss and learning rate



(c) DeepRange 1B - Validation curve



(d) DeepRange 8B - Validation curve

Figure A-10: DeepRange 1B/8B learning curves for ambient temperature classification using a sequence length of 500 s.

A.10 Few-shot Ambient Temperature Classification using a Sequence Length of 500 s

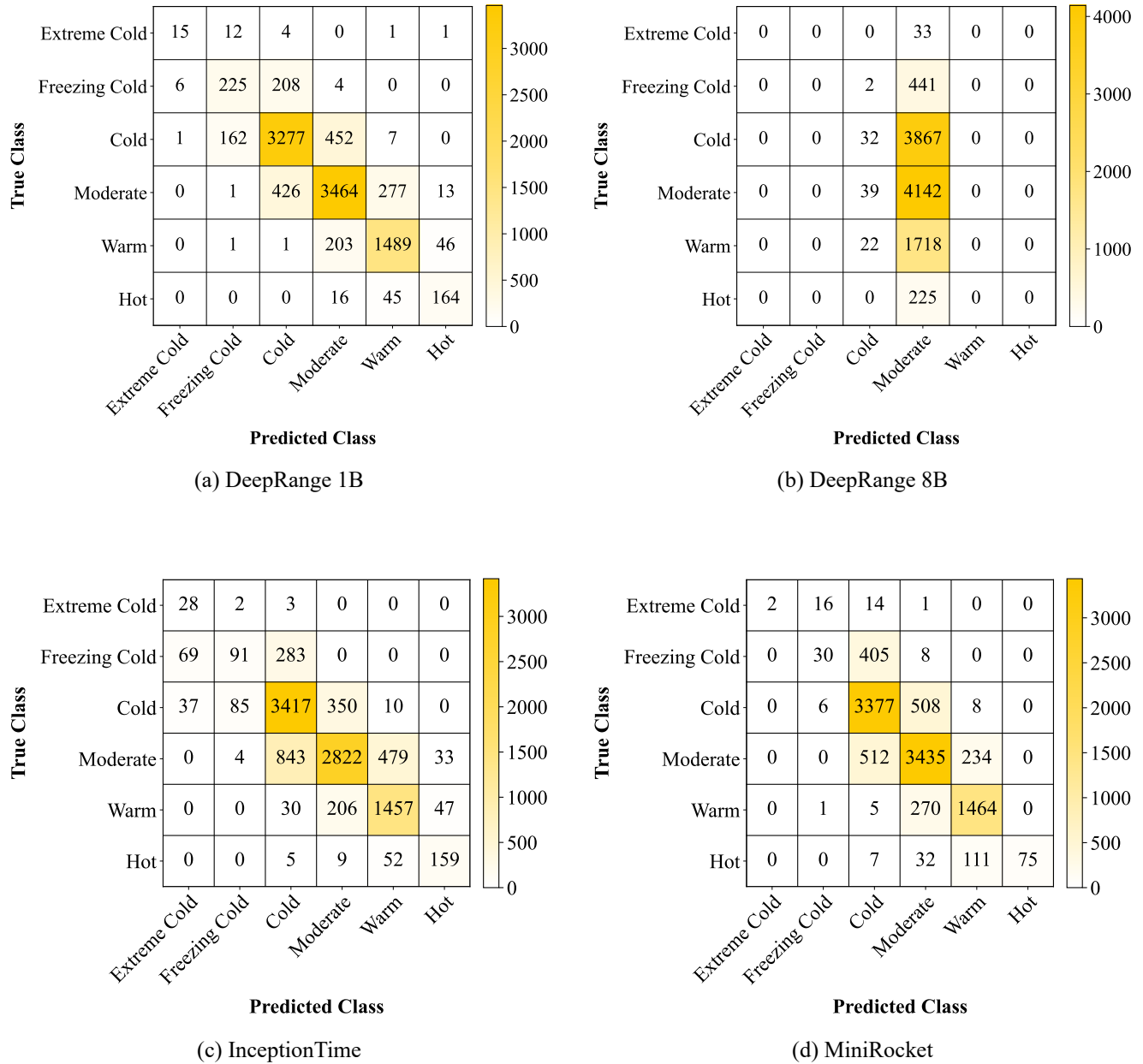
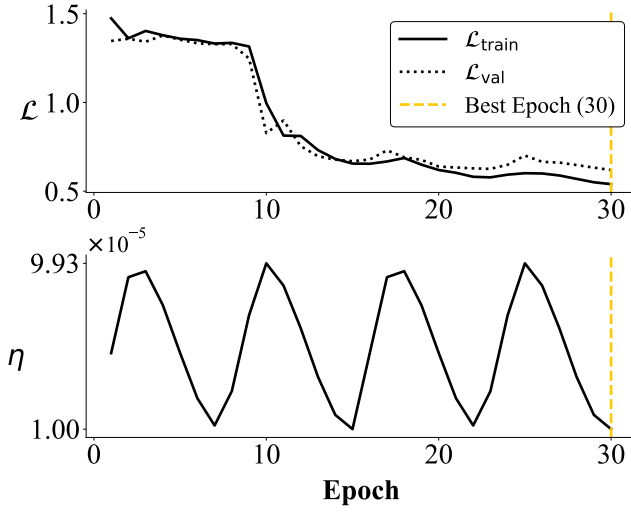


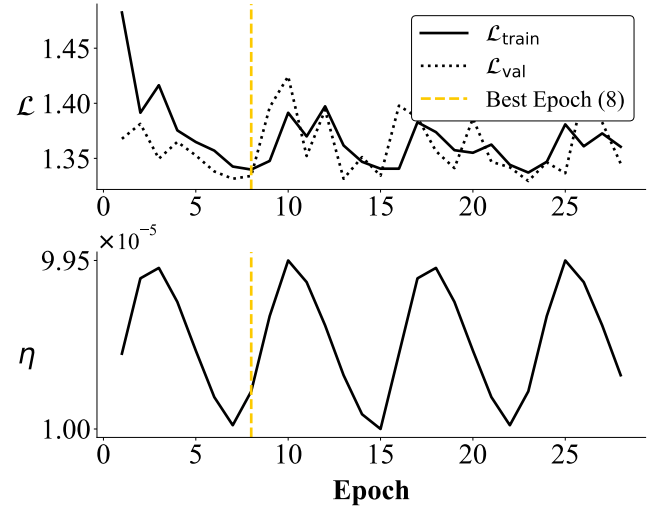
Figure A-11: Confusion matrices for few-shot ambient temperature classification using a sequence length of 500 s.

Table A-16: Per-class metrics for few-shot ambient temperature classification using a sequence length of 500 s.

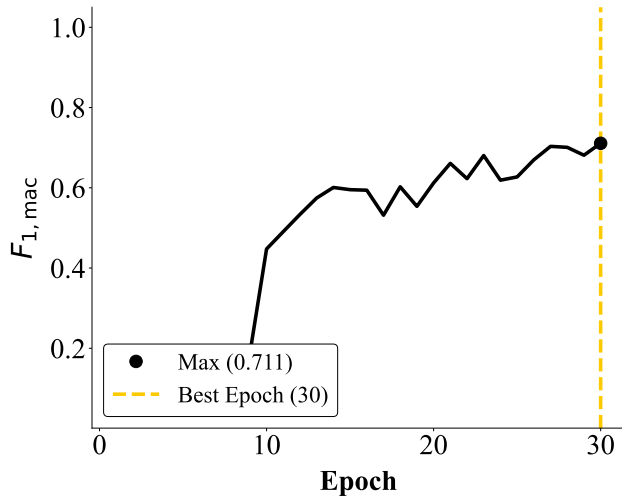
	Extreme Cold	Freezing Cold	Cold	Moderate	Warm	Hot
<i>DeepRange 1B</i>						
<i>Acc</i>	0.455	0.508	0.840	0.829	0.856	0.729
<i>Pr</i>	0.682	0.561	0.837	0.837	0.819	0.732
<i>R</i>	0.455	0.508	0.840	0.829	0.856	0.729
<i>F₁</i>	0.545	0.533	0.839	0.833	0.837	0.731
<i>DeepRange 8B</i>						
<i>Acc</i>	0.000	0.000	0.008	0.991	0.000	0.000
<i>Pr</i>	0.000	0.000	0.337	0.397	0.000	0.000
<i>R</i>	0.000	0.000	0.008	0.991	0.000	0.000
<i>F₁</i>	0.000	0.000	0.016	0.567	0.000	0.000
<i>InceptionTime</i>						
<i>Acc</i>	0.848	0.205	0.876	0.675	0.837	0.707
<i>Pr</i>	0.209	0.500	0.746	0.833	0.729	0.665
<i>R</i>	0.848	0.205	0.876	0.675	0.837	0.707
<i>F₁</i>	0.335	0.291	0.806	0.746	0.780	0.685
<i>MiniRocket</i>						
<i>Acc</i>	0.061	0.068	0.866	0.822	0.841	0.333
<i>Pr</i>	1.000	0.566	0.782	0.807	0.806	1.000
<i>R</i>	0.061	0.068	0.866	0.822	0.841	0.333
<i>F₁</i>	0.114	0.121	0.822	0.814	0.823	0.500
Support	33	443	3899	4181	1740	225



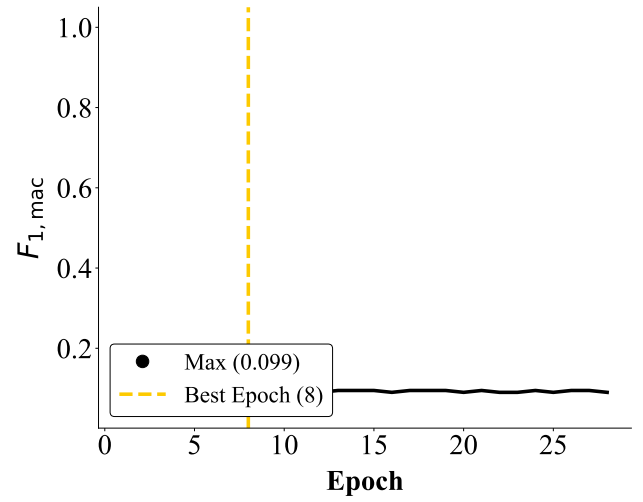
(a) DeepRange 1B - Loss and learning rate



(b) DeepRange 8B - Loss and learning rate



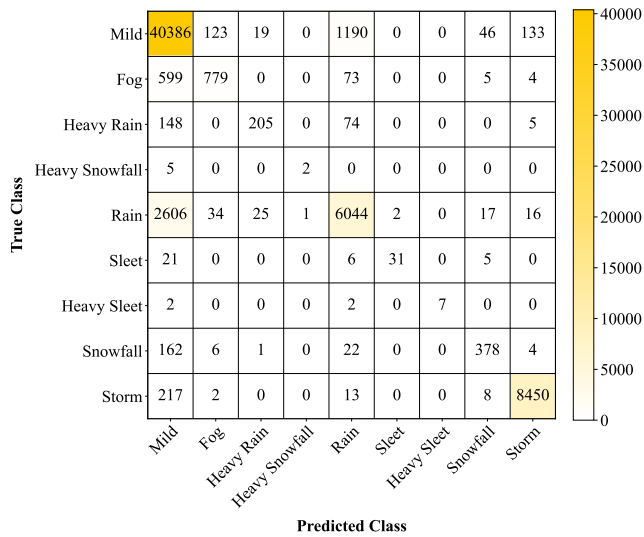
(c) DeepRange 1B - Validation curve



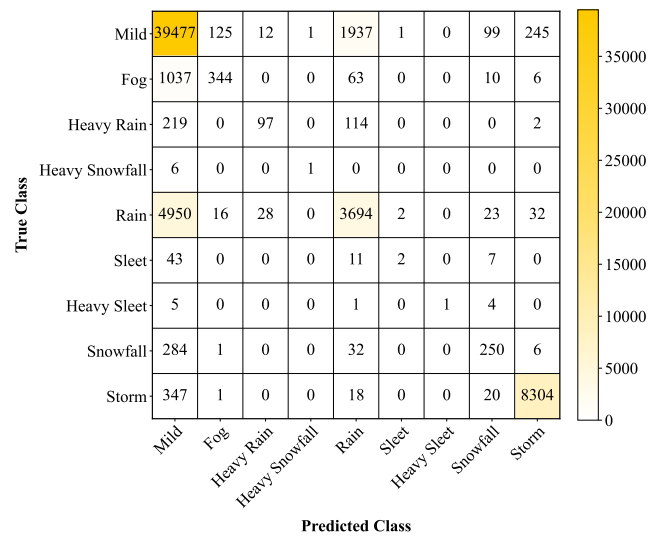
(d) DeepRange 8B - Validation curve

Figure A-12: DeepRange 1B/8B learning curves for few-shot ambient temperature classification using a sequence length of 500 s.

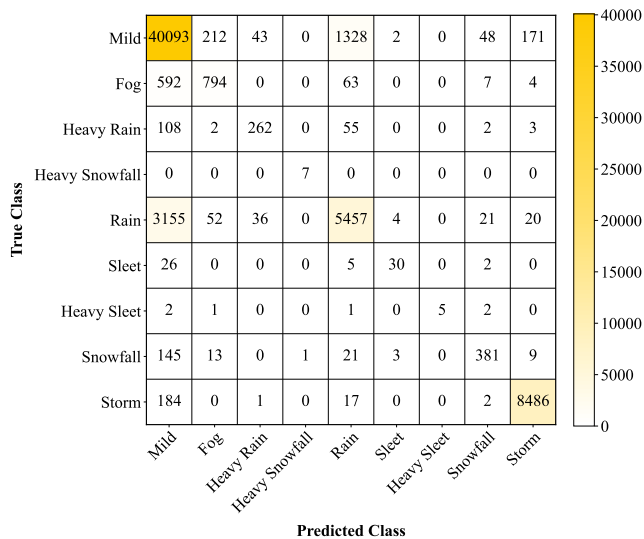
A.11 Weather Classification using a Sequence Length of 100 s



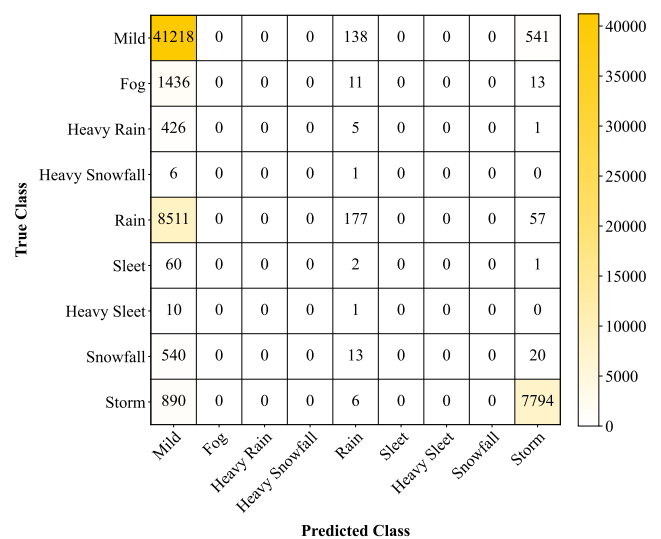
(a) DeepRange 1B



(b) DeepRange 8B



(c) InceptionTime

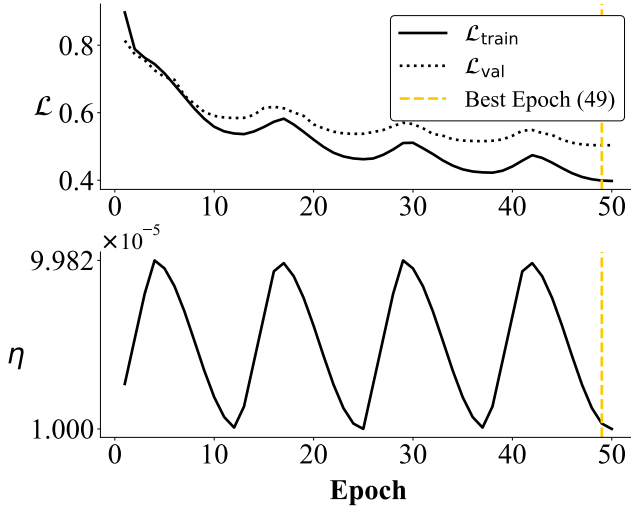


(d) MiniRocket

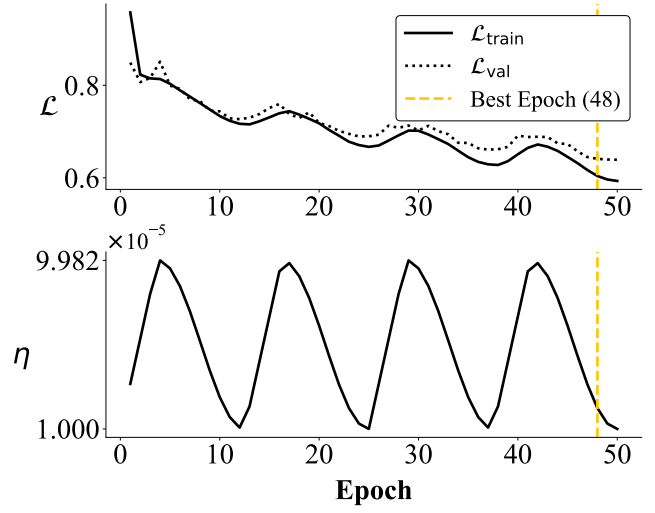
Figure A-13: Confusion matrices for weather classification using a sequence length of 100 s.

Table A-17: Per-class metrics for weather classification using a sequence length of 100 s.

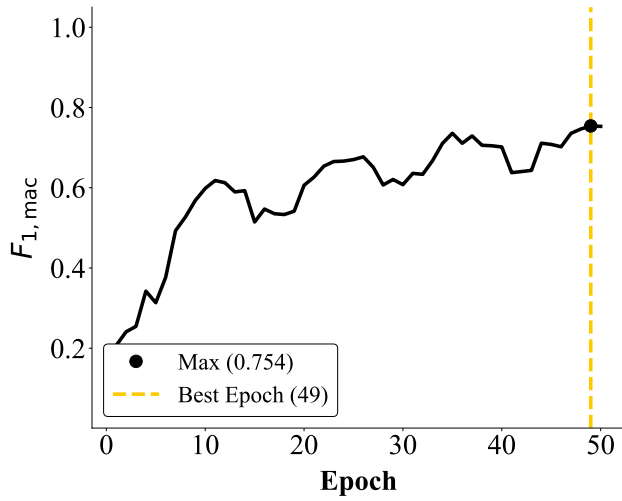
	Mild	Fog	Heavy Rain	Heavy Snowfall	Rain	Sleet	Heavy Sleet	Snowfall	Storm
<i>DeepRange 1B</i>									
<i>Acc</i>	0.964	0.534	0.475	0.286	0.691	0.492	0.636	0.660	0.972
<i>Pr</i>	0.915	0.825	0.820	0.667	0.814	0.939	1.000	0.824	0.981
<i>R</i>	0.964	0.534	0.475	0.286	0.691	0.492	0.636	0.660	0.972
<i>F₁</i>	0.939	0.648	0.601	0.400	0.748	0.646	0.778	0.733	0.977
<i>DeepRange 8B</i>									
<i>Acc</i>	0.942	0.236	0.225	0.143	0.422	0.032	0.091	0.436	0.956
<i>Pr</i>	0.851	0.706	0.708	0.500	0.629	0.400	1.000	0.605	0.966
<i>R</i>	0.942	0.236	0.225	0.143	0.422	0.032	0.091	0.436	0.956
<i>F₁</i>	0.895	0.353	0.341	0.222	0.506	0.059	0.167	0.507	0.961
<i>InceptionTime</i>									
<i>Acc</i>	0.957	0.544	0.606	1.000	0.624	0.476	0.455	0.665	0.977
<i>Pr</i>	0.905	0.739	0.766	0.875	0.786	0.769	1.000	0.819	0.976
<i>R</i>	0.957	0.544	0.606	1.000	0.624	0.476	0.455	0.665	0.977
<i>F₁</i>	0.930	0.627	0.677	0.933	0.696	0.588	0.625	0.734	0.977
<i>MiniRocket</i>									
<i>Acc</i>	0.984	0.000	0.000	0.000	0.020	0.000	0.000	0.000	0.897
<i>Pr</i>	0.776	0.000	0.000	0.000	0.500	0.000	0.000	0.000	0.925
<i>R</i>	0.984	0.000	0.000	0.000	0.020	0.000	0.000	0.000	0.897
<i>F₁</i>	0.868	0.000	0.000	0.000	0.039	0.000	0.000	0.000	0.911
Support	41897	1460	432	7	8745	63	11	573	8690



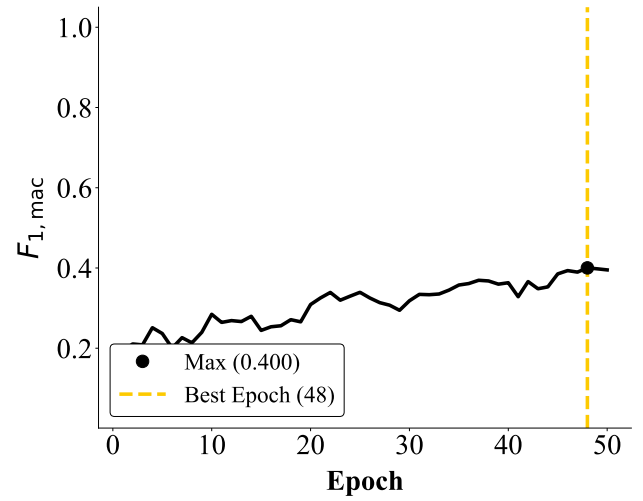
(a) DeepRange 1B - Loss and learning rate



(b) DeepRange 8B - Loss and learning rate



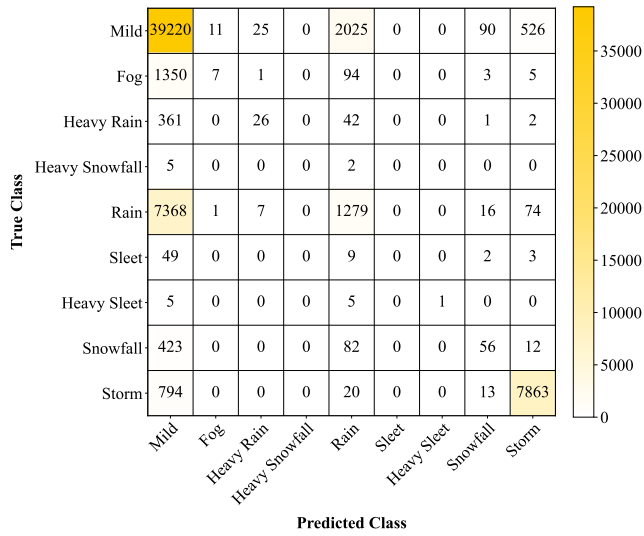
(c) DeepRange 1B - Validation curve



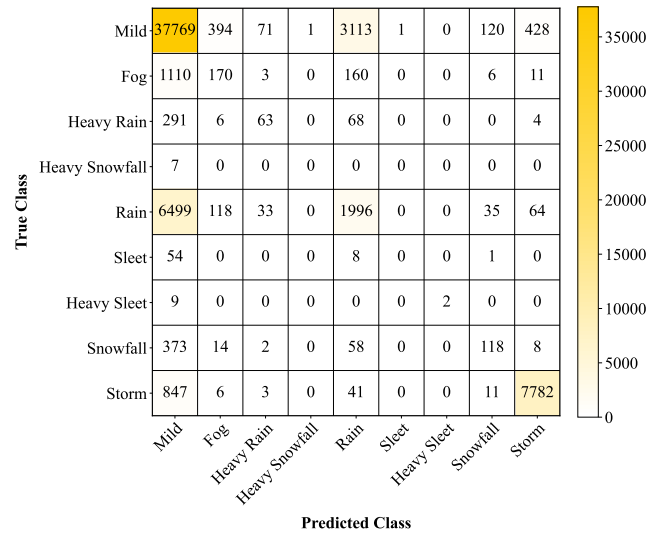
(d) DeepRange 8B - Validation curve

Figure A-14: DeepRange 1B/8B learning curves for weather classification using a sequence length of 100 s.

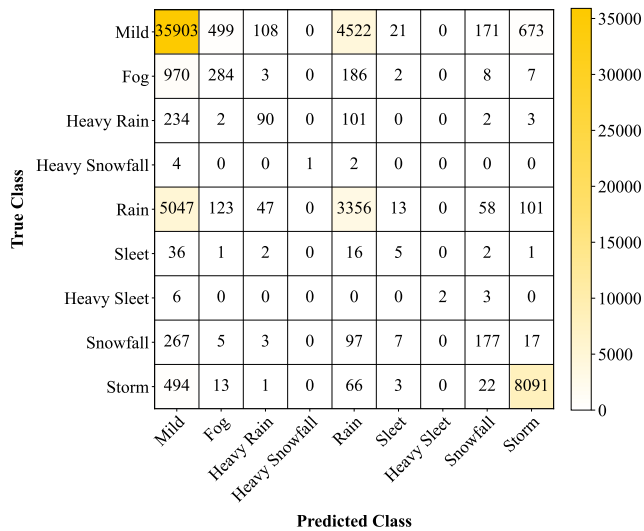
A.12 Few-shot Weather Classification using a Sequence Length of 100 s



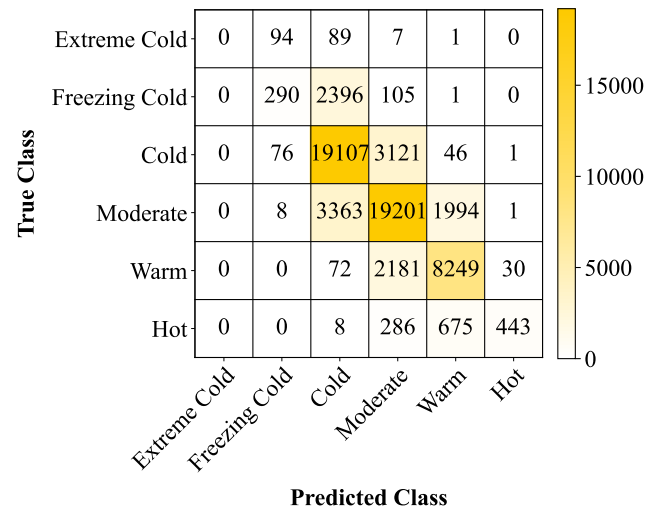
(a) DeepRange 1B



(b) DeepRange 8B



(c) InceptionTime

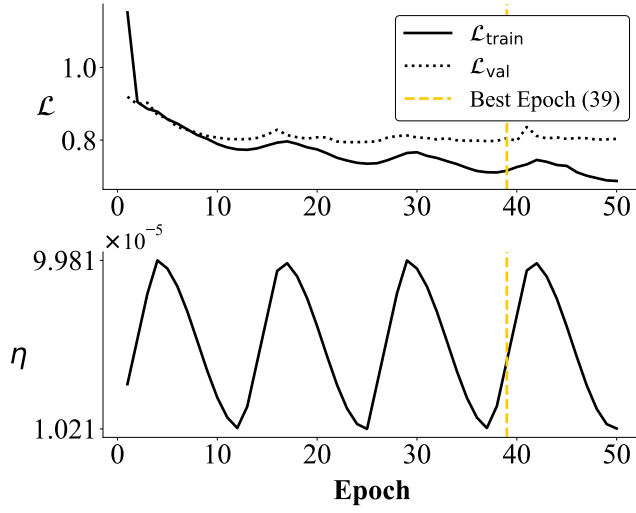


(d) MiniRocket

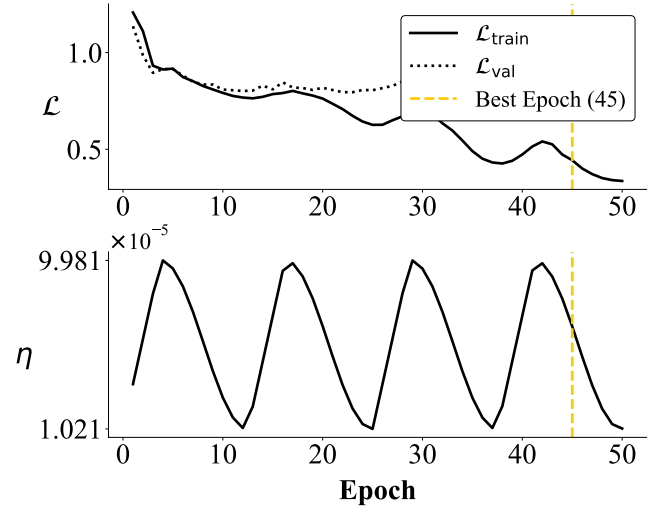
Figure A-15: Confusion matrices for few-shot weather classification using a sequence length of 100 s.

Table A-18: Per-class metrics for few-shot weather classification using a sequence length of 100 s.

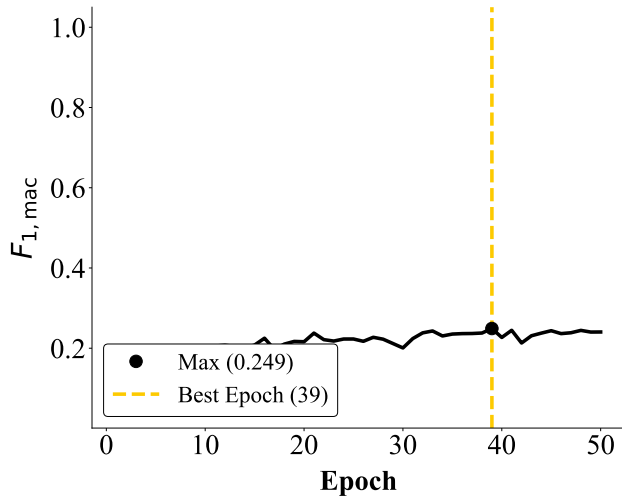
	Mild	Fog	Heavy Rain	Heavy Snowfall	Rain	Sleet	Heavy Sleet	Snowfall	Storm
<i>DeepRange 1B</i>									
<i>Acc</i>	0.936	0.005	0.060	0.000	0.146	0.000	0.091	0.098	0.905
<i>Pr</i>	0.791	0.368	0.441	0.000	0.359	0.000	1.000	0.309	0.927
<i>R</i>	0.936	0.005	0.060	0.000	0.146	0.000	0.091	0.098	0.905
<i>F₁</i>	0.858	0.009	0.106	0.000	0.208	0.000	0.167	0.149	0.916
<i>DeepRange 8B</i>									
<i>Acc</i>	0.901	0.116	0.146	0.000	0.228	0.000	0.182	0.206	0.896
<i>Pr</i>	0.804	0.240	0.360	0.000	0.367	0.000	1.000	0.405	0.938
<i>R</i>	0.901	0.116	0.146	0.000	0.228	0.000	0.182	0.206	0.896
<i>F₁</i>	0.850	0.157	0.208	0.000	0.281	0.000	0.308	0.273	0.916
<i>InceptionTime</i>									
<i>Acc</i>	0.857	0.195	0.208	0.143	0.384	0.079	0.182	0.309	0.931
<i>Pr</i>	0.836	0.306	0.354	1.000	0.402	0.098	1.000	0.400	0.910
<i>R</i>	0.857	0.195	0.208	0.143	0.384	0.079	0.182	0.309	0.931
<i>F₁</i>	0.846	0.238	0.262	0.250	0.393	0.088	0.308	0.348	0.920
<i>MiniRocket</i>									
<i>Acc</i>	0.974	0.000	0.000	0.000	0.043	0.000	0.000	0.002	0.891
<i>Pr</i>	0.778	0.000	0.000	0.000	0.377	0.000	0.000	1.000	0.921
<i>R</i>	0.974	0.000	0.000	0.000	0.043	0.000	0.000	0.002	0.891
<i>F₁</i>	0.865	0.000	0.000	0.000	0.076	0.000	0.000	0.003	0.906
Support	41897	1460	432	7	8745	63	11	573	8690



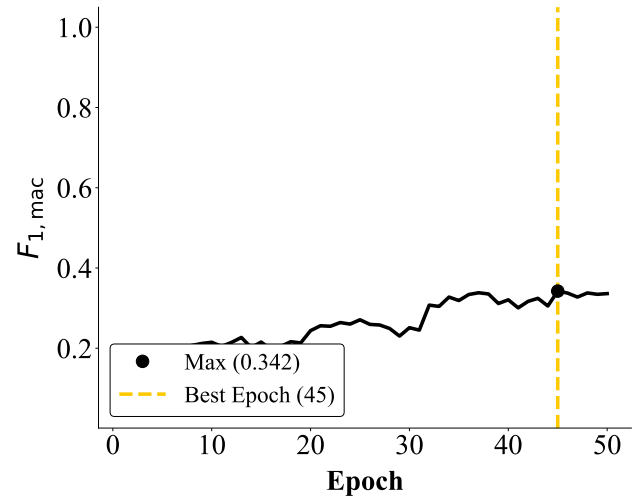
(a) DeepRange 1B - Loss and learning rate



(b) DeepRange 8B - Loss and learning rate



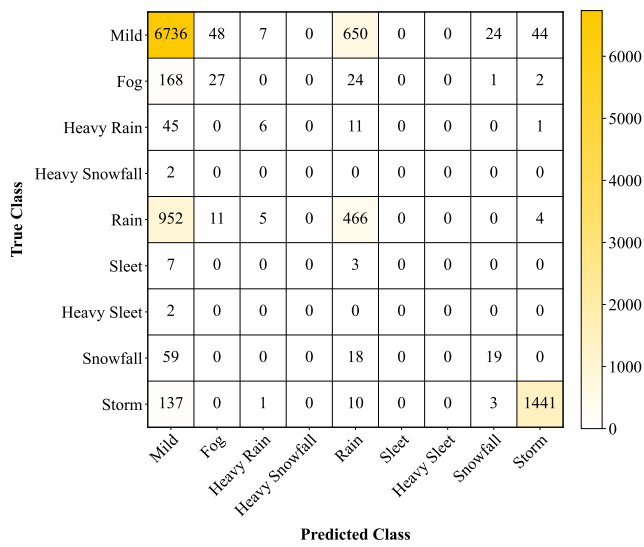
(c) DeepRange 1B - Validation curve



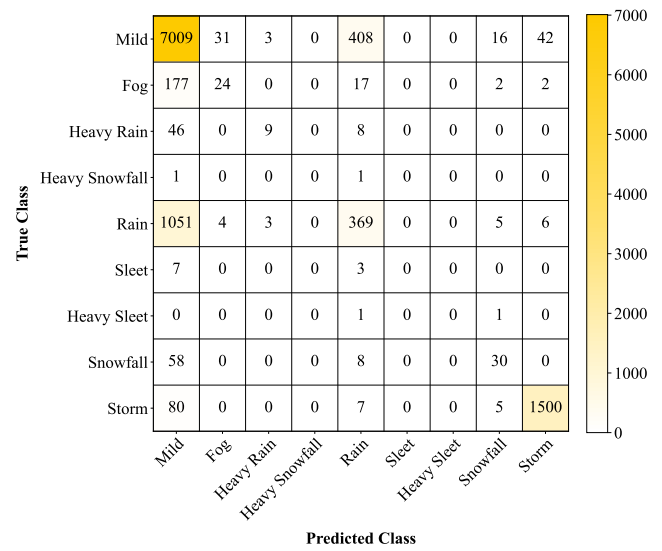
(d) DeepRange 8B - Validation curve

Figure A-16: DeepRange 1B/8B learning curves for few-shot weather classification using a sequence length of 100 s.

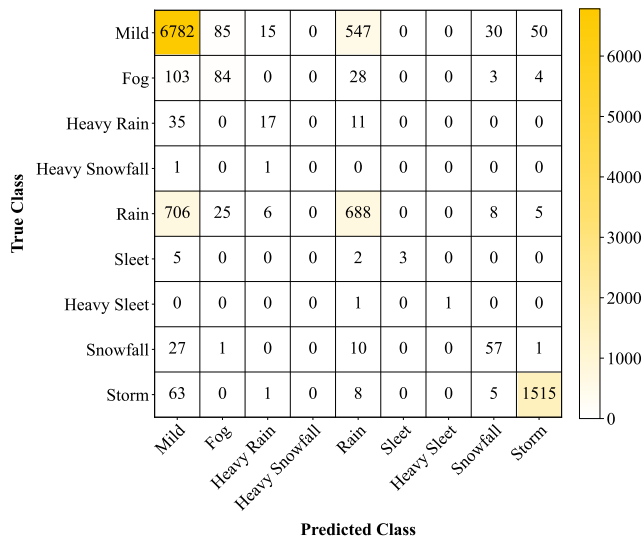
A.13 Weather Classification using a Sequence Length of 500 s



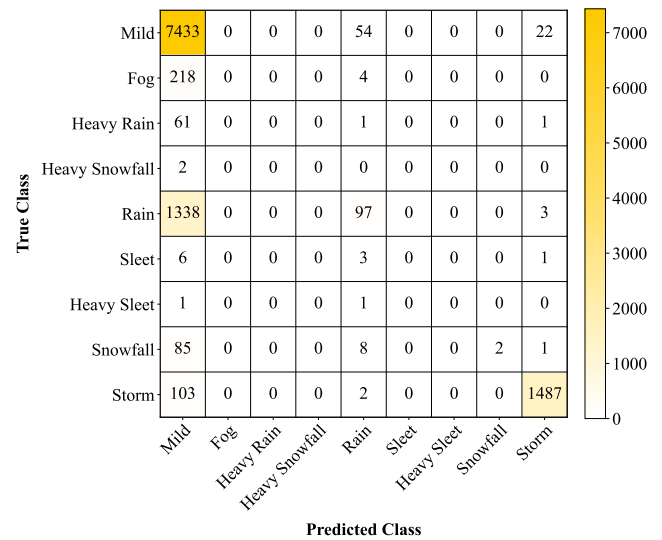
(a) DeepRange 1B



(b) DeepRange 8B



(c) InceptionTime

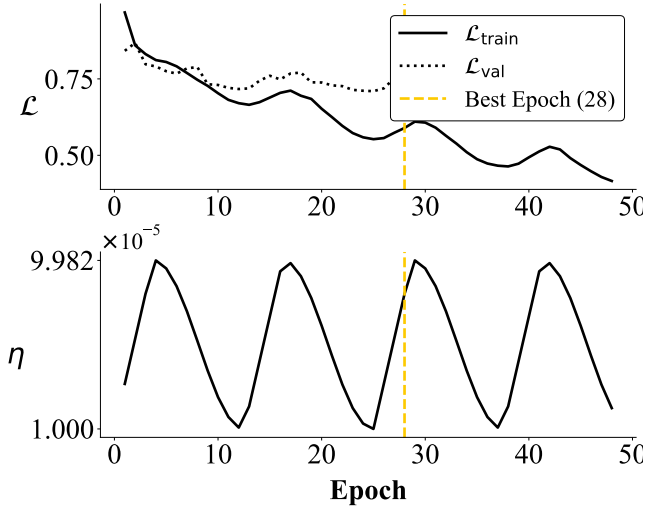


(d) MiniRocket

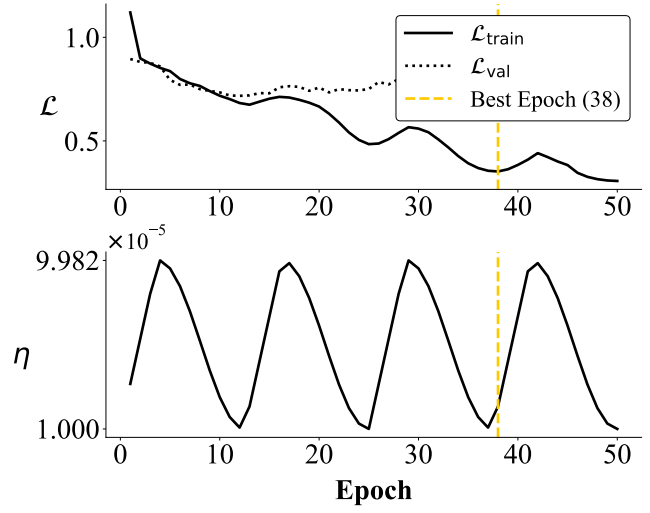
Figure A-17: Confusion matrices for weather classification using a sequence length of 500 s.

Table A-19: Per-class metrics for weather classification using a sequence length of 500 s.

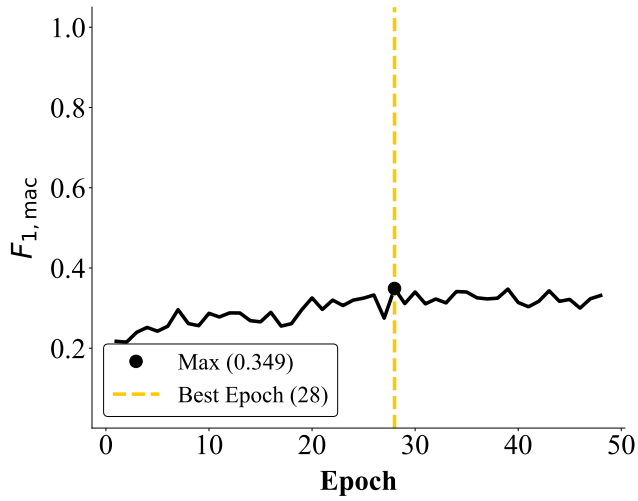
	Mild	Fog	Heavy Rain	Heavy Snowfall	Rain	Sleet	Heavy Sleet	Snowfall	Storm
<i>DeepRange 1B</i>									
<i>Acc</i>	0.897	0.122	0.095	0.000	0.324	0.000	0.000	0.198	0.905
<i>Pr</i>	0.831	0.314	0.316	0.000	0.394	0.000	0.000	0.404	0.966
<i>R</i>	0.897	0.122	0.095	0.000	0.324	0.000	0.000	0.198	0.905
<i>F₁</i>	0.863	0.175	0.146	0.000	0.356	0.000	0.000	0.266	0.935
<i>DeepRange 8B</i>									
<i>Acc</i>	0.933	0.108	0.143	0.000	0.257	0.000	0.000	0.313	0.942
<i>Pr</i>	0.832	0.407	0.600	0.000	0.449	0.000	0.000	0.508	0.968
<i>R</i>	0.933	0.108	0.143	0.000	0.257	0.000	0.000	0.313	0.942
<i>F₁</i>	0.880	0.171	0.231	0.000	0.327	0.000	0.000	0.387	0.955
<i>InceptionTime</i>									
<i>Acc</i>	0.903	0.378	0.270	0.000	0.478	0.300	0.500	0.594	0.952
<i>Pr</i>	0.878	0.431	0.425	0.000	0.531	1.000	1.000	0.553	0.962
<i>R</i>	0.903	0.378	0.270	0.000	0.478	0.300	0.500	0.594	0.952
<i>F₁</i>	0.891	0.403	0.330	0.000	0.503	0.462	0.667	0.573	0.957
<i>MiniRocket</i>									
<i>Acc</i>	0.990	0.000	0.000	0.000	0.067	0.000	0.000	0.021	0.934
<i>Pr</i>	0.804	0.000	0.000	0.000	0.571	0.000	0.000	1.000	0.982
<i>R</i>	0.990	0.000	0.000	0.000	0.067	0.000	0.000	0.021	0.934
<i>F₁</i>	0.887	0.000	0.000	0.000	0.121	0.000	0.000	0.041	0.957
Support	7509	222	63	2	1438	10	2	96	1592



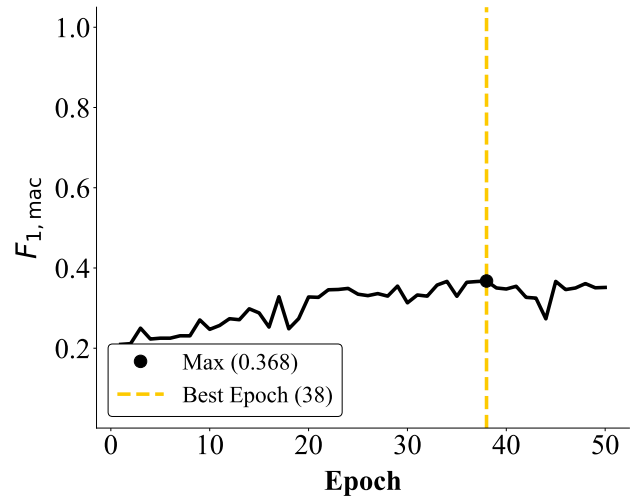
(a) DeepRange 1B - Loss and learning rate



(b) DeepRange 8B - Loss and learning rate



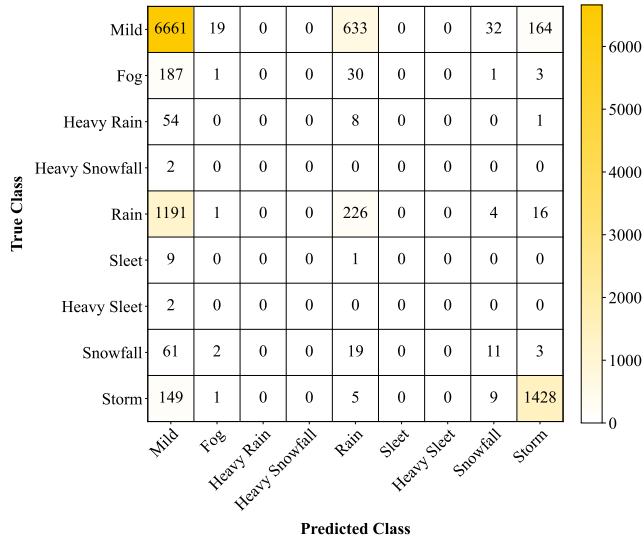
(c) DeepRange 1B - Validation curve



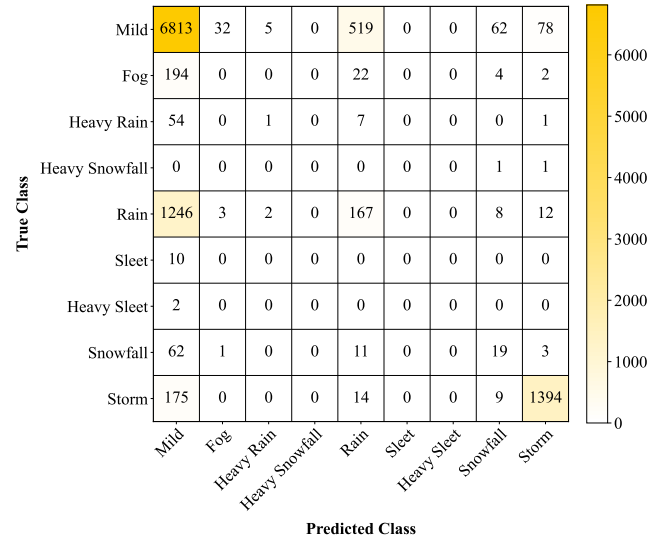
(d) DeepRange 8B - Validation curve

Figure A-18: DeepRange 1B/8B learning curves for weather classification using a sequence length of 500 s.

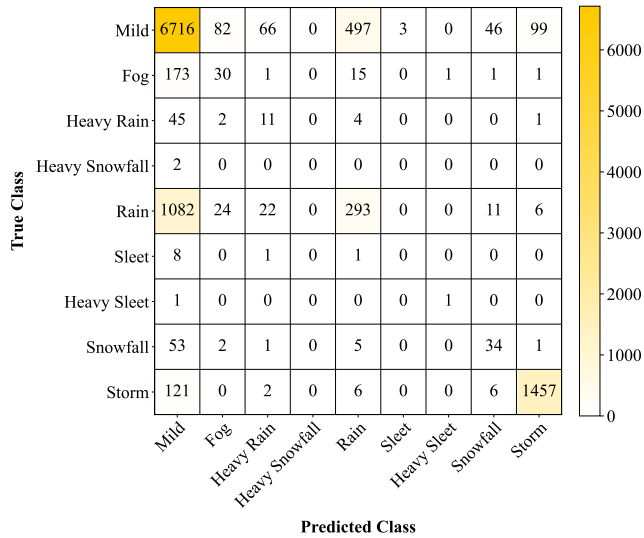
A.14 Few-shot Weather Classification using a Sequence Length of 500 s



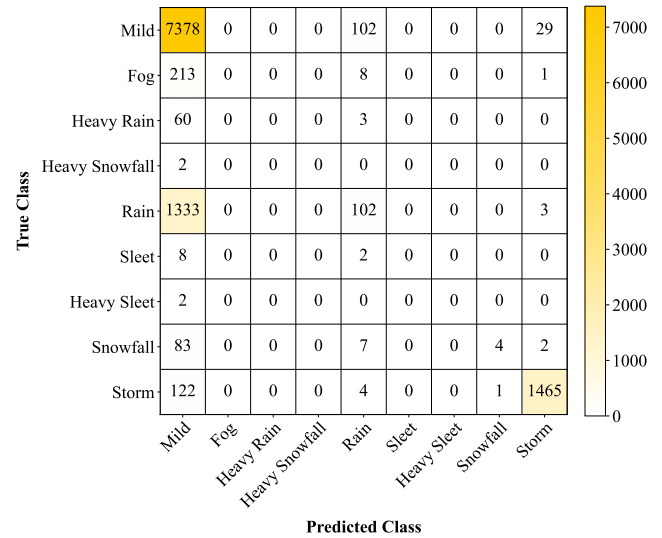
(a) DeepRange 1B



(b) DeepRange 8B



(c) InceptionTime

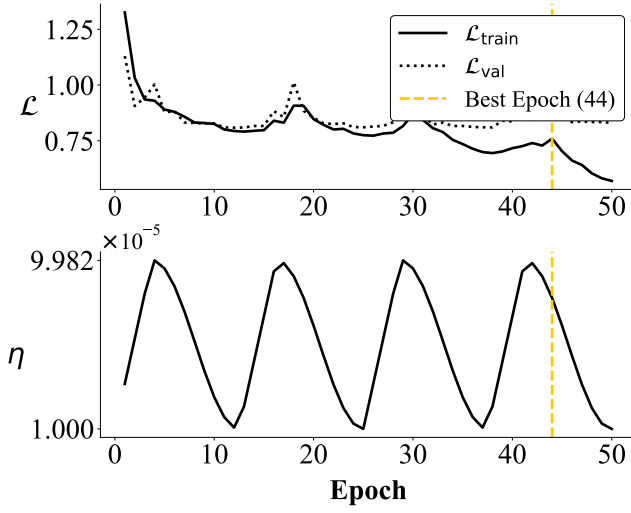


(d) MiniRocket

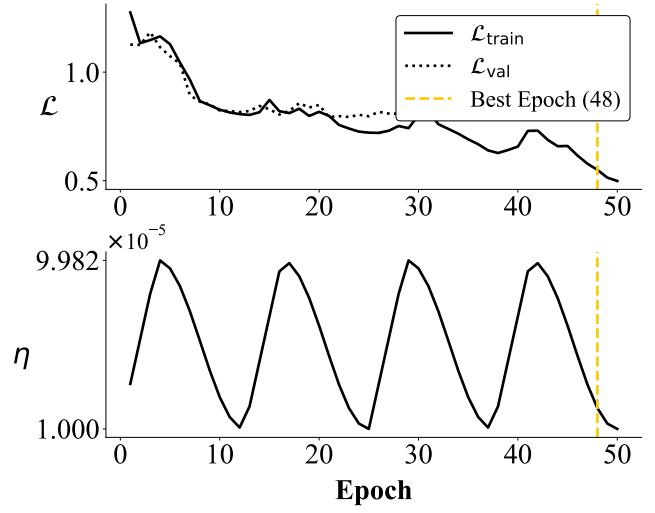
Figure A-19: Confusion matrices for few-shot weather classification using a sequence length of 500 s.

Table A-20: Per-class metrics for few-hot weather classification using a sequence length of 500 s.

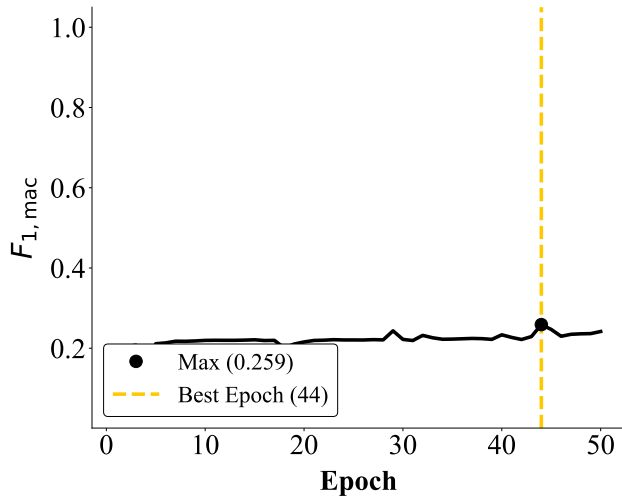
	Mild	Fog	Heavy Rain	Heavy Snowfall	Rain	Sleet	Heavy Sleet	Snowfall	Storm
<i>DeepRange 1B</i>									
<i>Acc</i>	0.887	0.005	0.000	0.000	0.157	0.000	0.000	0.115	0.897
<i>Pr</i>	0.801	0.042	0.000	0.000	0.245	0.000	0.000	0.193	0.884
<i>R</i>	0.887	0.005	0.000	0.000	0.157	0.000	0.000	0.115	0.897
<i>F₁</i>	0.842	0.008	0.000	0.000	0.192	0.000	0.000	0.144	0.891
<i>DeepRange 8B</i>									
<i>Acc</i>	0.907	0.000	0.016	0.000	0.116	0.000	0.000	0.198	0.876
<i>Pr</i>	0.796	0.000	0.125	0.000	0.226	0.000	0.000	0.184	0.935
<i>R</i>	0.907	0.000	0.016	0.000	0.116	0.000	0.000	0.198	0.876
<i>F₁</i>	0.848	0.000	0.028	0.000	0.153	0.000	0.000	0.191	0.904
<i>InceptionTime</i>									
<i>Acc</i>	0.894	0.135	0.175	0.000	0.204	0.000	0.500	0.354	0.915
<i>Pr</i>	0.819	0.214	0.106	0.000	0.357	0.000	0.500	0.347	0.931
<i>R</i>	0.894	0.135	0.175	0.000	0.204	0.000	0.500	0.354	0.915
<i>F₁</i>	0.855	0.166	0.132	0.000	0.259	0.000	0.500	0.351	0.923
<i>MiniRocket</i>									
<i>Acc</i>	0.983	0.000	0.000	0.000	0.071	0.000	0.000	0.042	0.920
<i>Pr</i>	0.802	0.000	0.000	0.000	0.447	0.000	0.000	0.800	0.977
<i>R</i>	0.983	0.000	0.000	0.000	0.071	0.000	0.000	0.042	0.920
<i>F₁</i>	0.883	0.000	0.000	0.000	0.122	0.000	0.000	0.079	0.948
Support	7509	222	63	2	1438	10	2	96	1592



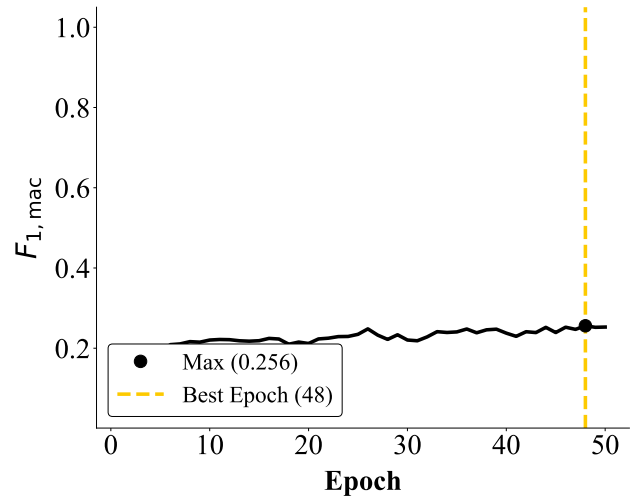
(a) DeepRange 1B - Loss and learning rate



(b) DeepRange 8B - Loss and learning rate



(c) DeepRange 1B - Validation curve



(d) DeepRange 8B - Validation curve

Figure A-20: DeepRange 1B/8B learning curves for few-shot weather classification using a sequence length of 500 s.

Note on the use of AI tools

In accordance with the guidelines of Technische Universität Darmstadt, AI tools were used to assist the work process and did not replace independent work. This work was written independently and linguistically revised with the help of ChatGPT (OpenAI), Claude (Anthropic) and DeepL. The code used in this work was written independently and partially revised with the help of ChatGPT (OpenAI) and Claude (Anthropic). The code parts revised in this way are marked with comments in the source code.

Bibliography

EU–Council: European Climate Law (2021)

EU–Council: European Climate Law, URL: <https://eur-lex.europa.eu/eli/reg/2021/1119/oj>, 2021, Accessed 26.07.2025

Aeon: Plot critical difference (2025)

Aeon: Plot critical difference, URL: https://www.aeon-toolkit.org/en/v1.2.0/api_reference/auto_generated/aeon.visualisation.plot_critical_difference.html#id2, 2025, Accessed 14.11.2025

Bagnall, A. et al.: The UEA archive (2018)

Bagnall, Anthony; Dau, Hoang Anh; Lines, Jason; Flynn, Michael; Large, James; Bostrom, Aaron; Southam, Paul; Keogh, Eamonn: The UEA multivariate time series classification archive, 2018, 2018

Bahdanau, D. et al.: Neural Machine Translation by Jointly Learning to Align and Translate (2016)

Bahdanau, Dzmitry; Cho, Kyunghyun; Bengio, Yoshua: Neural Machine Translation by Jointly Learning to Align and Translate, 2016

Bamdezh, M. et al.: Climate condition effects on EV performance (2025)

Bamdezh, M.A.; Molaeimanesh, G.R.; Mohammadi Saman Kand, R.; Mostajeran, M.: Effects of temperate, cold, and warm climate conditions on electric vehicle and battery thermal management system performance: Energy usage, battery aging, and carbon footprints, in: Energy Conversion and Management, Vol. 339, pp. 119935, 2025

Bass, L. (; Clements, P. (; Kazman, R. (; Software architecture in practice (2013)

Bass, Len (Verfasser); Clements, Paul (Verfasser); Kazman, Rick (Verfasser): Software architecture in practice, 3. edition. edition, Addison-Wesley, 2013

Beran, J.: Mathematical Foundations of Time Series (2018)

Beran, Jan: Mathematical Foundations of Time Series Analysis: A Concise Introduction, Springer International Publishing, 2018

Berecibar, M. et al.: Review of SoH estimation methods (2016)

Berecibar, M.; Gandiaga, I.; Villarreal, I.; Omar, N.; Van Mierlo, J.; Van den Bossche, P.: Critical review of state of health estimation methods of Li-ion batteries for real applications, in: Renewable and Sustainable Energy Reviews, Vol. 56, pp. 572–587, 2016

Bishop, C. M.: Pattern recognition and machine learning (2006)

Bishop, Christopher M.: Pattern recognition and machine learning, Information science and statistics, Springer New York, 2006

Borsos, Z. et al.: Language Modeling Approach to Audio Generation (2023)

Borsos, Zalán; Marinier, Raphaël; Vincent, Damien; Kharitonov, Eugene; Pietquin, Olivier; Sharifi, Matt; Roblek, Dominik; Teboul, Olivier; Grangier, David; Tagliasacchi, Marco; Zeghidour, Neil: AudioLM: A Language Modeling Approach to Audio Generation, in: IEEE/ACM Trans. Audio, Speech and Lang. Proc. Vol. 31, pp. 2523–2533, 2023

Breck, E. et al.: Data Validation for Machine Learning (2019)

Breck, Eric; Zinkevich, Marty; Polyzotis, Neoklis; Whang, Steven; Roy, Sudip: Data Validation for Machine Learning, in: Proceedings of SysML, 2019

Bridle, J.: Training Stochastic Model Recognition Algorithms (1989)

Bridle, John: Training Stochastic Model Recognition Algorithms as Networks can Lead to Maximum Mutual Information Estimation of Parameters, in: Advances in Neural Information Processing Systems, Vol. 2, pp. 211–217, 1989

Carlsson, R. et al.: Master's thesis, Weather Impact on ET Energy Consumption (2024)

Carlsson, Robert; Nordgren, Emrik: Weather Impact on Energy Consumption For Electric Trucks: Predictive modelling with Machine Learning, Dissertation Umeå University, Department of Mathematics und Mathematical Statistics, 2024

Caterini, A. L.; Chang, D. E.: Deep Neural Networks (2018)

Caterini, Anthony L.; Chang, Dong Eui: Deep Neural Networks in a Mathematical Framework, Springer-Briefs in Computer Science, Springer International Publishing, 2018

Chang, W.-Y.: Battery SoC Estimating Methods (2013)

Chang, Wen-Yeau: The State of Charge Estimating Methods for Battery: A Review, in: ISRN Applied Mathematics, Vol. 2013, pp. 1–7, 2013

Chaube, S. et al.: Multi-Criteria Decision Analysis (2024)

Chaube, Shshank; Pant, Sangeeta; Kumar, Anuj; Uniyal, Shaurya; Singh, Manoj Kumar; Kotecha, Ketan; Kumar, Akshay: An Overview of Multi-Criteria Decision Analysis and the Applications of AHP and TOPSIS Methods, in: International Journal of Mathematical, Engineering and Management Sciences, Vol. 9, pp. 581–615, 2024

Chen, X. et al.: Learning Time Series Classification (2025)

Chen, Xiwen; Zhu, Wenhui; Qiu, Peijie; Wang, Hao; Li, Huayu; LI, ZIHAN; Wang, Yalin; Sotiras, Aristeidis; Razi, Abolfazl: FIC-TSC: Learning Time Series Classification with Fisher Information Constraint, in: Forty-second International Conference on Machine Learning, 2025

Chen, Y. et al.: LLMs are few-shot multivariate time series classifiers (2025)

Chen, Yakun; Li, Zihao; Yang, Chao; Wang, Xianzhi; Xu, Guandong: LLMs are few-shot multivariate time series classifiers, in: Data Mining and Knowledge Discovery, Vol. 39, pp. 66, 2025

Chicco, D. et al.: Advantages of Matthews correlation coefficient (2020)

Chicco, Davide; Jurman, Giuseppe: The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation, in: BMC Genomics, Vol. 21, pp. 6, 2020

Corguillé, J. L. et al.: Analysis of battery electric trucks (2025)

Corguillé, Juliette Le; Hacker, Florian; Dolinga, Theresa: Real-world data analysis of battery electric trucks operating in Germany, in: 38th International Electric Vehicle Symposium and Exhibition (EVS38), 2025

Dang, C. et al.: Robust Electric Truck Range Estimation (2025)

Dang, Chuong; Siegle, William; Gehring, Ottmar; Peters, Steven: Physics-Informed Loss Function for Robust Electric Truck Range Estimation, in: 2025 IEEE Intelligent Vehicles Symposium IV, pp. 2396–2401, 2025

Dempster, A. et al.: TSC using random convolutional kernels (2020)

Dempster, Angus; Petitjean, François; Webb, Geoffrey I.: ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels, in: Data Mining and Knowledge Discovery, Vol. 34, pp. 1454–1495, 2020

Dempster, A. et al.: Fast Transform for TSC (2021)

Dempster, Angus; Schmidt, Daniel F.; Webb, Geoffrey I.: MiniRocket: A Very Fast (Almost) Deterministic Transform for Time Series Classification, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, pp. 248–257, 2021

DOE: Vehicle Weight Classes and Categories (2012)

DOE: Vehicle Weight Classes and Categories, URL: <https://afdc.energy.gov/data/10380>, 2012, Accessed 18.08.2025

Dollinger, M. et al.: Range Prediction for ETs (2021)

Dollinger, Manfred; Fischerauer, Gerhard: Model-Based Range Prediction for Electric Cars and Trucks under Real-World Conditions, in: Energies, Vol. 14, 2021

Dosovitskiy, A. et al.: Transformers for Image Recognition (2021)

Dosovitskiy, Alexey; Beyer, Lucas; Kolesnikov, Alexander; Weissenborn, Dirk; Zhai, Xiaohua; Unterthiner, Thomas; Dehghani, Mostafa; Minderer, Matthias; Heigold, Georg; Gelly, Sylvain; Uszkoreit, Jakob; Houlsby, Neil: An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, in: International Conference on Learning Representations, 2021

EAFO: EU classification of vehicle types (2025)

EAFO: EU classification of vehicle types, URL: <https://alternative-fuels-observatory.ec.europa.eu/general-information/vehicle-types>, 2025, Accessed 10.12.2025

Elfwing, S. et al.: Sigmoid-weighted linear units (2018)

Elfwing, Stefan; Uchibe, Eiji; Doya, Kenji: Sigmoid-weighted linear units for neural network function approximation in reinforcement learning, in: Neural Networks, Vol. 107, pp. 3–11, 2018

FZD: Automated Trucking Darmstadt (2025)

FZD: Automated Trucking Darmstadt, TUDa, URL: https://www.fzd.tu-darmstadt.de/forschung/research_projects_fzd/atda/ATDA.en.jsp, 2025, Accessed 27.07.2025

FZD: MINDED (2025)

FZD: MINDED, TU Darmstadt, URL: https://www.fzd.tu-darmstadt.de/forschung/research_projects_fzd/minded/standardseite_282.en.jsp, 2025, Accessed 27.07.2025

Geofabrik: OpenStreetMap Data Extracts (2025)

Geofabrik: OpenStreetMap Data Extracts, URL: <https://download.geofabrik.de/>, 2025, Accessed 04.07.2025

GeoPandas: geopandas.GeoDataFrame (2025)

GeoPandas: geopandas.GeoDataFrame, URL: <https://geopandas.org/en/latest/docs/reference/api/geopandas.GeoDataFrame.html>, 2025, Accessed 25.09.2025

Glinz, M.: On Non-Functional Requirements (2007)

Glinz, Martin: On Non-Functional Requirements, in: 15th IEEE International Requirements Engineering Conference (RE 2007), pp. 21–26, 2007

Goodfellow, I.; Bengio, Y.; Courville, A.: Deep Learning (2016)

Goodfellow, Ian; Bengio, Yoshua; Courville, Aaron: Deep Learning, MIT Press, 2016

Grandini, M. et al.: Metrics for Multi-Class Classification (2020)

Grandini, Margherita; Bagli, Enrico; Visani, Giorgio: Metrics for Multi-Class Classification: an Overview, 2020

Gruner, R. et al.: Driving scenarios and classification (2017)

Gruner, Richard; Henzler, Philip; Hinz, Gereon; Eckstein, Corinna; Knoll, Alois: Spatiotemporal representation of driving scenarios and classification using neural networks, in: 2017 IEEE Intelligent Vehicles Symposium (IV), pp. 1782–1788, 2017

Hazelwood, K. et al.: Applied Machine Learning at Facebook (2018)

Hazelwood, Kim; Bird, Sarah; Brooks, David; Chintala, Soumith; Diril, Utku; Dzhulgakov, Dmytro; Fawzy, Mohamed; Jia, Bill; Jia, Yangqing; Kalro, Aditya; Law, James; Lee, Kevin; Lu, Jason; Noordhuis, Pieter; Smelyanskiy, Misha; Xiong, Liang; Wang, Xiaodong: Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective, in: 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA), pp. 620–629, 2018

Hecht, Y.: Master thesis (confidential), Filling Data Gaps with GenAI (2025)

Hecht, Yanik: Filling Data Gaps with Generative AI for E-Mobility Range Prediction, Dissertation Technical University of Munich, 2025

Hendrycks, D. et al.: Gaussian Error Linear Units (2016)

Hendrycks, Dan; Gimpel, Kevin: Gaussian Error Linear Units (GELUs), in: arXiv e-prints, pp. arXiv:1606.08415, 2016

Irani, H. et al.: Time Series Embeddings for Classification (2025)

Irani, Habib; Ghahremani, Yasamin; Kermani, Arshia; Metsis, Vangelis: Time Series Embedding Methods for Classification Tasks: A Review, in: Expert Systems, Vol. 42, pp. e70148, 2025

Ishizaka, A.; Nemery, P.: Multi-criteria Decision Analysis (2013)

Ishizaka, Alessio; Nemery, Philippe: Multi-criteria Decision Analysis: Methods and Software, John Wiley & Sons, Ltd, 2013

Ismail Fawaz, H. et al.: AlexNet for time series classification (2020)

Ismail Fawaz, Hassan; Lucas, Benjamin; Forestier, Germain; Pelletier, Charlotte; Schmidt, Daniel F.; Weber, Jonathan; Webb, Geoffrey I.; Idoumghar, Lhassane; Muller, Pierre-Alain; Petitjean, François: InceptionTime: Finding AlexNet for time series classification, in: Data Mining and Knowledge Discovery, Vol. 34, pp. 1936–1962, 2020

ISO: 6469-1:2019-04 - Electrically propelled road vehicles (2019)

ISO: 6469-1:2019-04 - Electrically propelled road vehicles - Safety specifications - Part 1: Rechargeable energy storage system (RESS), 2019

Jin, M. et al.: Reprogramming Large Language Models (2024)

Jin, Ming; Wang, Shiyu; Ma, Lintao; Chu, Zhixuan; Zhang, James Y.; Shi, Xiaoming; Chen, Pin-Yu; Liang, Yuxuan; Li, Yuan-Fang; Pan, Shirui; Wen, Qingsong: Time-LLM: Time Series Forecasting by Reprogramming Large Language Models, in: The Twelfth International Conference on Learning Representations, 2024

Kim, T. et al.: Reversible Instance Normalization (2022)

Kim, Taesung; Kim, Jinhee; Tae, Yunwon; Park, Cheonbok; Choi, Jang-Ho; Choo, Jaegul: Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift, in: International Conference on Learning Representations, 2022

Liimatainen, H. et al.: The potential of electric trucks (2019)

Liimatainen, Heikki; Van Vliet, Oscar; Aplyn, David: The potential of electric trucks – An international commodity-level analysis, in: Applied Energy, Vol. 236, pp. 804–814, 2019

Liu, X. et al.: UniTime (2024)

Liu, Xu; Hu, Junfeng; Li, Yuan; Diao, Shizhe; Liang, Yuxuan; Hooi, Bryan; Zimmermann, Roger: UniTime: A Language-Empowered Unified Model for Cross-Domain Time Series Forecasting, URL: <http://arxiv.org/abs/2310.09751>, 2024, Accessed 31.07.2025

Maas, A. et al.: Rectifier Nonlinearities Improve Neural Network Acoustic Models (2013)

Maas, A.L.; Hannun, A.Y.; Ng, A.Y.: Rectifier Nonlinearities Improve Neural Network Acoustic Models, in: Proceedings of the 30th International Conference on Machine Learning, 2013

Matthews, B. W.: Comparison of T4 phage lysozyme (1975)

Matthews, Brian W.: Comparison of the predicted and observed secondary structure of T4 phage lysozyme, in: Biochimica et Biophysica Acta (BBA) - Protein Structure, Vol. 405, pp. 442–451, 1975

Meteostat: The Weather's Record Keeper (2025)

Meteostat: The Weather's Record Keeper, URL: <https://meteostat.net/en/>, 2025, Accessed 16.09.2025

Mielke, S. J. et al.: Tokenization in NLP (2021)

Mielke, Sabrina J.; Alyafeai, Zaid; Salesky, Elizabeth; Raffel, Colin; Dey, Manan; Gallé, Matthias; Raja, Arun; Si, Chenglei; Lee, Wilson Y.; Sagot, Benoît; Tan, Samson: Between words and characters: A Brief History of Open-Vocabulary Modeling and Tokenization in NLP, 2021

Mikolov, T. et al.: Word Representations in Vector Space (2013)

Mikolov, Tomas; Chen, Kai; Corrado, Gregory S.; Dean, Jeffrey: Efficient Estimation of Word Representations in Vector Space, in: International Conference on Learning Representations, 2013

Mohammadi Foumani, N. et al.: Deep Learning for Time Series (2024)

Mohammadi Foumani, Navid; Miller, Lynn; Tan, Chang Wei; Webb, Geoffrey I.; Forestier, Germain; Salehi, Mahsa: Deep Learning for Time Series, in: ACM Comput. Surv. Vol. 56, 2024

Mruzek, M. et al.: Parameters Influencing EV Range (2016)

Mruzek, Martin; Gajdáč, Igor; Kučera, Luboš; Barta, Dalibor: Analysis of Parameters Influencing Electric Vehicle Range, in: Procedia Engineering, Vol. 134, pp. 165–174, 2016

Nair, V. et al.: Rectified linear units (2010)

Nair, Vinod; Hinton, Geoffrey E.: Rectified linear units improve restricted boltzmann machines, in: Proceedings of the 27th International Conference on International Conference on Machine Learning, pp. 807–814, 2010

Nematirad, R. et al.: Data Embeddings in Time Series Forecasting (2025)

Nematirad, Reza; Pahwa, Anil; Natarajan, Balasubramaniam: Are Data Embeddings Effective in Time Series Forecasting?, in: Transactions on Machine Learning Research, 2025

Nie, Y. et al.: Long-term Forecasting with Transformers (2023)

Nie, Yuqi; Nguyen, Nam H.; Sinthong, Phanwadee; Kalagnanam, Jayant: A Time Series is Worth 64 Words: Long-term Forecasting with Transformers, in: The Eleventh International Conference on Learning Representations, 2023

Olson, D. L.: Smart (1996)

Olson, David L.: Smart, in: Olson, David L. (Ed.): Decision Aids for Selection Problems. Springer New York, 1996

OpenStreetMap: Key:highway (2025)

OpenStreetMap: Key:highway, URL: <https://wiki.openstreetmap.org/wiki/Key:highway>, 2025, Accessed 05. 07. 2025

OpenStreetMap: OpenStreetMap (2025)

OpenStreetMap: OpenStreetMap, URL: <https://www.openstreetmap.org>, 2025, Accessed 03. 07. 2025

OpenStreetMap: Overpass API (2025)

OpenStreetMap: Overpass API, URL: https://wiki.openstreetmap.org/wiki/Overpass_API, 2025, Accessed 04. 07. 2025

OSMnx: OSMnx (2025)

OSMnx: OSMnx, URL: <https://osmnx.readthedocs.io/en/stable/>, 2025, Accessed 04. 07. 2025

Ouyang, L. et al.: Training language models (2022)

Ouyang, Long; Wu, Jeffrey; Jiang, Xu; Almeida, Diogo; Wainwright, Carroll; Mishkin, Pamela; Zhang, Chong; Agarwal, Sandhini; Slama, Katarina; Ray, Alex; Schulman, John; Hilton, Jacob; Kelton, Fraser; Miller, Luke; Simens, Maddie; Askell, Amanda; Welinder, Peter; Christiano, Paul F; Leike, Jan; Lowe, Ryan: Training language models to follow instructions with human feedback, in: Advances in Neural Information Processing Systems, Vol. 35, pp. 27730–27744, 2022

Pyogrio: About (2025)

Pyogrio: About, URL: <https://pyogrio.readthedocs.io/en/latest/about.html>, 2025, Accessed 05. 07. 2025

Qiu, Y. et al.: Medium- and Heavy-Duty EVs (2023)

Qiu, Yin; Dobbelaere, Cristina; Song, Shuhan: Energy Cost Analysis and Operational Range Prediction Based on Medium- and Heavy-Duty Electric Vehicle Real-World Deployments across the United States, in: World Electric Vehicle Journal, Vol. 14, 2023

Radford, A. et al.: Improving Language Understanding by Generative Pre-Training (2018)

Radford, Alec; Narasimhan, Karthik; Salimans, Tim; Sutskever, Ilya: Improving Language Understanding by Generative Pre-Training, 2018

Radford, A. et al.: Language Models are Unsupervised Multitask Learners (2019)

Radford, Alec; Wu, Jeff; Child, Rewon; Luan, David; Amodei, Dario; Sutskever, Ilya: Language Models are Unsupervised Multitask Learners, 2019

Rainio, O. et al.: Evaluation metrics for machine learning (2024)

Rainio, Oona; Teuvo, Jarmo; Klén, Riku: Evaluation metrics and statistical tests for machine learning, in: Scientific Reports, Vol. 14, pp. 6086, 2024

Raschka, S.: Build A Large Language Model (2024)

Raschka, Sebastian: Build A Large Language Model (From Scratch), Manning, 2024

SAE International: J2910_201404 - Design and Test of ETs (2014)

SAE International: J2910_201404 - Recommended Practice for the Design and Test of Hybrid Electric and Electric Trucks and Buses for Electrical Safety, 2014

Schmidt, R. M.: Introduction to RNNs (2019)

Schmidt, Robin M.: Recurrent Neural Networks (RNNs): A gentle Introduction and Overview, URL: <https://arxiv.org/abs/1912.05911>, 2019

Siegle, W.: Master thesis (confidential), Range Prediction with PINNs (2025)

Siegle, William: Range Prediction for Electric Commercial Vehicles with Physics-Informed Neural Networks, Dissertation University of Stuttgart, 2025

Smodej, P. et al.: 3000 kilometers with eActros 300 (2023)

Smodej, Peter; Burkhart, Ulrike: Around 3,000 kilometers covered completely electrically: eActros 300 as a tractor unit glides from the Arctic Circle to Stuttgart, Daimler Truck, URL: <https://www.daimlertruck.com/en/newsroom/pressrelease/around-3000-kilometers-covered-completely-electrically-eactros-300-as-a-tractor-unit-glides-from-the-arctic-circle-to-stuttgart-52192677>, 2023, Accessed 14.07.2025

Smodej, P. et al.: eActros 400 and 600 (2025)

Smodej, Peter; Burkhart, Ulrike; Fischer, Katrin: Mercedes-Benz Trucks presents the new eActros 400 based on the eActros 600 - numerous new variants available, Daimler Truck, URL: <https://www.daimlertruck.com/en/newsroom/pressrelease/mercedes-benz-trucks-presents-the-new-eactros-400-based-on-the-eactros-600-numerous-new-variants-available-53205175>, 2025, Accessed 23.10.2025

Smodej, P. et al.: Mercedes-Benz Battery Technology Center (2024)

Smodej, Peter; Freiberg, Dorothea; Minne, Arnd: Battery Technology Center officially opened at the Mercedes-Benz plant in Mannheim - series assembly of the next battery generation at the site, Daimler Truck, URL: <https://www.daimlertruck.com/en/newsroom/pressrelease/battery-technology-center-officially-opened-at-the-mercedes-benz-plant-in-mannheim-series-assembly-of-the-next-battery-generation-at-the-site-52770559>, 2024, Accessed 20.07.2025

Smuts, M. et al.: Factors influencing remaining driving range (2017)

Smuts, Martin; Scholtz, Brenda; Wesson, Janet: A critical review of factors influencing the remaining driving range of electric vehicles, in: 2017 1st International Conference on Next Generation Computing Applications (NextComp), pp. 196–201, 2017

Studer, S. et al.: Towards CRISP-ML(Q) (2021)

Studer, Stefan; Bui, Thanh Binh; Drescher, Christian; Hanuschkin, Alexander; Winkler, Ludwig; Peters, Steven; Müller, Klaus-Robert: Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology, in: MAKE, Vol. 3, pp. 392–413, 2021

Taherdoost, H. et al.: Using SMART Method for MCDA (2024)

Taherdoost, Hamed; Mohebi, Atefeh: Using SMART Method for Multi-criteria Decision Making: Applications, Advantages, and Limitations, in: Archives of Advanced Engineering Science, Vol. 2, pp. 190–197, 2024

Tang, X. et al.: The pace of artificial intelligence (2020)

Tang, Xuli; Li, Xin; Ding, Ying; Song, Min; Bu, Yi: The pace of artificial intelligence innovations: Speed, talent, and trial-and-error, in: Journal of Informetrics, Vol. 14, pp. 101094, 2020

UNFCCC: Paris Agreement (2016)

UNFCCC: Paris Agreement, URL: https://unfccc.int/sites/default/files/english_paris_agreement.pdf, 2016, Accessed 24. 07. 2025

Varga, B. O. et al.: Prediction of EV Range (2019)

Varga, Bogdan Ovidiu; Sagoian, Arsen; Mariasiu, Florin: Prediction of Electric Vehicle Range: A Comprehensive Review of Current Issues and Challenges, in: Energies, Vol. 12, pp. 946, 2019

Vaswani, A. et al.: Attention is all you need (2017)

Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N.; Kaiser, Łukasz; Polosukhin, Illia: Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, pp. 6000–6010, 2017

Wu, H. et al.: Decomposition Transformers with Auto-Correlation (2021)

Wu, Haixu; Xu, Jiehui; Wang, Jianmin; Long, Mingsheng: Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting, in: Advances in Neural Information Processing Systems, Vol. 34, pp. 22419–22430, 2021

Zaharia, M. A. et al.: Accelerating Machine Learning with MLflow (2018)

Zaharia, Matei A.; Chen, Andrew; Davidson, Aaron; Ghodsi, Ali; Hong, Sue Ann; Konwinski, Andy; Murching, Siddharth; Nykodym, Tomas; Ogilvie, Paul; Parkhe, Mani; Xie, Fen; Zumar, Corey: Accelerating the Machine Learning Lifecycle with MLflow, in: IEEE Data Eng. Bull. Vol. 41, pp. 39–45, 2018

Zeng, A. et al.: Are Transformers Effective for Time Series? (2023)

Zeng, Ailing; Chen, Muxi; Zhang, Lei; Xu, Qiang: Are Transformers Effective for Time Series Forecasting?, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, pp. 11121–11128, 2023

Zerveas, G. et al.: Transformer for Time Series Representation (2021)

Zerveas, George; Jayaraman, Srideepika; Patel, Dhaval; Bhamidipaty, Anuradha; Eickhoff, Carsten: A Transformer-based Framework for Multivariate Time Series Representation Learning, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, pp. 2114–2124, 2021

Zhang, X. et al.: Large Language Models for Time Series (2024)

Zhang, Xiyuan; Chowdhury, Ranak Roy; Gupta, Rajesh K.; Shang, Jingbo: Large Language Models for Time Series: A Survey, in: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, pp. 8335–8343, 2024

Zhou, H. et al.: Transformer for Long Sequence Time-Series (2021)

Zhou, Haoyi; Zhang, Shanghang; Peng, Jieqi; Zhang, Shuai; Li, Jianxin; Xiong, Hui; Zhang, Wancai: Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, pp. 11106–11115, 2021

Zhou, T. et al.: Frequency Enhanced Transformer for Time Series (2022)

Zhou, Tian; Ma, Ziqing; Wen, Qingsong; Wang, Xue; Sun, Liang; Jin, Rong: FEDformer: Frequency Enhanced Decomposed Transformer for Long-term Series Forecasting, in: Proceedings of the 39th International Conference on Machine Learning, Vol. 162, pp. 27268–27286, 2022

Zhou, T. et al.: Time series analysis by pretrained LM (2023)

Zhou, Tian; Niu, Peisong; Wang, Xue; Sun, Liang; Jin, Rong: One fits all: power general time series analysis by pretrained LM, in: Proceedings of the 37th International Conference on Neural Information Processing Systems, 2023

Zyoud, S. H. et al.: AHP and TOPSIS techniques (2017)

Zyoud, Shaher H.; Fuchs-Hanusch, Daniela: A bibliometric-based survey on AHP and TOPSIS techniques, in: Expert Systems with Applications, Vol. 78, pp. 158–181, 2017